

—— // POWERLINERS // ——

powerliners

v0.2.15

THE COMPANION

the segments · the daemon · the wire protocol

2026-07-18

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

Before You Begin	3
Chapter 1 — The Pitch	4
Chapter 2 — The Segment Model	7
Chapter 3 — The Built-in Segment Library	11
Chapter 4 — The Daemon and the Wire Protocol	16
Chapter 5 — The Performance Story	21
Chapter 6 — Themes and Colorschemes	24
Chapter 7 — Renderers	29
Chapter 8 — Bindings	33
Chapter 9 — Fidelity to Upstream	36
Chapter 10 — The Port Methodology	38
Chapter 11 — Packaging and Deployment	41
Chapter 12 — Status and the Honest Gaps	43
Chapter 13 — Where powerliners Fits	46
Appendix A — The Full Segment Catalog	47
Appendix B — The Config Schema Reference	53
Appendix C — The Color and Highlight-Group Reference	56
Appendix D — The Renderer Reference	59
Appendix E — The Daemon Protocol and Lifecycle Reference	62
Appendix F — The Parity Harness, the Tiers, and the Bug Log	65
Appendix G — Performance Measurement Methodology	67
Appendix H — A Complete Worked Example: From Config to Bytes	69
Appendix I — The Bindings Cookbook	71
Appendix J — The Custom-Segment Cookbook	74
Appendix K — Listers and <code>segment_list</code> Expansion	76

Appendix L — The Five Binaries in Detail	78
Appendix M — The Migration and Rollback Runbook	80
Appendix N — The Config Cascade, Worked	82
Appendix O — Glossary	84
Appendix P — The Truncation Algorithm, Worked	85
Appendix Q — Platform Notes: macOS, Linux, and the BSDs	87
Appendix R — The Extension Provenance Map	89
Appendix S — The Test Suite Map	91
Appendix T — The <code>segment_info</code> Reference	92
Appendix U — Worked Segment Outputs	94
Appendix V — The <code>tmux</code> Conf Files and Version Dispatch	96
Appendix W — Design-Rationale FAQ	97
Appendix X — The Divider Math, Worked	98
Appendix Y — Upstream Lineage and the Vendor Fixture	100
Appendix Z — Man-Page Cross-Reference	101
Appendix AA — Reading the Bundled Shell Theme	102
Appendix AB — Common Pitfalls and Their Fixes	104
Appendix AC — Quick-Reference Tables	106
Appendix AD — The <code>mergedicts</code> Deep-Merge, Worked	109
Appendix AE — The Divider Glyph Cascade and the Fallback Chain	111
Appendix AF — The Mode Machine, End to End	113
Appendix AG — Signal Handling and Async-Signal-Safety in the Daemon	115
Appendix AH — The <code>poll</code> Loop and What Changed from <code>select.select</code>	117
Appendix AI — The <code>powerline-lint</code> Check Catalog	119
Appendix AJ — The <code>segment_info</code> Environment Contract	121
Appendix AK — Building, Installing, and Cross-Compiling the Suite	123
Appendix AL — The Cost of Absence: None-Returning Segments	125
Appendix AM — The Reproducibility Ledger	127

Before You Begin

This is the book about powerliners — the one you read in a chair, front to back, to understand not only *what* powerliners does but *why* it is shaped the way it is. It is not a man page. The man pages — `powerline(1)`, `powerline-daemon(1)`, `powerline-config(1)`, `powerline-render(1)`, and `powerline-lint(1)` — enumerate every flag and every subcommand, and you reach for them when you need one fact fast. This book is the other thing: the continuous account of the program, from the keystroke that triggers a prompt redraw to the `#[fg=...,bg=...]...` markup that lands in your statusline, and of the design decisions that let a Rust binary drop straight into a configuration written for a Python program.

powerliners is a Rust port of `powerline-status`, the canonical statusline and prompt renderer used in tmux, zsh, bash, vim, and the ipython REPL. The Python implementation pays a ~50–150 ms interpreter-startup tax on every render — every prompt redraw, every tmux statusline refresh, every window switch. On an interactive shell that redraws after keystrokes, that tax compounds into visible lag. powerliners is a single static binary: zero import cost, no garbage collector, sub-millisecond render, and — this is the point that governs the whole design — **drop-in compatible with the existing `~/.config/powerline` JSON theme and segment files**, so users keep their themes verbatim.

This book is honest about its foundation, because the honesty is the point. powerliners does not invent a statusline format; it reimplements one. The segment model, the theme grammar, the colorscheme cascade, and the daemon wire protocol are all inherited from upstream powerline, documented here faithfully and completely. The original work — the work that makes powerliners a MenkeTechnologies product rather than a rebuild — is the *engineering* of that port: a multi-binary suite of native clients, a UNIX-socket daemon that speaks the upstream C client's wire format, and a parity harness that runs the real Python interpreter on the upstream source and asserts byte-identical output.

Everything here is grounded in the program. Every JSON snippet is config the renderer actually parses; every segment named is one of the 54 adapters wired into the render runtime; every claim about the daemon is grounded in the running binary. Where a number or a behaviour is provisional this book marks it rather than rounding up: powerliners is at version `0.2.15` at the time of writing, with 134 of 137 upstream files at the DONE tier, 2473 library tests, and 462 parity assertions run against the live Python interpreter. powerliners is the work of MenkeTechnologies, built on the foundation of the powerline authors before him, and this book is grounded in that code.

Let's start.

Chapter 1 – The Pitch

A prompt is, on paper, a solved problem: shells have printed one for fifty years. Powerline made it beautiful — segmented, colored, divider-chevrons, aware of your git branch and your Kubernetes context — and it did so in Python, which was the right call for a configurable, plugin-rich renderer and the wrong call for something invoked on every keystroke. This chapter states the problem powerliners exists to solve and the one constraint that shaped every decision that follows.

The whole argument is visible in one measurement. The Python renderer pays its interpreter-startup cost on every invocation; the daemon pays it once:

```
# Python powerline: interpreter + imports on every prompt redraw
$ time powerline shell left           # ~100 ms, dominated by startup

# powerliners: a static binary talking to a warm daemon over a UNIX socket
$ time powerline shell left           # sub-millisecond after first render
```

The Python startup tax

The tax is not a bug in powerline; it is the unavoidable cost of the runtime it runs on. When a shell renders its prompt, it forks a process, and that process must exist, do its work, and exit before the prompt can print. For a native binary “exist” means the kernel maps a static executable into memory and jumps to `main`, which is microseconds. For a Python program “exist” means starting the CPython interpreter, initializing its object system, and then importing the powerline package — `powerline`, `powerline.renderers`, `powerline.segments`, the color and theme machinery, and whatever `psutil`, `netifaces`, or `vcs` libraries the active segments pull in. Each import is a filesystem walk, a bytecode compile-or-load, and a module-init pass. On a warm page cache this is tens of milliseconds; cold, or on a network filesystem, or under a slow disk, it is well over a hundred. The powerliners README pins the range at ~50–150 ms per render, and that range matches what anyone who has profiled a Python powerline setup has seen.

Crucially, none of that startup work is the *render*. Resolving your git branch, formatting the time, and packing a colored segment into tmux markup is cheap — microseconds of actual computation. The interpreter startup is pure overhead paid before the useful work begins, and it is paid in full every single time, because each prompt render is a fresh process with no memory of the last one. You cannot amortize it within the Python model because the process that paid it has already exited by the time the next prompt needs rendering. This is the structural fact that makes a daemon the answer rather than a faster interpreter or a leaner import graph: the only way to stop paying startup on every render is to stop starting up on every render.

powerliners attacks the tax from both ends. The direct renderer, `powerline-render`, is a native binary, so even its cold path — used as the daemon-less fallback — is roughly ~30 ms per call rather than a hundred, because there is no interpreter to boot. And the warm path, the daemon, removes startup from the equation entirely: the config tree is parsed once, the segment adapters are instantiated once, the color palettes live in memory, and each render is a socket round-trip plus the microseconds of the render itself. The measurement in the code fence above is the whole pitch compressed to two lines — same command, same arguments, same output, two orders of magnitude apart in latency, and the difference is entirely startup.

Why this matters per-keystroke, not per-session

If a prompt rendered once per shell session, a hundred milliseconds would be invisible and this book would not exist. It renders far more often than that, and the frequency is what turns a tolerable per-invocation cost into a felt one. In zsh, any `precmd` hook fires before each prompt is drawn, and prompts are redrawn not only after a command completes but on many interactive events; with a right-hand prompt (`R_PROMPT`) that also updates, a single command line can trigger several renders. In tmux the statusline refreshes on a timer — `status-interval`, often every one to five seconds — and it refreshes *per window*, so a session with eight windows multiplies the render count by eight on every tick. In vim the statusline redraws on mode changes, buffer switches, window focus changes, and on old vim versions on every cursor move. The pattern is the same everywhere: the prompt is not a one-time cost, it is a recurring one, and recurring costs are where latency becomes lag.

The per-keystroke case is the sharpest. A shell that redraws its prompt in response to typing — because a `precmd` or a continuation prompt or a completion UI triggers a render — pays the startup tax on the critical path between your keystroke and the screen updating. A hundred milliseconds there is not a background annoyance; it is the difference between a shell that feels instant and one that feels like a slideshow, in the README's phrase. Humans notice input latency above roughly the tens-of-milliseconds mark, and a hundred-millisecond render blows past it. This is the workload powerliners is tuned for, and it is why the daemon is the centerpiece: the goal is not to make one render fast, it is to make the ten-thousandth render of the day as fast as the first.

The multiplication is what makes the numbers stark. A developer with a tmux session of a dozen windows, a `status-interval` of two seconds, and a zsh prompt that redraws on interaction is triggering renders continuously through an eight-hour day. At a hundred milliseconds each that is real wall-clock time and real CPU spent booting an interpreter to do microseconds of work, over and over. At sub-millisecond each it disappears into the noise. powerliners does not make your prompt prettier — it renders the exact same prompt — it makes the prompt stop costing you anything to draw.

The one rule: keep the user's themes verbatim

There is exactly one constraint that outranks performance in powerliners' design, and it is compatibility. A faster prompt renderer that required you to rewrite your themes would be a different program that you would have to learn, migrate to, and maintain; the whole value of powerline is the ecosystem of themes, colorschemes, and segment configurations that people have accumulated and tuned over years. So the governing rule is stated plainly in the README and honored throughout the code: powerliners loads the user's existing `~/.config/powerline` JSON tree and resolves it *exactly* as upstream does. Your `config.json`, your `themes/`, your `colorschemes/`, your `colors.json` — all of it works unchanged, byte-for-byte.

This is not a compatibility *layer* bolted onto a new design; it is the design. The config cascade (`_find_config_files`, `load_json_config`, `mergedicts`) is a port of powerline's own resolution, so the same file in the same place merges the same way. The colorscheme resolution — alias chasing, cterm-to-hex, gradient picking — is a port of powerline's `Colorscheme` class, so a highlight group resolves to the same colors. The theme grammar — `segments.left` / `segments.right`, `extends` inheritance, `segment_data` overrides, per-mode `mode_translations` — is powerline's grammar. Even the segment *names* are the same, so a theme that names `cpu_load_percent` or `branch` finds an adapter of that name in powerliners' registry.

The payoff is a migration that touches nothing you have written. The upstream C client shipped with `pip install powerline-status` talks to powerliners' daemon unchanged, because they speak the same wire protocol; you can leave the Python client in `$PATH` and only swap the daemon. Your `~/.tmux.conf` invocations (`run-shell "powerline-config tmux setup"`) work as-is. Your `.vimrc` sources the same kind of plugin. There are no config edits, no `.tmux.conf` edits, no shell-rc edits — Chapter 11 walks the whole swap, and Chapter 9 is candid about the two or three places where output can legitimately differ (live-sampled numeric values, mainly), which are exactly the places where “identical” was never physically possible.

What “a port,” not “an inspired-by,” commits us to

Calling powerliners a *port* rather than a reimplementation-in-the-spirit-of is a commitment, and the project takes the word seriously. An “inspired-by” statusline would be free to make its own choices — a nicer config format, a cleaner segment API, different divider math — and would ask users to adopt them. A port is not free in that way. It is bound to reproduce the original’s behavior, including the behavior nobody would design on purpose: the exact merge semantics of overlapping config files, the exact fallback order when a highlight group is missing, the exact way `#` is doubled to `##[ ]` in tmux markup, the exact five-tuple shape that `gen_segment_getter` returns. Fidelity to those details is the product.

The commitment shows up structurally in the source tree. Ported code lives under `src/ported/`, mirroring upstream’s module layout — `src/ported/segment.rs` is the port of `powerline/segment.py`, `src/ported/colourscheme.rs` of `powerline/colourscheme.py`, `src/ported/renderers/tmux.rs` of the tmux renderer, and so on down to the scripts under `src/ported/scripts/`. Every function body carries `// py:NNN` citations pinning each line to the upstream source line it ports (Chapter 10), and a drift gate asserts that every ported function name matches `docs/powerline_py_functions.txt`. This is what “a faithful port” means in practice: not a vibe, but a mechanically-checked correspondence between two source trees.

The port also commits us to a boundary, and drawing it honestly is part of the deal. The behavior powerliners inherits is powerline’s; the segments that are *new* — `git_status`, `ci_status`, the cloud-context segments, the cache-stat segments for the sibling MenkeTechnologies tools — live under `src/extensions/`, explicitly outside the ported tree, and this book keeps them separate (Chapter 3 and Chapter 12). The MenkeTechnologies project bar kills “X but faster” and “X but in Rust” as products; powerliners survives that bar not by pretending the segment model is novel but because the *engineering* — the daemon, the wire-compatible client suite, the parity harness, and the new native segments — is the original work built on a faithfully-ported foundation.

Chapter 2 – The Segment Model

A powerline line is a list of *segments*: independent producers of a `{contents, highlight_group}` pair, laid out left-to-right (or split across a center gap), joined by hard and soft dividers, and padded. Everything visible is a segment. This chapter defines the segment as powerliners models it and walks the lifecycle of one from configuration to rendered cell.

Segments, highlight groups, and dividers

A segment is the atom of a powerline line. Conceptually it is a small function of the environment that returns a piece of text plus a name telling the renderer what color to paint it: the `hostname` segment returns your host name and asks for the `hostname` highlight group; the `branch` segment returns your git branch and asks for `branch`. The renderer never decides colors itself — it asks the colorscheme to resolve the segment's highlight group into concrete foreground, background, and attribute values (Chapter 6). This separation is why the same theme renders differently under different colorschemes: the segment says *what* and *which group*, the colorscheme says *what color that group is*, and the renderer says *how to encode that color for this target*.

A highlight group is just a name — `information:regular`, `branch_dirty`, `cpu_load_gradient` — that both the segment and the colorscheme agree on. A segment can offer several groups in priority order, and the colorscheme resolution walks them until one matches (Chapter 6's `get_highlighting`), which is how a segment degrades gracefully: a git segment might ask for `branch_dirty` first and fall back to `branch`, so a colorscheme that never defined `branch_dirty` still paints the branch in a sensible color rather than erroring. Some segments also carry a `gradient_level` — a number from 0 to 100 — that selects a color from a gradient list rather than a flat color, which is how `cpu_load_percent` shades from green through red as load climbs.

Between segments sit *dividers*, and the divider is where powerline's signature look comes from. Every segment pair is joined by either a **hard** divider or a **soft** divider, and which one is used depends on color: if two adjacent segments share a background, the boundary between them is soft (a thin separator drawn in a muted color); if their backgrounds differ, the boundary is hard (the solid chevron *'that fills with the neighboring background, producing the arrow effect'*). The renderer computes this per boundary — `divider_type = 'soft'` if the backgrounds match else `'hard'` in `render_length` — so the theme author does not place dividers manually; they fall out of the coloring. Around the whole line sits *outer padding* and per-segment *spaces*, small amounts of blank cells that keep the text from crowding the terminal edges and the dividers.

The three segment kinds: string, filler, and segment-list

Not every segment runs a function. powerliners, following upstream, recognizes three segment *types*, and `process_segment` (at `src/ported/segment.rs:692`) dispatches on the `type` field of the segment's JSON. The check is explicit: `process_segment` handles `"function"` and `"segment_list"` together at the top, and treats everything else — chiefly `"string"` — as a literal.

A **string** segment is a constant. Its `contents` come straight from the theme JSON, not from any function; it exists so a theme can drop a literal label, a fixed icon, or a separator into the line. In `gen_segment_getter` the string case stamps `type = "string"` and carries the literal contents forward. A **function** segment is the interesting kind: its `function` field names a segment adapter (by bare name like `hostname`, or by dotted path

like `powerliners.k8s.kubecontext`), and at render time that adapter runs against the environment and returns the contents. The vast majority of visible segments are function segments; the whole built-in library of Chapter 3 is function adapters. A **segment_list** (or “filler” / lister) segment is a segment that *expands into more segments*: it runs a lister (Chapter 7’s i3 workspaces, for example) that produces a variable number of sub-segments, each processed in turn. `process_segment_list` (at `src/ported/segment.rs:578`) handles that expansion.

The three types share one output shape but reach it differently, and `gen_segment_getter` is the function that normalizes them. It validates that a segment’s declared type is one of `"function" | "string" | "segment_list"` (it rejects anything else), and it produces, for each, the resolved four-or-five-tuple that the render loop consumes. The filler/lister case is what makes powerline able to render “one segment per tmux window” or “one segment per i3 workspace” without the theme knowing how many there will be — the count is discovered at render time and the lister fans it out.

Each kind is easiest to recognize from a one-line theme entry. A **string** segment is a literal label — `{ "type": "string", "contents": " | ", "highlight_groups": ["separator"] }` — whose `contents` come straight from the JSON, never from an adapter; `gen_segment_getter` stamps `type = "string"` and carries the literal forward, and at render time `process_segment` takes the `string` branch and uses the contents verbatim. It is how a theme drops a fixed icon, a label, or a divider glyph into a line without a function. A **function** segment — `{ "function": "cpu_load_percent" }` — is the common kind: its `function` names an adapter that runs against the environment at render time. A **segment_list** segment — `{ "type": "segment_list", "function": "workspace_lister", "segments": [ ... ] }` — carries a lister and a `segments` template, and expands into one instance of the template per item the lister yields (Appendix K). The key distinction is *when the contents exist*: a string segment’s contents exist at load time (they are in the JSON), a function segment’s exist at render time (the adapter produces them), and a segment_list’s `count` exists only at render time (the lister discovers it). All three converge on the same `{contents, highlight}` record the renderer consumes; they differ only in how that record is reached.

gen_segment_getter and the segment table

`gen_segment_getter` (at `src/ported/segment.rs:934`) is the factory that turns a segment’s JSON spec into a resolved, ready-to-run getter, and it is the hinge of the whole model. Its job is to take the raw segment dictionary from the theme — which might say `{ "function": "cpu_load_percent", "type": "function", "args": { ... } }` — and resolve it into a concrete record: the resolved function, the module it came from, the segment’s name, and (for string segments) its literal contents. It mirrors upstream’s `gen_segment_getter` line for line, including validating the type and routing string segments through the `get_string` path and function / segment_list segments through the `get_function` path.

The resolution of *which function* a segment names is `get_function` (also in `segment.rs`), and its output is the `SegmentGetterResult` struct — the direct port of the Python five-tuple. Its fields are worth naming exactly, because they recur throughout the render path:

```
pub struct SegmentGetterResult {
    pub contents: Option<String>,           // literal contents (string segments)
    pub function_name: Option<String>,      // resolved function name
    pub module: Option<String>,            // resolved module name
    pub function_name_dup: Option<String>,  // duplicate of function_name (py tuple shape)
    pub name: Option<String>,              // segment.get('name')
}
```

The fourth field, `function_name_dup`, is a duplicate of the second, and it exists for exactly one reason: the Python original returns a five-tuple in which the function name appears twice, and a faithful port preserves the shape even where it is redundant (the field’s own doccomment says “preserved for parity with py:70”). This is the port discipline of Chapter 1 made concrete — powerliners does not “clean up” the upstream tuple, because cleaning it up would be a behavioral change, and the goal is byte-and-structure fidelity, not a nicer API.

The “segment table” is where names become adapters. `get_function` resolves the dotted `function` string

to a `(module, function_name)` pair by `rpartition` on the last `.` — `powerliners.k8s.kubecontext` splits into module `powerliners.k8s` and function `kubecontext`, while a bare `hostname` falls back to the default module. That resolved name is then looked up in the `ADAPTERS` registry (Chapter 3) at render time. Upstream did this lookup by Python's dynamic `__import__`; `powerliners` cannot import Python modules, so the dotted path is resolved against a static registry instead — same names, same dispatch, no dynamic import. One consequence, documented as a known divergence, is that for `"type": "function"` segments the port currently uses the resolved `function_name` as the sole highlight group (`gen_segment_getter` at `src/ported/segment.rs:1011`) rather than honoring a theme's explicit `highlight_groups` override; Chapter 3's custom-segment section explains how to work with that.

process_segment and process_segment_lister

`process_segment` (at `src/ported/segment.rs:692`) is the per-segment driver. Its Rust signature carries the pieces the render needs — the side (`left` or `right`), the `segment_info` environment map, the mutable `parsed_segments` accumulator, the segment spec, the mode, the colorscheme, and a `contents_func` closure that actually invokes the adapter:

```
pub fn process_segment<C>{
    pl: &(),
    side: &str,
    segment_info: &Map<String, Value>,
    parsed_segments: &mut Vec<Value>,
    segment: &Map<String, Value>,
    mode: Option<&str>,
    colorscheme: &Colorscheme,
    contents_func: &C,
} where C: Fn(&(), &Map<String, Value>, &Map<String, Value>) -> Option<Value>
```

Its first act is to *copy* the segment (`segment = segment.copy()` at `py:168`), because processing mutates it — stamping the resolved highlight, the computed contents, the divider bookkeeping — and the theme's original must not be scribbled on, since the same theme dictionary may be rendered again on the next tick. It then branches on `type`: a `function` or `segment_list` segment runs its adapter (via `contents_func`) to produce contents and resolves its highlight group through the colorscheme; a `string` segment takes its contents literally. The output — a segment with resolved `contents` and a resolved `highlight` of `{fg, bg, attrs}` — is pushed onto `parsed_segments`, which is the list the renderer will later turn into bytes.

`process_segment_lister` (at `src/ported/segment.rs:578`) is the fan-out sibling. It takes a `lister` closure (which enumerates the things to render one segment each — workspaces, windows), a slice of `subsegments`, a `display_condition` predicate, and a `contents_func`, and for each item the lister yields it filters by `display_condition` and processes each subsegment against that item's patched environment. This is how “render one segment per X” works: the lister discovers the Xs at render time, and for each one the subsegment template is instantiated with that X's data folded into `segment_info`. It is more machinery than a single segment needs, which is why most themes never touch it directly — but it is what makes powerline's window-lister and workspace-lister segments possible, and the port carries it faithfully, generic closures and all.

The two functions together are the segment model's runtime. `gen_segment_getter` prepares the getters once (when the theme is loaded); `process_segment` and `process_segment_lister` run them on every render, producing the flat `parsed_segments` list of `{contents, highlight}` records that Chapter 6's renderer consumes. Everything upstream of the renderer — every adapter in Chapter 3, every colorscheme lookup in Chapter 6 — feeds through this pair. When you read a theme and wonder how a line of JSON becomes a colored chunk on screen, this is the answer: `gen_segment_getter` resolved it, `process_segment` ran it, and the renderer painted its result.

A segment, traced from JSON to `parsed_segments`

Abstractions are easiest to trust once you have watched one concrete case walk the whole path, so here is a single segment doing exactly that. Suppose a theme's left side contains this entry:

```
{ "function": "cpu_load_percent", "type": "function",  
  "args": { "format": "{0:.0f}%" }, "priority": 30 }
```

When the theme loads, `gen_segment_getter` runs once over this dictionary. It reads `type` and validates it against `"function" | "string" | "segment_list"`; `"function"` passes. It routes to the `get_function` path, which `rpartitions` the `function` string on its last `.` — there is no dot, so the module falls back to the default segment module and the function name is `cpu_load_percent`. The result is a `SegmentGetterResult` with `function_name = Some("cpu_load_percent")`, `module = Some(<default>)`, `contents = None` (this is not a string segment), the redundant `function_name_dup = Some("cpu_load_percent")` carried for py-tuple shape, and `name = None` (the theme set no explicit `name`). That record is cached on the getter; the JSON is never re-parsed on later renders.

On each render, `process_segment` takes over. It copies the segment so it can stamp results without scribbling on the theme's original, sees `type == "function"`, and calls `contents_func`, which looks `cpu_load_percent` up in the `ADAPTERS` registry (Chapter 3) and invokes the adapter. The adapter samples CPU utilization — say 71% — formats it through the `{0:.0f}%` argument into `"71%"`, and returns a one-element list whose single dict is `{"contents": "71%", "highlight_groups": ["cpu_load_percent_gradient", "cpu_load_percent"], "gradient_level": 71.0}` — the exact shape the port's `sys.rs::render` builds at `src/ported/segments/common/sys.rs:212`. Note the two highlight groups in priority order and the `gradient_level` equal to the load.

`process_segment` then resolves color. It hands the two groups and the `gradient_level` of 71.0 to `Colorscheme::get_highlighting` (Chapter 6), which walks the groups, resolves `cpu_load_percent_gradient` to a gradient list, and picks the color at 71% of the way along it — a warm orange on a green-to-red gradient. The segment now carries a resolved `contents` of `"71%"` and a resolved `highlight` of `{fg, bg, attrs}`, and it is pushed onto `parsed_segments`. That flat record — text plus concrete colors plus the `priority` of 30 the theme set for truncation — is everything the renderer of Chapter 7 needs; it never looks back at the JSON, the adapter, or the colorscheme again. One line of theme configuration has become one entry in the list the renderer will pack into bytes.

Chapter 3 – The Built-in Segment Library

powerliners ships 54 segment adapters (55 `ADAPTERS` keys counting the bare `exec` alias), wired in `src/bin/shared/render_runtime.rs` and shared between the daemon and the direct renderer. This chapter is the field guide: what each adapter reads, what it emits, and where the Rust port matches or extends the Python original.

The `ADAPTERS` registry

Where upstream powerline resolved a segment name to a callable by importing a Python module and reaching for an attribute, powerliners resolves it against a static table. The `render_runtime.rs` header says so directly: the Rust port “replaces that with a static `ADAPTERS` registry.” Each entry is a `(name, adapter)` pair, and the render path looks a name up three ways, in order: the full dotted path (`powerliners.k8s.kubecontext`), then a bare-name fallback (the trailing component, `kubecontext`), then — for the codegen'd getters — the resolved tail again. The relevant lines in `render_runtime.rs` (`ADAPTERS.iter().any(|(k, _)| *k == full)`, then the bare-name and tail fallbacks around lines 478–514) are exactly this three-tier lookup, and it is what lets a theme write either the full dotted path or the short name and hit the same adapter.

Watching the three tiers resolve a real name makes the fallback concrete. A theme that writes `{"function": "powerline.segments.common.net.hostname"}` presents the string `powerline.segments.common.net.hostname` to the render path. Tier one asks: is that *exact* string a key in `ADAPTERS`? It is, so it resolves immediately. Now consider a theme that writes the short form `{"function": "hostname"}`. Tier one fails — `hostname` is not the full registered key — so tier two takes the trailing component after the last `.` (here the whole string, since there is no dot) and matches it against the registered keys' trailing components, finding `...net.hostname` and resolving to the same adapter. Tier three handles the codegen'd getters by re-resolving the tail. The upshot for a theme author is that the short name and the fully-qualified name are interchangeable — the bundled themes tend to use the qualified form for precision, but a hand-written theme can say `branch` or `cpu_load_percent` and hit the same closure. This is the port's static substitute for upstream's dynamic `__import__`: where Python resolved a dotted string by importing a module and reaching for an attribute, powerliners resolves the same string against a compile-time table, preserving the naming while removing the import.

An adapter is a Rust closure with a fixed signature — the man page states it as `fn(args: &Map<String, Value>, info: &Map<String, Value>) -> Option<Value>` — that takes the segment's `args` (from the theme JSON) and the `segment_info` environment (cwd, environ, the client's arguments) and returns either `Some` contents or `None` to render nothing. Returning `None` is how a segment *conditionally* appears: the `branch` segment returns `None` outside a git work tree, so it simply is not there rather than showing an empty box; the `kubecontext` segment returns `None` when there is no active context. This “return nothing” convention is load-bearing across the library and is why a busy-looking theme collapses to a clean prompt when its context is absent.

The 54 adapters divide roughly into upstream ports and MenkeTechnologies additions. The ports are the segments a longtime powerline user already knows — `hostname`, `user`, `cwd`, `branch`, `date`, `battery`, the player family, the shell-status family. The additions, all under `src/extensions/`, are the net-new native segments — `git_status`, `ci_status`, the cloud contexts, the cache-stat segments — that Chapter 12 returns to. The registry does not distinguish them at lookup time; a theme names any of them the same way. The

sections below walk the library by function, naming what each adapter reads and emits.

Environment and identity: `hostname`, `user`, `environment`, `internal_ip`, `external_ip`

The identity segments answer “where and who am I,” and they are the cheapest in the library because their data is already in the process. `hostname` emits the machine’s host name — useful the moment you have more than one shell open across more than one box, which is the whole reason it exists; it honors the upstream argument that toggles showing the fully-qualified domain name versus the short name. `user` emits the current user name and, following upstream, can highlight differently when you are root, so a red `root` segment is a standing warning that you are operating with privilege. Neither forks a process or touches the network; both read what the kernel already handed the process at startup.

`environment` is the general-purpose escape valve of the identity group: it emits the value of a named environment variable, chosen by the segment’s `variable` argument. It is how a theme surfaces anything the shell exports — a deployment stage in `$APP_ENV`, a cluster name, a custom marker — without a bespoke adapter. Because it reads the environment that the client forwarded to the daemon over the wire (Chapter 4), it sees the variables as they were in the shell that triggered the render, not as they are in the daemon’s own process, which is the subtle but correct behavior: the render must reflect the *caller*’s environment.

`internal_ip` and `external_ip` answer “what is my address,” and they split along the cost line that runs through the whole library. `internal_ip` reads a local interface address — cheap, local, no network round-trip — and is the one you put in a prompt you want to stay instant. `external_ip` asks the outside world what your public address is, which means a network request, which means latency and the possibility of hanging; it belongs in a statusline you refresh on a timer, not a prompt you redraw on every keystroke. The distinction is not academic: it is the difference between a segment that costs microseconds and one that can cost seconds if the network is down, and Chapter 5’s performance discussion returns to why the daemon’s on-demand sampling makes the placement of segments like this a real decision.

System load: `cpu_load_percent`, `mem_swap`, `disk_io`, `disk_usage`, `gpu_usage_percent`, `gpu_vram`

The system-load segments are the ones people put powerline on for — a live glance at the machine’s vitals in the corner of every window. `cpu_load_percent` emits CPU utilization as a percentage, and it is the canonical gradient segment: it carries a `gradient_level` equal to the load, so the colorscheme shades it from a calm color at idle through a hot color under pressure (Chapter 6’s `pick_gradient_value` is what turns that number into a color). On the sampling side the port diverges deliberately from upstream: where Python powerline leans on `psutil`, powerliners samples via OS subprocess probes — `top`, `vm_stat`, `netstat`, `pmset`, `uptime` on macOS, `/proc` and `/sys` on Linux — so it renders successfully on a machine with no `psutil` installed. The README documents this as an inherent divergence: the framing is identical, the numeric value may differ by a sampling-window’s worth of data.

The memory family — `mem_swap`, `mem_usage`, `mem_swap_percentage`, `mem_usage_percent` — reports RAM and swap in two shapes each: an absolute `USED/TOTAL` in human-readable bytes, and a percentage. These are ports of `mKaloer/powerline_mem_segment` living under `src/extensions/mem_usage.rs`, and their human-byte formatting (the `humanize_bytes` helper, ported from powerline’s own `lib/humanize_bytes.py`) is the same K/M/G rounding upstream uses, so a memory segment reads the same in both.

The storage and GPU segments are MenkeTechnologies additions with a vendor- dispatch pattern worth noting. `disk_usage` and `disk_usage_percent` report a filesystem’s used space for any mount; `disk_io` reports live read/write throughput for a device. `gpu_usage_percent` and `gpu_vram` report GPU compute and VRAM, and they dispatch across vendors in a fallback chain — `nvidia-smi`, then `rocm-smi`, then `intel_gpu_top`, then an `ioreg` fallback on macOS — so one segment works on a machine regardless of whose GPU is in

it. `thermal` rounds the family out with CPU/GPU temperature and fan RPM, from `/sys/class/hwmon` on Linux and `powermetrics/istats` on macOS. All of these live under `src/extensions/` and are wired into `ADAPTERS` as ordinary built-ins — no filesystem lookup, no custom-segment ceremony (Chapter 12).

Version control and orchestration: `branch`, `stash`, `kubecontext`, `containers`

Version control is powerline's most-loved feature and `branch` is its face. The `branch` adapter emits the current git branch name (falling back to a short SHA or tag on a detached HEAD) and returns `None` outside a work tree, so it appears exactly when it is meaningful. `stash` complements it with the count of stashed changes. Alongside the ported `branch`, powerliners ships a heavier native VCS segment, `git_status` (`powerliners.vcs.git_status`), which is p10k-style: a single chunk with the branch glyph, the name, and count badges for unstaged, untracked, staged, conflicted, ahead, behind, and stashed — all from a single `git status --porcelain=v2 --branch` fork. Where `branch` is the faithful upstream port, `git_status` is the richer MenkeTechnologies addition; a theme picks whichever fits.

It is worth seeing what `git_status` actually produces, because the single-fork p10k shape is its whole point. In a work tree on branch `main`, three files edited but unstaged, one new file untracked, two commits ahead of the remote and one stashed change, the adapter runs one `git status --porcelain=v2 --branch` (plus a `git stash list` and a `git config --get remote.origin.url`), parses the porcelain-v2 output in `parse_porcelain_v2`, and assembles a single chunk that reads roughly `main ↑2 *1 !3 ?1` — the branch glyph and name from the header, then the badges in the fixed order action, behind, ahead, stash, conflicts, staged, unstaged, untracked, each emitted only when its count is nonzero (so a clean tree collapses to just `main`). On a detached HEAD the header falls back through `git describe --tags --exact-match` to a tag, or to `@<shortsha>` from `git rev-parse --short HEAD`, and an in-progress operation (a stat of `MERGE_HEAD/rebase-merge/CHERRY_PICK_HEAD` and friends) prepends an action marker. With `status_colors` on, the chunk requests `["git_status_dirty", "git_status", "branch_dirty", "branch"]` when anything is dirty and the `_clean` equivalents otherwise, so a colorscheme paints the whole chunk red-on-dirty without the theme doing anything. The contrast with the ported `branch` is instructive: `branch` produces just the name in its own chunk and defers everything else to separate `stash` and status segments, faithful to upstream's decomposition; `git_status` folds all of it into one fork and one chunk, which is faster and denser but is new work, not a port — exactly the `src/ported` versus `src/extensions` boundary the book keeps drawing.

The orchestration segments bring cloud-native context into the prompt. `kubecontext` (`powerliners.k8s.kubecontext`) emits the current kubectl context and active namespace, honoring the `$KUBECONFIG` cascade and in-context namespace overrides, with a `hide_default` argument that suppresses the namespace when it matches the configured default — so the segment stays quiet when you are in the boring case and speaks up when you have switched to something that could bite you. This is the kind of segment where powerline earns its keep: a namespace you forgot you were in is how a `kubectl delete` hits the wrong cluster, and a persistent reminder in the prompt is cheap insurance.

`containers` (`powerliners.docker.containers`) reports Docker/OCI container counts — running, total, images, stopped — probing via `docker ps` and falling through silently when the daemon is unreachable, which is the right failure mode: a container segment on a machine where Docker is not running should vanish, not error. The broader cloud family (`aws.context`, `gcp.context`, `ci_status`) follows the same discipline — pure filesystem probes where possible, silent fall-through when the underlying tool or context is absent — and Chapter 12 catalogs them in full. The common thread across orchestration is graceful absence: every one of these segments knows how to render nothing, and does so the moment its context disappears.

Time, jobs, and status: `date`, `fuzzy_time`, `jobnum`, `last_status`, `last_pipe_status`

The time segments are the simplest adapters in the library and a good place to see the model with nothing else in the way. `date` emits the current date/time formatted by a strftime-style `format` argument from the theme — no environment read, no subprocess, just the clock and a format string. `fuzzy_time` is its playful sibling: instead of `14:37` it emits a human phrasing — “twenty to three,” “quarter past” — the same rounded-to-the-nearest-five-minutes prose powerline’s `fuzzy_time` produces. Neither costs anything to render, which is why they are safe in a prompt redrawn on every keystroke.

The shell-status segments are where the client’s forwarded arguments matter, and they are why the wire protocol carries `argv` and not just the environment. `last_status` emits the exit code of the previous command — but only when it is nonzero, so a successful command leaves no trace and a failed one paints its exit code in a warning color. It gets that exit code from the shell binding, which passes `--last-exit-code=$?` on the `powerline shell left` invocation (Chapter 7); the segment reads it out of the client’s arguments. `last_pipe_status` is the richer version: it shows the exit status of every stage in a pipeline (`$pipestatus` in `zsh`, `$PIPESTATUS` in `bash`), so a failure in the middle of a `a | b | c` is visible even when the last stage succeeded and `$?` is zero. This is the segment that catches the classic “the pipeline lied to me” bug.

`jobnum` emits the count of background jobs — the `[2]` that reminds you that you suspended two things and forgot them. Like the status segments, it is fed by the shell binding, which knows the job count and forwards it, because the daemon itself has no way to see the calling shell’s job table. This is a recurring theme in the shell-status group: the daemon is a separate process and cannot inspect the shell that called it, so anything shell-local — exit codes, pipe status, job counts, the current `vi` mode for the mode indicator — must be handed over on the wire. The segments read it from `segment_info` where the daemon deposited the parsed client arguments (Chapter 4), and this is the concrete reason the wire protocol forwards `argv` at all: without it, the whole shell-status family would be blind.

Media and the long tail: `itunes`, `cmus`, `dbus_player`, `email_imap_alert`

The player family is a small class hierarchy in upstream and a faithful one in the port. Powerline models “what is playing” through a common `PlayerSegment` base with per-player adapters, and powerliners mirrors that — `itunes`, `cmus`, `mocp`, `mpd`, `rhythmbox`, `clementine`, `spotify`, and the generic `dbus_player` all produce the same shape of output (track, artist, play state) through per-player probes. `dbus_player` is the generic Linux path, querying MPRIS over D-Bus so any compliant player works; the named adapters are the specific integrations for players that predate or sidestep MPRIS. The man page notes the port “faithfully mirroring upstream’s class hierarchy,” which is the point: the shape is inherited, not redesigned.

`email_imap_alert` is the long tail’s most characterful member — a segment that logs into an IMAP mailbox and emits your unread count. It is the archetype of a segment you never put in a prompt: it makes a network connection, it authenticates, and it can block, so it belongs only in a slowly-refreshed statusline where a second of latency now and then is invisible. It also illustrates why powerline’s per-segment placement matters — the same renderer that draws a microsecond `date` segment can draw a segment that talks to a mail server, and the theme author’s job is to keep the expensive ones off the interactive-latency path.

The long tail as a whole is where the library’s breadth shows and where the “return `None`” convention does its quietest work. Most of these segments are absent most of the time — no player running, no mail configured — and they cost nothing when absent because the adapter returns `None` before doing any expensive work. A theme can list a dozen of them and pay only for the ones that have something to say. This is the design that lets powerline be simultaneously feature-rich and fast: features you are not using are segments returning `None`, and a segment returning `None` is close to free.

The `exec` escape hatch

No fixed library can anticipate every segment a user wants, and upstream powerline handled that with dynamic import: drop a `.py` file in `~/.config/powerline/segments/`, reference it by dotted path, and `__import__` made it callable. powerliners is a static binary with no Python runtime, so it cannot import a `.py` at render time — and rather than pretend that limitation away, it provides two subprocess-based escape hatches that reach the same expressive power through a different mechanism, both wired through `src/extensions/exec_segment.rs`.

Pattern A is the explicit `exec` adapter. A theme references the built-in `exec` segment directly and spells out, inline, the command to run, its arguments, a format template, and the highlight groups: `{"function": "exec", "args": {"command": "/usr/local/bin/cpu_temp.sh", "args": ["--unit", "C"], "format": "%s°C", "highlight_groups": ["cpu_load"]}}`. The daemon forks the command, reads its stdout, and wraps the result as a segment. Pattern B is dotted-path filesystem dispatch: reference your segment by dotted path (`{"function": "myseg.cpu_temp"}`) and drop an executable under `<config_path>/segments/myseg/cpu_temp.{sh,py,rb,pl,lua,js}`. The daemon resolves the path against its full config cascade and runs the first match; the search order lives in `SCRIPT_EXTENSIONS` at `src/extensions/exec_segment.rs:31` — `["sh", "py", "rb", "pl", "lua", "js", ""]`, with `sh` winning over `py` when both exist and the empty string matching a bare executable file.

The output protocol is the same for both patterns and it is deliberately forgiving. The daemon inspects the script's stdout at the first non-whitespace byte: if it is `[`, the output is parsed as a JSON array of segment dicts and used verbatim — giving the script full control over `highlight_groups`, `gradient_level`, `divider_highlight_group`, and multi-chunk output; anything else is treated as plain text and wrapped as `[{"contents": <trimmed-stdout>, "highlight_groups": [...]}]`, with the `format` template applied (`%s` → contents, `%%` → literal `%`). So a two-line shell script that echos a number becomes a segment, and a Python script that prints a JSON array becomes a fully-styled multi-chunk segment — same adapter, two levels of ambition.

There is one Rust-port divergence to know when using `exec`, and the README states it plainly: `gen_segment_getter` (at `src/ported/segment.rs:1011`) uses the segment's resolved `function_name` as the *only* highlight group for function segments, ignoring a theme's `highlight_groups` override (upstream respects the override). The practical consequence is to name your colorscheme group to match the function name — a `"exec"` group for Pattern A, or the dotted path's trailing component (`"cpu_temp"`) for Pattern B, which is why Pattern B is the tidier of the two: its natural function name *is* a sensible group name. It is a small wrinkle, honestly documented rather than hidden, and Chapter 10's port methodology explains why a known divergence like this is tracked rather than papered over.

Chapter 4 – The Daemon and the Wire Protocol

The startup tax is defeated by never starting up: `powerline-daemon` binds a UNIX socket, loads the config tree once, and answers render requests over the same wire format the upstream C client speaks. This chapter is the heart of the performance story.

Why a daemon, and why a UNIX socket

The daemon exists because the only way to stop paying startup on every render is to render in a process that is already running. `powerline-daemon` is that process: one per UID, it loads the JSON config tree once, instantiates the segment adapters once, and holds the theme dictionaries and color palettes in memory (the man page's architecture section states exactly this). Every subsequent render is a message to the running daemon rather than a new process, so the config parse, the adapter setup, and — in the Python original — the interpreter boot all happen exactly once, at daemon start, and never again. The client that talks to it can be trivially cheap because all the expensive state lives on the daemon side.

A UNIX domain socket is the right transport for this because the client and daemon are always on the same machine and the message is small and synchronous. A UNIX socket is a local IPC channel with no network stack, no TCP handshake, and kernel-mediated delivery — a round-trip is microseconds, which is the budget a per-keystroke prompt render has to fit inside. It also gives natural per-user scoping: the socket is named per-UID, so your daemon serves your shells and only yours, with filesystem or abstract-namespace permissions doing the isolation. A TCP socket would add latency and a security surface for no benefit, since there is nothing remote to talk to; a named pipe would be clumsier for the request/response shape. The UNIX socket is the upstream choice and the port keeps it.

The single-daemon-per-UID design is what makes the whole thing scale to a real workflow. One `powerline-daemon` process handles your tmux statuslines, your zsh and bash prompts, and every running vim simultaneously — there is no per-vim daemon, no per-shell daemon, no per-window daemon. Start it once at login (`powerline-daemon -q` in `.zshrc` or `.profile`) and every powerline-aware program you launch for the rest of the session finds the socket already bound and warm. This is why the README recommends pre-starting the daemon for vim: with it running, each statusline refresh is a microsecond socket round-trip; without it, every refresh fork-execs `powerline-render` cold at ~30 ms.

The upstream wire format: `argv + cwd + env`

The wire format is the contract that makes powerliners a drop-in replacement, and it is copied exactly from upstream's C client (`vendor/powerline/client/powerline.c`, lines 126–148). The request the client sends is a single flat, NUL-delimited byte blob:

```
hex(argc-1) "\0" argv[0] "\0" argv[1] "\0" ... cwd "\0" env[0] "\0" ... "\0\0"
```

The first field is the argument count minus one, in hexadecimal, as text; then come the arguments themselves, each NUL-terminated; then the current working directory; then the environment as `KEY=VAL` entries, each NUL-terminated; and finally a double NUL to mark end-of-message. There is no length prefix on the whole

blob — the reader accumulates until it sees the terminating double NUL — and no encoding beyond raw bytes, which is what lets it carry arbitrary environment values.

The daemon's `parse_args` (at `src/ported/scripts/powerline_daemon.rs:254`) demarshals this, and its doccomment states the format verbatim: `<numargs_hex>\0<arg>\0<arg>...\0<cwd>\0<env=val>\0...`. It splits on NUL, reads the hex arg count, takes that many arguments, then the cwd, then parses the remaining entries as `KEY=VAL` environment pairs — and, mirroring upstream's `cwd = cwd or environ.get('PWD', '/')`, it falls back to the `$PWD` environment value when the cwd field is empty. The result is a `(Vec<String>, HashMap<String, String>, String)` — `argv`, `environ`, `cwd` — which is precisely the trio a render needs: the arguments say *what* to render (`tmux right, shell left --last-exit-code=1`), the environment says *in what context*, and the cwd anchors path-relative segments like `cwd` and `branch`.

Forwarding the *whole* environment and the cwd is what makes a render in the daemon faithful to the shell that requested it. The daemon is a long-lived process with its own stale environment and its own working directory; if it rendered against those, every segment that reads the environment (`environment`, the shell-status family, `$KUBECONFIG`-aware `kubecontext`) or the path (`cwd`, `branch`) would be wrong. By carrying the caller's `argv`, `cwd`, and `env` over the wire and rendering against *those*, the daemon produces exactly the output the caller would have gotten from a fresh in-process render — which is the entire correctness requirement for a drop-in daemon, and why cross-version clients and servers interoperate as long as they preserve this format.

The wire request, byte by byte

Nothing makes a protocol concrete like a hexdump, so here is exactly what the client writes for the invocation `powerline shell left --last-exit-code=1`, from a shell whose working directory is `/home/j` and whose environment (trimmed to two variables for the example) is `PWD=/home/j` and `TERM=xterm-256color`. The client's marshalling loop (`src/bin/powerline.rs:88-130`) produces, in order:

33 00	"3\0"	argc-1 = 3, in hex
73 68 65 6c 6c 00	"shell\0"	argv[1]
6c 65 66 74 00	"left\0"	argv[2]
2d 2d 6c 61 73 74 2d 65 78 69 74 2d 63 6f 64 65	"--last-exit-code=1\0"	argv[3]
3d 31 00		
2f 68 6f 6d 65 2f 6a 00	"/home/j\0"	cwd
50 57 44 3d 2f 68 6f 6d 65 2f 6a 00	"PWD=/home/j\0"	env[0]
54 45 52 4d 3d 78 74 65 72 6d 2d 32 35 36 63 6f	"TERM=xterm-256color\0"	env[1]
6c 6f 72 00		
00	"\0"	final terminator

Every field worth naming is visible in those bytes. The very first field, `33` = ASCII `3`, is the argument count minus one written in *hexadecimal text*, not a binary integer — a three-argument invocation (`shell`, `left`, `--last-exit-code=1`) writes `3`, and a seventeen-argument one would write `11`. Then come the three arguments, each terminated by a single NUL; then the working directory `/home/j`; then the environment as `KEY=VAL` pairs, each NUL-terminated. The message ends with a NUL, which — following the final NUL of the last env entry — forms the terminating double NUL `\0\0` that `do_read` watches for (`ends_with(b"\0\0")`).

On the receiving side, `parse_args` (`src/ported/scripts/powerline_daemon.rs:249`) runs this in reverse. It splits the blob on NUL and drops empty fragments, reads `usize::from_str_radix("3", 16)` to learn there are three arguments, slices `parts[1..3]` as `argv`, takes `parts[4]` (`/home/j`) as the `cwd`, and parses everything after as `KEY=VAL` by splitting each fragment at its *first* = (so a value containing `=`, like a base64 token in an env var, survives intact). Two guards matter here: an empty request returns `None` (the daemon replies `"malformed request"` rather than crashing), and an empty cwd field falls back to the `PWD` environment value and then to the literal `"/"`, mirroring upstream's `cwd = cwd or environ.get('PWD', '/')`. The result is the `(Vec<String>, HashMap<String, String>, String)` trio — `argv`, `environ`, `cwd` — that a faithful render needs, reconstructed exactly as the calling shell had it.

powerline – the native client and its daemon-less fall-back

`powerline` (the binary at `src/bin/powerline.rs`, a 1:1 port of `vendor/powerline/client/powerline.c`) is the thin client that shells and `tmux` and `vim` actually invoke, through the `$POWERLINE_COMMAND` environment variable. Its job is small and fixed: build the wire request from its own `argv`, the current working directory, and the inherited environment; connect to the daemon's UNIX socket; send the request; and copy the daemon's rendered response to standard output. That is the entire happy path — a marshal, a socket write, a read, a copy — and it is why the client can be so much cheaper than the render it triggers. All the cost lives on the daemon side; the client is just a courier.

The client's one interesting decision is what to do when the courier finds no one home. If the daemon socket is not reachable — the daemon was never started, or was killed, or crashed — `powerline` does not error and does not hang; it execs `powerline-render` as a fallback, mirroring the upstream C client's behavior exactly. `powerline-render` is the direct, daemon-less renderer: it parses the config, runs the segments, and emits the same markup, all in one process. It is slower than the warm daemon path (a cold native render, ~30 ms, versus a sub-millisecond socket round-trip) but it is still an order of magnitude faster than the Python interpreter path, and — critically — it is never *broken*. A prompt that falls back to `powerline-render` still renders; you lose speed, not function.

This fallback is the resilience story of the whole design. A daemon can die — a reboot, an OOM kill, a manual `kill` — and if the prompt depended on it absolutely, a dead daemon would mean a broken shell. Because the client transparently degrades to direct rendering, a dead daemon means a *slower* shell that quietly keeps working until the daemon is restarted (or until the next login re-runs `powerline-daemon -q`). The `--socket PATH` option lets the client target a non-default socket, and it must be the first argument, before the extension name, per the C-client convention; absent it, the client uses the platform default socket path (the next section).

Bind, daemonize, pidfile lock, accept loop, EOF shutdown

Starting the daemon is a sequence of careful steps, all ported from upstream's `scripts/powerline-daemon`, and the port keeps each one. First it takes a **pidfile lock** so two daemons cannot race for the same socket; the port implements this both as a `PidLock` RAIL guard (released on `Drop`) and by the classic lock-the-pidfile mechanism, so a stale pidfile from a crashed daemon does not permanently block a new one. Then it **binds** the UNIX socket. `check_existing` (at `src/ported/scripts/powerline_daemon.rs:1018`) attempts the bind and returns `Ok(None)` on `EADDRINUSE`, which is how the daemon detects that another instance is already serving that socket — the basis for the `--replace` and plain-startup behaviors.

Then, unless `--foreground` was passed, it **daemonizes**. `daemonize` (at `src/ported/scripts/powerline_daemon.rs:925`) is the textbook double-fork: `fork`, exit the parent; `setsid` to detach from the controlling terminal; `fork` again so the process can never re-acquire a terminal; `chdir /`; `umask 0`; and `dup2` the standard descriptors to `/dev/null`. This is what the `-q` flag's "double-forks and detaches" means — after it, the daemon is a session-leaderless background process with no terminal, no controlling shell, and no stray file descriptors, exactly as a long-lived server should be. `--foreground` skips this entirely and keeps the daemon attached to the terminal, which is what a systemd `Type=simple` unit or an interactive debugging session wants.

With the socket bound and the process detached, the daemon enters its **accept loop** — `main_loop` (at `src/ported/scripts/powerline_daemon.rs:854`), which multiplexes over the listener and the live connections. Where upstream's Python used `select.select`, the port uses `libc::poll` (a documented, behavior-neutral substitution: `POLLHUP` is folded into "readable" so a peer hangup reads as EOF, exactly as `select` reported it). When the listener is readable it accepts a new connection (through `eintr_retry_call`, so an `EINTR`-interrupted `accept` retries rather than dying); when a connection is readable it reads a full request up to the terminating double NUL — `do_read` loops until the buffer `ends_with(b"\0\0")` or a two-second read timeout elapses — checks it, and either renders and replies or shuts down. The **shutdown sentinel** is

the special request `EOF = b"EOF\0\0"` (at `src/ported/scripts/powerline_daemon.rs:73`): if a client sends exactly that, `get_answer` returns the exit signal and the daemon exits cleanly with status 0. This is how `powerline-daemon -k` stops a running daemon — it connects and sends the EOF sentinel — and it is a faithful port down to the exact byte sequence, because a client written against the Python daemon must be able to stop the Rust one and vice versa. The daemon exits cleanly on `SIGTERM` — the handler (`sigterm_handler` at `powerline_daemon.rs:1069`) unlinks the pidfile and `_exits` in an async-signal-safe way — and treats `SIGINT` as a clean exit-0, mirroring upstream's `KeyboardInterrupt` → `SystemExit(0)`. One lifecycle detail is an honest omission rather than a port: upstream's `SIGHUP`-driven re-exec (reload config without dropping the socket) is *not* implemented. A config edit is instead picked up by the per-`ext` mtime-staleness reload in `src/bin/powerline-daemon.rs`, which rebuilds the config cache when the underlying files change, and a full restart is available via `--replace`. It is exactly the kind of divergence Chapter 10's methodology tracks in the open rather than papering over.

The socket paths and the two namespaces

Where the socket lives depends on the platform, and the split is inherited from upstream's `USE_FILESYSTEM = not sys.platform.lower().startswith('linux')`. The port's `USE_FILESYSTEM()` (at `src/ported/scripts/powerline_daemon.rs:63`) is `!cfg!(target_os = "linux")` — Linux gets one behavior, everything else the other. On macOS, FreeBSD, OpenBSD, and NetBSD the socket is a real filesystem path, `/tmp/powerline-ipc-<uid>`, an actual file you can see with `ls`. On Linux the socket is an *abstract-namespace* socket, `\0powerline-ipc-<uid>` — the leading NUL marks it as abstract, meaning it has no filesystem entry at all; it appears only in `ss -x` and `lssof -U` under the NUL marker.

The abstract namespace is not an arbitrary choice; it solves a real problem that filesystem sockets have. A filesystem socket file persists after the daemon dies — a crash or a hard kill leaves the socket file behind, and a `/tmp` cleanup job or a race can delete a *live* socket out from under a running daemon. An abstract socket has no filesystem presence, so it cannot be orphaned by a crash or swept by a `/tmp` cleaner: it exists exactly as long as the daemon holds it and vanishes the instant the daemon exits. The port's bind logic (around `powerline_daemon.rs:1227`) is careful here — it unlinks any non-abstract address before binding regardless of the platform default, so an explicit `--socket /some/path` (a filesystem path) still cleans up a stale file, while the default Linux path uses the abstract namespace and needs no cleanup at all.

Both paths are overridable with `--socket PATH`, which is how the migration workflow runs a throwaway daemon on a probe socket (`--socket /tmp/powerliners-probe`) without disturbing the real one, and how a user with an unusual setup can put the socket wherever they need it. The default per-UID naming (`powerline-ipc-<uid>`) is the same on both namespaces, so the *identity* of the socket — “this UID's powerline daemon” — is stable across platforms even though its physical form differs. The matching pidfile, when used, is `/tmp/powerline-ipc-<uid>.pid`, which is what the migration guide's `ps -p $(cat /tmp/powerline-ipc-$UID.pid)` check reads to confirm the running daemon is the Rust port and not the Python shebang.

The five binaries: `powerline`, `-daemon`, `-render`, `-config`, `-lint`

powerliners ships not one binary but five, and the split mirrors upstream's own division of labor so exactly that each Rust binary names the upstream file it replaces. Together they are the whole CLI surface a powerline user touches, and the Cargo manifest declares them explicitly (`[[bin]]` entries for each) so `cargo install` and `cargo publish` ship the complete suite.

`powerline` (mirrors `client/powerline.c`) is the native client of the previous sections — the thin courier that forwards `argv + cwd + env` to the daemon and falls back to `powerline-render` when the socket is unreachable. `powerline-daemon` (mirrors `scripts/powerline-daemon`) is the server — bind, daemonize, pidfile lock, accept loop, EOF shutdown — rendering real `#[fg=...,bg=...]...` tmux markup from a user's config root. `powerline-render` (mirrors `scripts/powerline-render`) is the direct, daemon-less renderer:

`argparse` plus extension lookup plus the full render path through the `Powerline` orchestrator; it is both a standalone tool and the client's fallback.

`powerline-config` (mirrors `scripts/powerline-config`) is the setup and dispatch tool — it is what `run-shell "powerline-config tmux setup"` in your `.tmux.conf` invokes to install the tmux integration, and what `powerline-config vim source-path` invokes to extract the bundled vim plugin (Chapter 7). It handles the tmux/shell known-function dispatch that upstream did in its config script. `powerline-lint` (mirrors `scripts/powerline-lint`) is the config validator: `argparse` plus the full check pipeline — a markedjson loader, the `Spec` checks, and orchestrator integration — that flags unknown segment names, missing color references, and malformed `before/after` formatting strings before they become a broken prompt (Chapter 9's fidelity work leans on it). Five binaries, one config tree, one wire protocol — the suite is the whole of what `pip install powerline-status` gave you, reimplemented as native code.

Chapter 5 – The Performance Story

The daemon is the mechanism; performance is the payoff, and this chapter states it in numbers and in the honest shape of where the wins come from. `powerliners` is fast for a few specific, nameable reasons, and it is worth separating them from each other rather than blurring them into “it’s written in Rust.”

The startup tax, measured

The headline number is the one from Chapter 1, and it bears restating precisely because it is the whole justification for the architecture. Python `powerline` pays ~50–150 ms per render, essentially all of it interpreter startup and module import, before any useful work begins. `powerliners`’ warm daemon path is sub-millisecond: a UNIX-socket round-trip plus the microseconds of the actual render. That is two to three orders of magnitude, and the gap is not from a cleverer render algorithm — the render itself was always cheap — it is from *deleting startup* as a per-render cost.

The cold path matters too, because it is the fallback and because not every invocation is served by a warm daemon. `powerline-render`, the daemon-less direct renderer, runs cold at roughly ~30 ms per call — the README’s figure for the vim statusline case when no daemon is running. That is still three to five times faster than Python’s cold path, because even without a warm process there is no interpreter to boot: a native binary’s cold start is the kernel mapping a static executable, not CPython initializing its object system and walking an import graph. So the performance story has two tiers — sub-millisecond warm, ~30 ms cold — and both beat the thing they replace, with the warm tier beating it by a margin large enough to change how a prompt *feels*.

The reason the tax is so total in Python and so absent in `powerliners` is structural, not incidental, and Chapter 1 made the case: a fresh process cannot amortize its own startup, so a model built on fresh-process-per-render pays startup every time by construction. The daemon breaks that construction. It is not that `powerliners` optimized startup; it is that `powerliners` arranged for startup to happen once. This is why “rewrite it in Rust” alone would not have solved the problem — a Rust binary that still forked per render would pay a Rust binary’s (small) startup every time, which is better than Python’s but not sub-millisecond. The daemon is the load-bearing idea; the native language is what makes the daemon’s client cheap and its cold fallback tolerable.

The warm daemon and its caches

A warm daemon is fast for more reasons than “it is already running.” Its state is built once and reused: the config tree is parsed once (the cascade of Chapter 6), the theme dictionaries and color palettes are held in memory, and the segment adapters are wired once. The daemon also keeps a `Powerline`-instance cache keyed by the things that determine which orchestrator to use — the `PowerlineKey` struct (at `src/ported/scripts/powerline_daemon.rs`) is a port of upstream’s cache tuple: the extension, the renderer module, the config/theme/path overrides, and the `POWERLINE_THEME_OVERRIDES` / `POWERLINE_CONFIG_OVERRIDES` / `POWERLINE_CONFIG_PATHS` environment values. Two renders that share those keys share a warm orchestrator; a render that differs in them gets its own, built once and then cached.

The consequence is that the *second* render of a given kind is even cheaper than the first, because the first built and cached the orchestrator and the second just reuses it. In a real session almost every render is a second render — the same tmux statusline, the same shell prompt, over and over — so the amortized cost settles to the floor: socket round-trip plus render, with all the setup already paid. This is the daemon doing what a daemon is for, keeping warm state warm across the thousands of renders a day produces.

The cache key makes “the same kind of render” precise, and it is worth seeing what shares a warm orchestrator and what does not. `PowerlineKey` (Appendix E.7) is a tuple of the extension, the renderer module, the config/theme/path overrides, and the three environment values `POWERLINE_THEME_OVERRIDES` / `POWERLINE_CONFIG_OVERRIDES` / `POWERLINE_CONFIG_PATHS`. Consider a real session: a `powerline shell left` from `zsh` and a `powerline shell right` from the same `zsh` differ in *side* — but *side* is not part of the key (it is a render argument, not a cache dimension), so both hit the same warm `shell` orchestrator, and only the second one ever paid to build it. A `powerline tmux left` from `tmux` differs in *extension* (`tmux` vs `shell`), so it keys to a different orchestrator, built once on the first `tmux` render and reused thereafter. A `vim` statusline keys to a third. So a developer living in `zsh` under `tmux` with `vim` open warms exactly three orchestrators — one per extension — and every subsequent render of any of the three reuses its warm instance. The key is deliberately coarse in the dimensions that do not change the orchestrator (*side*, the exit code, the job count — all render arguments) and precise in the ones that do (extension, renderer, overrides), which is what lets a handful of warm orchestrators serve an entire day’s tens of thousands of renders. The first render of each kind pays setup; the second through ten-thousandth pay only the socket round-trip and the render, which is the floor the whole architecture drives toward.

There is a deliberate trade-off in how the daemon samples live data, and the README is candid about it. Where Python `powerline`’s `ThreadedSegment` polls expensive data (network, disk) in a background thread and renders the last-known value, `powerliners`’ daemon samples on demand at render time. The upside is simplicity and freshness — no background threads, no stale cached value. The downside is a different latency profile: a render that hits a network or disk segment may block briefly while it samples, where the threaded model would have returned instantly with a slightly-old value. The *content* matches; the timing differs. This is why segment placement is a real decision (Chapter 3): an `external_ip` or `email_imap_alert` segment on the interactive-prompt path can occasionally stall the prompt, and a theme author who cares about per-keystroke latency keeps the expensive samplers on the timer-refreshed statusline instead.

Why “in Rust” is necessary but not sufficient

It would be easy to summarize `powerliners` as “`powerline`, but in Rust, so it’s fast,” and the summary would be wrong in an instructive way. Rust is necessary — a static binary with microsecond process startup and no garbage collector is what makes the client cheap and the cold fallback tolerable — but Rust is not where the two-orders- of-magnitude win comes from. That win comes from the *daemon*, an architectural choice that could in principle have been made in Python too (and upstream did make it, which is why upstream has a `powerline-daemon` at all). What Rust adds on top of the daemon is that the client and the cold path are both cheap in absolute terms, and that the whole suite is a single static artifact with no runtime to install per machine.

The honest decomposition is three separate wins, and this book keeps them separate the way Chapter 1 promised. First, the daemon deletes per-render startup — the big win, available in principle to any language. Second, the native binary makes the client and the cold render cheap — a Rust win, and the reason `powerliners`’ fallback path is ~30 ms rather than Python’s ~100. Third, the single static binary makes deployment trivial (Chapter 11) — no interpreter, no `pip` tree, no shared library to keep in sync. Conflating these into “it’s fast because Rust” obscures which lever does what, and the whole design reads more clearly when you keep them apart: the daemon is the architecture, Rust is the implementation, and the static binary is the packaging.

None of this makes the render *algorithm* faster, and it is worth being clear that `powerliners` does not claim to. The render was never the bottleneck; resolving a branch and packing `tmux` markup is microseconds in any competent implementation. What `powerliners` removes is the overhead *around* the render — the startup, the import, the per-invocation process cost — which is where all the time actually went. A faithful port that rendered identically but paid Python’s startup would be no faster; a native rewrite that still forked per render would be only somewhat faster. It is the combination — same faithful render, no per-render startup, cheap native client — that produces the number in the code fence of Chapter 1.

The latency budget, accounted

“Sub-millisecond” is a claim worth spending, because a budget accounted line by line is harder to hand-wave than a single headline number. The warm path is a fixed sequence of steps, each with a nameable cost, and it is instructive to walk them in order and see where a microsecond can and cannot hide. The client’s half is a marshal and two syscalls: it builds the `argv + cwd + env` blob in a single owned buffer (`powerline.rs:88-130`), which is a handful of `pushes` and no allocation surprises, then `connect(2)`s the UNIX socket and `write(2)`s the blob. A UNIX-domain `connect` and `write` to a listening peer on the same machine are kernel-local operations — no routing, no handshake, no copy across a network stack — so the pair costs single-digit microseconds on a warm scheduler. The client then blocks in `read(2)` for the reply, which is where it spends most of its wall-clock time: not computing, but waiting for the daemon to answer.

On the daemon’s half the budget divides into three visible slices, and only one of them is the render. The first slice is transport: `poll` returns the listener or the connection as readable, `do_read` accumulates the request until it `ends_with(b"\0\0")` (`powerline_daemon.rs:296`), and `parse_args` splits it on NUL into the `(argv, environ, cwd)` trio (:249). For a typical prompt request — a few arguments and a few dozen environment entries — this is a bounded number of byte scans and small-string allocations, tens of microseconds at most, dominated by the size of the forwarded environment rather than by any computation. It is the one slice that grows with the caller’s `env`, which is why a shell with a pathologically large environment can make even the warm path measurably slower: the daemon must demarshal every `KEY=VAL` the client sent, whether or not any segment reads it.

The second slice is the cache lookup, and in the common case it is nearly free. The daemon builds the `PowerlineKey` (Appendix E.7) — extension, renderer module, the override triple — and probes its orchestrator cache. On the first render of a given kind this misses and the daemon pays the one-time setup (config cascade, colorscheme build, adapter wiring) that Chapter 6 details; on the second through ten-thousandth it hits, and the whole of that setup is skipped for a hash lookup on a small tuple. This is the slice the daemon architecture exists to shrink to zero, and in steady state it succeeds: the amortized cost of orchestrator setup across a day’s renders rounds to the cost of one setup divided by tens of thousands, which is noise.

The third slice is the render itself, and it is smaller than intuition suggests. Each adapter runs — most of them read a value already in `segment_info` (the exit code, the job count, the hostname) and format it, which is microseconds; the probe segments (`cpu_load_percent`, `git_status`) cost a syscall or a subprocess and are the reason segment *placement* is a real decision (Chapter 3, Appendix AB.5). The colorscheme resolves each segment’s groups through `get_highlighting` (a walk of a short list and a dict lookup), the truncation pass measures widths and drops low-priority segments if the line overflows (Appendix P), the divider decision compares adjacent backgrounds (Appendix X), and the target encoder emits the markup byte string. On a line of cheap segments this whole slice is tens of microseconds; the expensive-segment case is bounded by the segment, not the renderer. The daemon then `write(2)`s the reply and the client copies it to stdout. Summed, the cheap-segment warm path is comfortably under a millisecond end to end, and the honest caveat is the one Chapter 5 already made: a network or disk segment on the interactive path lands its own latency *inside* the third slice, because the daemon samples on demand rather than in a background thread. The budget is small everywhere the port controls it, and where it is not small, it is not small for a reason the theme author can see and move.

Chapter 6 — Themes and Colorschemes

This is the compatibility contract made concrete. `powerliners` loads the user's existing `powerline/config`, `themes/`, and `colorschemes/` JSON trees and resolves them exactly as upstream does. This chapter walks the cascade.

The config cascade: `_find_config_files`, `load_json_config`, `mergedicts`

Configuration in `powerline` is not a single file but a *cascade* of files found across a search path and merged, and reproducing the cascade exactly is what lets an existing `~/.config/powerline` tree work unchanged. The search order, documented in the comprehensive man page and honored by the daemon and direct renderer, is: first `$XDG_CONFIG_HOME/powerline/` (falling back to `~/.config/powerline/`); then the colon-separated roots in `$POWERLINE_CONFIG_PATHS`; then system-wide `/etc/powerline/`; and finally the bundled defaults shipped inside the `powerliners` package itself. “First hit wins per key” — a value defined in your user config beats the same key in the system config beats the bundled default.

`_find_config_files` (at `src/ported/mod.rs:104`) is the function that locates a named config file across the search paths. Its logic is a faithful port: append `.json` to the config name, walk the search paths, collect every path where that file exists, and — matching upstream's behavior — raise an error naming the searched paths if none is found. Note that it collects *all* matches, not just the first; the merging happens afterward, so a `themes/tmux/default.json` present in both your user tree and the bundled defaults yields two paths whose contents are then merged, rather than one shadowing the other wholesale.

`load_json_config` (at `src/ported/lib/config.rs:48`) is the loader — it reads a single JSON file and parses it into the in-memory `Value` tree, returning an error on malformed JSON (its tests assert both the happy parse and the error-on-bad-JSON path). It is the narrow, boring function that every config read goes through, and its faithfulness matters precisely because it is boring: the same JSON must parse to the same structure it did under Python, including the handling of the config's quirks, so that everything downstream sees what upstream saw.

`mergedicts` (at `src/ported/lib/dict.rs:117`) is where the cascade becomes a single effective config, and it is a *deep* merge, not a shallow one. For each key in the incoming dict: if both sides hold an object, recurse and merge them; if the incoming value carries the removal sentinel (`__powerliners_remove_this_key__`, the port's spelling of upstream's REMOVE marker) and removal is enabled, delete the key; otherwise overwrite. This recursion is what lets a user config override *one nested field* of a theme — a single segment's `args`, a single color — without restating the whole structure around it, and the REMOVE sentinel is what lets a user *delete* an inherited key rather than only override it. The port also carries `mergedefaults` (fill gaps without overwriting, `d1` wins) and `mergedicts_copy` (a non-mutating variant), the same three merge flavors upstream uses for different phases of config assembly.

Colorscheme alias chasing and cterm color resolution

A colorscheme maps highlight-group names to colors, but the mapping is indirect in two ways, and both indirections are ported faithfully. The first is *named colors*: a colorscheme does not write `#ac4242` on a group, it writes a color name like `red`, and a separate `colors.json` maps `red` to a concrete value. This is why a palette change (edit `colors.json`) restyles every group that uses a named color at once — the groups reference names, the names reference values, and only the values change.

The second indirection is that every color is *two* values, not one — a 256-color terminal index (cterm) and a 24-bit hex value — because the same colorscheme must render on both a truecolor terminal and a 256-color one. The `Colorscheme` constructor (in `src/ported/colorscheme.rs`) builds `self.colors` as a dict of `(cterm_int, hex)` tuples, and it handles both input shapes upstream allows: a color given as `[cterm, "#hex"]` is used directly, while a color given as a bare cterm integer has its hex *derived* from a lookup table. That table is `cterm_to_hex` (at `src/ported/colorscheme.rs`), a 256-entry `const [u64; 256]` that is a line-for-line port of upstream's module-level table — the canonical mapping from an 8-bit terminal color index to its 24-bit RGB value, laid out in the same rows as upstream (`0x000000`, `0xc00000`, `0x008000`, ...). Carrying this table is what lets powerliners render a 256-color-only theme on a truecolor terminal and vice versa without the author maintaining two palettes.

Alias chasing is the third piece. A colorscheme may define a group as an *alias* of another group rather than a set of colors, and resolution must follow the alias chain until it reaches a group with real properties. `get_group_props` (at `src/ported/colorscheme.rs:242`) does this resolution, consulting the mode translations along the way (below), and it is what `get_highlighting` calls per group. The aliasing lets a colorscheme say “the `branch_dirty` group looks like the `warning` group” without duplicating the warning's colors — and if you later restyle `warning`, `branch_dirty` follows, because it was an alias, not a copy.

Colorscheme::get_highlighting and highlight-group lookup

`get_highlighting` (at `src/ported/colorscheme.rs:292`) is the function every segment render calls to turn its list of highlight groups into concrete `{fg, bg, attrs}`, and its structure is a faithful port worth walking. It takes the segment's `groups` (in priority order), the current `mode` (for per-mode overrides), and an optional `gradient_level`. It first looks up the mode's `translations` — the per-mode overrides that let, say, insert mode recolor the whole line. Then it walks the groups in order, calling `get_group_props` for each, and *stops at the first one that resolves*:

```
let mut group_props: Option<Value> = None;
for group in groups {
    let gp = self.get_group_props(mode, &trans, &Value::String(group.clone()), true);
    if gp.is_some() { group_props = gp; break; }
}
let group_props = group_props.ok_or_else(|| format!(
    "Highlighting groups not found in colorscheme: {}", groups.join(", ")
));
```

This first-match-wins walk is the fallback mechanism of Chapter 2 made concrete: a segment offers `["branch_dirty", "branch"]`, and a colorscheme that defines only `branch` skips past the missing `branch_dirty` and resolves `branch`. If *none* of the groups resolves, the function errors with a message naming all the groups it tried — the same loud failure upstream produces, which is what `powerline-lint` (Chapter 9) exists to catch before it reaches a live prompt.

Once a group's properties are in hand, `get_highlighting` extracts the foreground and background color names and resolves each through a `pick` closure: with no gradient level, it looks the color name up in `self.colors` (yielding the `(cterm, hex)` tuple); with a gradient level, it calls `get_gradient` to select a color from the group's gradient list at that level. The attributes — bold, italic, underline — are read from the

group's `attrs` array and packed into an integer flag by `get_attrs_flag`. The function returns a map of `{fg, bg, attrs}`, and that map is what the renderer (Chapter 7) hands to its target-specific encoder. Everything about how a segment gets its color runs through these dozen lines, and their fidelity to upstream is why a colorscheme resolves identically in both implementations.

Per-mode recoloring: `mode_translations` walked

The one piece of `get_highlighting` that a walkthrough clarifies is the *mode* handling, because it is how the same line renders differently depending on what the editor or shell is doing. A colorscheme's `mode_translations` key (Appendix B.5) is a map from mode name to a pair of overrides — a `groups` map that redefines specific groups for that mode, and a `colors` map that remaps color names for that mode. Consider a vim colorscheme with:

```
{ "groups": { "mode": { "fg": "white", "bg": "darkblue", "attrs": ["bold"] } },
  "mode_translations": {
    "i": { "colors": { "darkblue": "darkgreen" },
          "groups": { "mode": { "fg": "white", "bg": "darkgreen", "attrs": ["bold"] } } }
  }
```

In normal mode (`mode = "n"`), `get_highlighting(["mode"], Some("n"), None)` finds no translation for "n" (the `translations` lookup returns an empty map), so `get_group_props` resolves `mode` straight from `self.groups` to white-on-darkblue. In insert mode (`mode = "i"`), the same call first looks up `translations["i"]`, and now two things happen: `get_group_props` consults the mode's `groups` override *first* (recurring with `translate_colors = false`, `colorscheme.rs:242`), finding the insert-mode `mode` group redefined to white-on-darkgreen; and the mode's `colors` remap would rewrite any `darkblue` reference to `darkgreen` for groups that fall through to the base definition. The net effect is that entering insert mode repaints the mode indicator — and, through the `colors` remap, any group that referenced the recolored name — without the theme touching a single segment. This is the mechanism behind the familiar “the whole prompt turns green in insert mode” behavior, and the port carries it faithfully: the mode's `groups` win over the base `groups`, the `colors` remap applies to the base fall-through, and the segment never knows any of it happened — it asked for the `mode` group and got whatever that group means in the current mode.

Truecolor vs 256-color: the `term_truecolor` path

The final resolution — turning a `(cterm, hex)` pair into actual terminal bytes — depends on whether the target terminal supports 24-bit color, and the choice is governed by a `term_truecolor` flag threaded through the renderers. When truecolor is on, the renderer emits the hex value (the second element of the color tuple); when it is off, it emits the cterm index (the first). This is the payoff of carrying both values through the whole pipeline: the same resolved color can be encoded either way, so one colorscheme serves both a modern truecolor terminal and an older 256-color one with no author intervention.

The gradient path respects the same split. `pick_gradient_value` (at `src/ported/colorscheme.rs:72`) selects an index into a gradient list from a 0-to-100 level — `grad_list[int(round(gradient_level * (len(grad_list) - 1) / 100))]`, ported exactly, rounding included — and because the colorscheme built two gradient lists per gradient (a cterm list and a hex list, from the constructor's `(cterm_list, hex_str_list)` handling), the same level picks a matching color in whichever encoding the terminal needs. A `cpu_load_percent` segment at 70% load thus shades correctly whether it lands on a truecolor or a 256-color terminal, because the gradient exists in both encodings and `term_truecolor` chooses between them at the last step.

This is where the tmux renderer's cterm/truecolor branch (Chapter 7) becomes concrete. The README's parity notes list “cterm/truecolor encoding with falsy-hex fallback” among the byte-identical behaviors, and the falsy-hex fallback is a detail worth naming: when a color's hex value is missing or zero (falsy), the renderer falls back to the cterm index even on a truecolor terminal, matching upstream's exact handling of the degenerate

case. The care taken here — two encodings carried through the whole pipeline, chosen at the last moment, with a defined fallback for the missing-hex case — is representative of the port's method: reproduce not just the common path but the edge cases, because the edge cases are where an “inspired-by” reimplementaion would quietly diverge and a faithful port must not.

A colorscheme, resolved: from `colors.json` to `{fg, bg, attrs}`

The abstractions of this chapter are best trusted by watching one highlight group resolve all the way to concrete numbers. Start with three small files, each in the shape the port's constructor reads. First `colors.json`, the palette:

```
{
  "colors": {
    "gray0": 236,
    "white": [231, "ffffff"],
    "orange": [166, "d75f00"]
  },
  "gradients": {
    "cpu_load_percent_gradient": [
      [154, 148, 142, 136, 130],
      ["9fd700", "afaf00", "d7af00", "d78700", "d70000"]
    ]
  }
}
```

`Colorscheme::new (src/ported/colorscheme.rs:109)` reads `colors` into `self.colors` as `name → [cterm, hex]` pairs. Note the two input shapes it accepts: `gray0` is a bare cterm integer, so its hex is *derived* from the `cterm_to_hex` table (entry 236 is `0x303030`); `white` and `orange` give both values explicitly, used verbatim. The gradient reads as two parallel lists — a cterm list and a hex list — into `self.gradients["cpu_load_percent_gradient"]`.

Second, the colorscheme's `groups`:

```
{
  "name": "Demo",
  "groups": {
    "cpu_load_percent_gradient": { "fg": "white", "bg": "gray0", "attrs": ["bold"] },
    "cpu_load_percent": { "fg": "white", "bg": "gray0", "attrs": [] },
    "branch_dirty": "warning",
    "warning": { "fg": "white", "bg": "orange", "attrs": [] }
  }
}
```

Now trace `get_highlighting(["cpu_load_percent_gradient", "cpu_load_percent"], None, Some(71.0))` — the call the `cpu_load_percent` segment of Chapter 2's worked example makes. It looks up the mode's `translations` (none, so an empty map), then walks the groups in order. `get_group_props` resolves `cpu_load_percent_gradient` in `self.groups` on the first try, yielding `{fg: "white", bg: "gray0", attrs: ["bold"]}`; the loop stops there and never consults the fallback `cpu_load_percent`. Because a `gradient_level` of `71.0` was passed, `fg` and `bg` are *not* looked up in `self.colors` — instead the gradient path runs. `pick_gradient_value` computes the index `round(71.0 * (5 - 1) / 100) = round(2.84) = 3` (its `py_round` uses banker's rounding to match Python 3's `round`), selecting element 3 of each gradient list: cterm `136`, hex `0xd78700` — a warm orange. The `attrs` array `["bold"]` runs through `get_attrs_flag`, which bit-ORs `ATTR_BOLD` (`1`) to produce the integer `1`. The function returns `{fg: [136, 0xd78700], bg: [136, 0xd78700], attrs: 1}` — concrete `(cterm, hex)` pairs plus a packed attribute flag, which is exactly what the renderer of this chapter turns into terminal bytes.

The alias case is the other half of the mechanism. Resolving `branch_dirty` finds a *string* value, `"warning"`, in `self.groups` — an alias, not a property object — so `get_group_props` recurses on `"warning"`, which resolves to real properties `{fg: "white", bg: "orange", attrs: []}`. That is why restyling `warning` in one place restyles `branch_dirty` too: the dirty-branch group was never a copy of the warning colors, it was a pointer to them, chased at resolution time. One palette file, one colorscheme file, and a dozen lines of resolution turn a segment's request for a highlight group into the four numbers the terminal actually needs.

Chapter 7 – Renderers

A resolved segment list still has to become bytes for a specific target: tmux markup, a zsh P51, a bash prompt, a vim statusline. This chapter covers the renderer loop and the per-target emitters.

The render loop: `do_render`, `_render_segments`, `_render_length`

`do_render` (at `src/ported/renderer.rs:792`) is the top of the render loop — the function that takes a resolved segment list and produces the final byte string for a target. It is a faithful port, and its `// py:NNN` citations map it line-by-line onto upstream's `Renderer.do_render`. Its job is layout: given the segments (already carrying resolved contents and `{fg, bg, attrs}` highlights from Chapter 6), it computes divider widths, measures the line, and — when the line must fit a width — runs the truncation-and-priority logic that decides which segments to drop or shorten. The output is a string of the target's markup with every segment, divider, and pad in place.

The width math runs through `_render_length` (at `src/ported/renderer.rs:1075`), which computes the rendered width of the segment list including contents, dividers, and outer padding, and returns the running length. This is the function `do_render` calls repeatedly as it applies the priority-based truncation: a segment can declare a `priority`, and when the line is too wide for the terminal, low-priority segments are removed or truncated until it fits — so a narrow tmux pane drops the least-important segments rather than wrapping or overflowing. `_render_length` is called before and after each such adjustment (its call sites are threaded all through `do_render`, at `py:347`, `py:353`, `py:358`, `py:374`, `py:401`) to re-measure, which is why the port keeps it a separate function: the loop measures, adjusts, and re-measures until the line fits.

`compute_divider_widths` (at `src/ported/renderer.rs:314`) supplies the per-side hard and soft divider widths that the length math needs. It takes a `get_divider(side, kind)` closure — the port's stand-in for upstream's `theme.get_divider` — and returns a map of `{left: {hard, soft}, right: {hard, soft}}`, each value the display width of the corresponding divider string (via `strwidth`, the port's width function at `renderer.rs:298`). The widths are computed once per render and reused, because the divider strings do not change within a render; only *which* divider (hard or soft) goes at each boundary does, and that is decided per boundary in the length loop.

The width calculations lean on `strwidth`, and its behavior is one of the places where the port is honest about a simplification rather than overselling it. A powerline line is full of characters that are not a single byte — the divider chevrons, powerline glyphs, CJK text, emoji in a media segment — and measuring the line as `str.len()` (a byte count) would be catastrophically wrong for any of them, throwing off the truncation math and producing a line that overflows or under-fills. The port's `strwidth` counts *Unicode scalar values* — `s.chars().count()` — which is correct byte-for-byte for the overwhelming common case (ASCII, the powerline glyphs, Latin text) where every scalar is one cell. What it does not yet do is East-Asian *wide-character* dispatch: upstream's `strwidth_ucs_2/strwidth_ucs_4` count a CJK ideograph or a wide emoji as two cells, and the port's char-count returns one for those, so a line that is heavy with wide glyphs can measure a cell short. This is a documented parity gap, exercised (as a fallback path) by `lib_strwidth_fallback.rs`, and it is the kind of edge the port names rather than hides — the width is exact for the character classes a prompt is actually built from, and the wide-glyph case is a known, bounded divergence rather than a silent one.

Hard and soft dividers, outer padding, side handling

The divider logic is where powerline's signature chevrons come from, and the rule is purely color-driven. Inside the render loop, for each pair of adjacent segments, the divider type is chosen by comparing backgrounds: `divider_type = 'soft' if compare_segment['highlight']['bg'] == segment['highlight']['bg'] else 'hard'` (the port reproduces this exactly at `src/ported/renderer.rs:1186`). Same background → soft divider (a thin separator in a muted color, drawn *within* a color block); different background → hard divider (the solid chevron that fills with the neighbor's background, producing the arrow that visually “points” from one color into the next). The theme author never places these; they emerge from the coloring, which is why re-coloring a theme automatically re-flows its dividers.

Outer padding and spaces frame the line. `theme.get_spaces()` supplies the per-segment `divider_spaces` — the blank cells placed around dividers so text does not touch the chevrons — and `outer_padding` is the blank space at the very ends of the line, keeping the first and last segments off the terminal edges. Both feed into `_render_length`'s width accounting, because padding takes cells and the truncation math must know about every cell. The README's parity list names `outer_padding` and `spaces` explicitly among the byte-identical behaviors, which is the point: even the whitespace is faithful, because a prompt that padded differently would render one cell off and no longer be a drop-in.

Side handling is the last layout concern. A powerline line has a left side and a right side (and, for statuslines, a gap between them), and the two sides render with mirrored divider orientation — the left side's hard dividers point right, the right side's point left, so both “flow” toward the center gap. The renderer handles each side, including the empty-side case (a line with only left segments, or only right), which the README calls out among the parity scenarios (“left/right side handling, empty sides”). The `before/after` wrapping — strings a theme can wrap around a segment's contents — is applied here too, and it too is on the byte-identical list. The whole of this layout logic is inherited; what powerliners adds is that it runs in a native binary in microseconds rather than in a freshly-booted interpreter.

The TmuxRenderer and its `#[...]` markup

The tmux renderer (`src/ported/renderers/tmux.rs`) is the most-used emitter, because the tmux statusline is where most people first meet powerline, and it is a compact, faithful port. Its central function is `hlstyle` (at `src/ported/renderers/tmux.rs:114`), which turns a resolved `{fg, bg, attrs}` into tmux's `#[...]` style directive. The encoding mirrors upstream line-for-line: a foreground of `default` emits `fg=default`; a truecolor foreground emits `fg=#{0:06x}` (the six-hex-digit form); a 256-color foreground emits `fg=colour` followed by the cterm index. Background is the same with `bg=`. The finished directive is `#[` plus the comma-joined attributes plus `]` — for example `[fg=#ac4242,bg=#181818,bold]`.

The attribute handling is its own small port. tmux does not have a “no attributes” state that clears the previous style, so the renderer must *explicitly* turn attributes off when a segment lacks them: `attrs == None` returns `["nobold", "noitalics", "nounderscore"]`, and a segment that is bold-but-not-italic emits `nobold` cleared to `bold` and `noitalics` for the italic slot. This is why the tmux attribute list carries the `no*` forms — without them, a bold segment would leave every following segment bold until something explicitly turned it off, because tmux markup is stateful. The port reproduces this exactly (the `attrs` handling at `tmux.rs:29-64`), because getting it wrong would produce the classic “everything after the bold segment is bold” bug.

Character translation is the last tmux-specific wrinkle, and it is the reason a literal `#` in your prompt does not break tmux markup. In tmux's status format, `#` is the escape character that introduces `#[...]` and `#{...}` — so a `#` in segment *contents* (a channel name, a path) must be doubled to survive. The renderer's `character_translations` maps `#` to `##[ ]` (the port's `vec!(['#', "##[ ]")`) at `tmux.rs:104`, citing upstream's `character_translations[ord('#')] = '##[ ]'`, and the README's parity notes list this escaping (`#` → `##[ ]`) among the byte-identical behaviors. It is a one-character rule, but it is exactly the kind of detail that separates a port from an approximation: an implementation that forgot to double `#` would render most prompts correctly and then mangle any prompt containing a hash.

Two segments, rendered to tmux bytes

Take the two-segment left side from the worked examples of Chapters 2 and 6: a `hostname` segment resolved to `{fg: [231, 0xffffffff], bg: [236, 0x303030], attrs: 0}` with contents `"jbox"`, followed by the `cpu_load_percent` segment resolved to `{fg: [231, 0xffffffff], bg: [136, 0xd78700], attrs: 1}` with contents `"71%"`. On a truecolor terminal (`term_truecolor = true`), the renderer walks the list and calls `hlstyle` per segment. For `hostname`, `color_spec` sees a truecolor value and emits `fg=#ffffff` and `bg=#303030`; `attrs` is `0`, so `attrs_to_tmux_attrs` returns the three *off* forms `nobold,nobold,nobold`; the directive is:

```
#[fg=#ffffff,bg=#303030,nobold,nobold,nobold]jbox
```

For `cpu_load_percent`, the foreground is the same white, the background is the gradient orange `#d78700`, and `attrs` is `1` (`ATTR_BOLD`), so the attribute list is `bold,nobold,nobold` — *bold on*, the other two explicitly *off*:

```
#[fg=#ffffff,bg=#d78700,bold,nobold,nobold]71%
```

Two details are load-bearing and both are faithful ports. First, the `no*` forms are always present because tmux markup is *stateful*: without an explicit `nobold` on the `hostname` directive, a `bold` set by an earlier segment would bleed forward — this is the “everything after the bold segment is bold” bug the port exists to avoid. Second, the boundary between the two segments is a *hard* divider, because their backgrounds differ (`0x303030` versus `0xd78700`): `_render_length` compares the two `bg` values, finds them unequal, and selects `divider_type = "hard"`, so the chevron between them is drawn `fg = 0x303030` (the left segment’s background) on `bg = 0xd78700` (the right segment’s background), producing the signature arrow that appears to point the gray block into the orange one. Had both segments shared a background, the same comparison would have chosen a *soft* divider drawn in the muted `divider_highlight` color instead. On a 256-color terminal the only change is at the last step: `color_spec` emits `fg=colour231` and `bg=colour136/bg=colour236` — the cterm indices — instead of the hex forms, chosen by the same `term_truecolor` flag, from the same resolved color tuple.

Shell renderers and the readline escape markers (\x01 / \x02)

The shell renderers face a problem tmux does not: a shell prompt contains *non-printing* bytes — the color escape sequences — and the shell’s line editor (readline in bash, zle in zsh) needs to know which bytes are non-printing so it can compute the prompt’s visible width correctly. Get this wrong and the shell mis-measures the prompt, and long command lines wrap in the wrong place, overwrite the prompt, or corrupt the display on redraw. The mechanism for telling the line editor “these bytes take no space” is a pair of marker bytes wrapped around each escape sequence, and the shell renderers emit them.

Those markers are `\x01` (start of non-printing) and `\x02` (end of non-printing), and they live in the upstream `renderers/shell/readline.py` as the constants `escape_hl_start = '\x01'` and `escape_hl_end = '\x02'`. In powerliners these are two of the three files still tagged NEAR (Chapter 12) precisely because they are pure lipstick — a class shell holding two string constants and nothing else. The Rust port holds the same two byte constants directly on the equivalent renderer struct, without the Python ceremony of a subclass per shell flavor. The per-flavor renderers then override the markers with what their line editor expects: the zsh renderer (`ZshPromptRenderer`) uses `%{ / %}`, the bash renderer (`BashPromptRenderer`) uses `\[ /\]`, and the base readline pair (`ReadlineRenderer`) stays `\x01 / \x02`. The upstream `zsh.py` is the second of the NEAR-tagged class shells, the same zero-function shape as `readline.py`. Either way the principle is identical: mark the non-printing bytes so the line editor measures the prompt correctly.

The bytes make it concrete. A zsh prompt segment resolved to white (`0xffffffff`) on dark gray (`0x303030`), bold, on a truecolor terminal, produces from `shell/mod.rs`’s `hlstyle` the SGR sequence `\x1b[0;38;2;255;255;255;48;2;48;48;48;1m` — a reset (`0`), a truecolor foreground

(38;2;255;255;255), a truecolor background (48;2;48;48;48), and the single bold code (1), the mutually-exclusive attribute chain meaning only one of bold/italic/underline is ever emitted. The zsh renderer then wraps that whole escape in its markers, yielding `%{\x1b[0;38;2;255;255;255;48;2;48;48;48;1m%}` followed by the visible contents. The `%{...%}` is what tells zle “these bytes take zero columns,” so a prompt full of such sequences still measures at its visible width. Swap to bash and the identical SGR sequence is wrapped `\[...\]` instead; swap to a 256-color terminal and the color codes become `38;5;231 / 48;5;236` (the cterm indices) — same structure, different encoding at the last step, exactly as the tmux renderer chose between hex and `colour<n>` earlier in this chapter.

The reason these files are NEAR rather than DONE is a classifier artifact, not a gap, and it is worth stating because it is exactly the kind of thing an honest port documents rather than rounds away. The DONE tier (Chapter 10) requires a citation density and a `/// Port of <py_fn>()` doccomment *per Python function* — but these files contain zero functions with bodies (`py_fn_total == 0`), only class shells with constants. There is no behavior to port because the behavior is the two constants, and the port holds them; the classifier simply cannot promote a 0/0 ratio to DONE. The escape markers themselves are fully present and correct — a shell prompt from powerliners measures exactly as one from Python powerline — which is the only thing that matters for the user. Chapter 12 returns to why “3 files not at DONE” overstates the remaining work.

The shell renderer path is also where the other emitters live for completeness: the vim renderer (`src/ported/renderers/vim.rs`) emits vim’s `%#HighlightGroup#` statusline markup and is exercised by `vim_e2e.rs`; the lemonbar, i3bar, pango-markup, and pdb renderers round out the set under `src/ported/renderers/`. Each is a port of the corresponding upstream renderer, sharing the same `do_render` layout core and differing only in the final encoding — tmux `#[...]`, vim `%#...#`, shell `\x01...\x02`, i3bar JSON, and so on. The layout is computed once, faithfully, and each target’s `hlstyle` translates the resolved colors into that target’s bytes.

Chapter 8 – Bindings

powerliners is only useful when a shell, multiplexer, or editor calls it. This chapter covers the integration surface: how each target is wired to the daemon.

tmux: statusline and continuation lines

tmux is the flagship binding, and the wiring is two lines in `~/.tmux.conf` that most users already have from their Python powerline setup and do not need to change:

```
run-shell "powerline-config tmux setup"
run-shell -b "powerline-daemon -q &>/dev/null || exit 0"
```

The first line runs `powerline-config tmux setup`, which installs the tmux integration — it points tmux's `status-left`, `status-right`, and window-status formats at the `powerline` client so tmux invokes it on every statusline refresh. The second line starts the daemon in the background (`-q` double-forks and detaches), so the client's invocations hit a warm socket. From then on, every `status-interval` tick, and every window switch, tmux runs `powerline tmux left/right/window-status`, the client forwards the request to the daemon, and the daemon returns `#[fg=...,bg=...]...` markup that tmux paints directly into the statusline.

The `tmux setup` step is `include_str!`-bundled and install-method-agnostic as of 0.2.3, which is a small but important packaging detail (Chapter 11). Upstream powerline baked config-file paths at install time, which broke when the tool was moved or installed a different way. powerliners instead embeds the eight tmux conf files — `powerline-base.conf` plus seven version-specific variants for different tmux releases — directly into the binary, and `powerline-config tmux setup` extracts them to `$XDG_CACHE_HOME/powerliners/tmux/` (default `~/.cache/powerliners/tmux/`) on first call. So the same command works identically whether powerliners was installed via `cargo install`, `brew install`, or a manual `cp` into `$PATH` — there is no compile-time path to get wrong.

Continuation lines and per-window rendering fall out of the same mechanism. tmux's window-status format invokes the client once per window, so a session with eight windows produces eight renders per tick — which is exactly the multiplication that made the Python startup tax so painful (Chapter 1) and exactly where the warm daemon pays off most. The migration is confirmed by killing and reattaching tmux (`tmux kill-server`; `tmux new-session`): the status bar looks identical, but the `powerline-daemon` in `ps aux` is now the Rust binary rather than the Python shebang, and the renders are sub-millisecond.

zsh: PS1 / R PROMPT and the precmd hook

The zsh binding is a `precmd` hook that rebuilds the prompt before each render by shelling out to the `powerline` client. The canonical shape, from the comprehensive man page's shell-invocation section, is a function registered with zsh's `precmd` machinery:

```
function powerline_precmd() {
    PS1="$(powerline shell left --last-exit-code=$? ...)"
    R PROMPT="$(powerline shell right ...)"
}
```

The key detail is what the invocation *carries*. `powerline shell left` renders the left prompt; `--last-exit-code=$?` forwards the previous command's exit status so the `last_status` segment can show it (and

the pipe-status variant forwards `$pipestatus` for `last_pipe_status`); the job count and the current vi mode are forwarded the same way. The daemon cannot see any of this — it is a separate process with no view into the calling shell's state — so the binding's job is to gather the shell-local facts and hand them over on the wire, where the shell-status segments of Chapter 3 read them back out of `segment_info`. This is the concrete reason the wire protocol forwards `argv` at all.

`RPPROMPT` gets the same treatment for the right prompt, and because `zsh` redraws `RPPROMPT` on interactive events, the right prompt can trigger renders on its own — which is again where the daemon earns its keep, since each of those redraws would otherwise be a Python interpreter boot. The whole `PS1/RPPROMPT` pair is rebuilt in `precmd`, both sides sub-millisecond against the warm daemon, so a `zsh` prompt that updates on interaction stays instant rather than lagging.

The migration story for `zsh` is the same “change nothing” story as `tmux`. The daemon is started at login (`powerline-daemon -q` in `.zshrc`), the `precmd` hook and the `powerline shell left/right` invocations are unchanged from the Python setup, and the only difference is which `powerline/powerline-daemon` resolves in `$PATH`. Because the client speaks the same wire protocol, you can even leave the Python client in place and only swap the daemon; the client does not know or care which language the daemon behind the socket was written in.

bash: PS1 / PROMPT_COMMAND

The `bash` binding is the same idea as `zsh` with `bash`'s own hook name. `bash` rebuilds its prompt through `PROMPT_COMMAND` — a command `bash` runs before each primary prompt — and the binding registers a function there that sets `PS1` from the `powerline` client's output. As in `zsh`, the invocation forwards the previous exit code and the shell-local state the status segments need, and the client forwards it all to the daemon over the wire.

The one `bash`-specific wrinkle is the `readline` escape marking of Chapter 7. `bash`'s line editor is `readline`, and `readline` needs its non-printing bytes wrapped in `\[ / \]` markers to measure the prompt width correctly (`zsh` uses `%{...%}` instead, and the base `readline` renderer uses the raw `\x01\x02` pair those `\[ / \]` sequences stand in for). The shell renderer emits the right markers for the target, so a `bash` prompt wraps its color escapes in `\[... \]` and `readline` computes the visible width correctly — which is what keeps long command lines from wrapping into the prompt or corrupting the display on history navigation. Get this wrong and `bash` line editing breaks in subtle, maddening ways; the port carries the exact upstream constants so it does not.

`bash`'s continuation prompt (`PS2`) and the shell `continuation` subtype are handled by the same client, invoked with the `continuation` argument. The migration, again, is a no-op on the user's config: the `PROMPT_COMMAND` function and its `powerline shell left/right` calls are unchanged, and only the resolved binary differs. The daemon serves `bash`, `zsh`, `tmux`, and `vim` from the same process simultaneously, so a user who lives in `bash` under `tmux` with `vim` open has one warm daemon handling all of it.

vim statusline

The `vim` binding is the one that most visibly benefits from powerliners not needing Python, because the Python `powerline vim` plugin required a `+python3`-compiled `vim` and a `pip install powerline-status` — a real friction point on minimal or remote `vim` installs. `powerliners`' `vim` support ships in the same `powerline` binary with no `+python3` requirement: the bundled `powerline.vim` plugin is `include_str!`-embedded in the binary and extracted to `~/.cache/powerliners/vim/powerline.vim` on first source, and it works on `vim 7.4+`, `vim 9`, and `neovim`, using only `vim8`-compatible syntax (no `vim9script` lock-in).

Installation is one line in `.vimrc` — `execute 'source' trim(system('powerline-config vim source-path'))` — which extracts the plugin and sources it. The plugin sets `laststatus=2` automatically so the statusline shows even in a single window, and it wires an `augroup powerliners` of autocmds that trigger renders: `VimEnter` for the initial render, `WinEnter/BufWinEnter/BufEnter/TabEnter` for context switches, `ModeChanged` for mode transitions on modern `vim`, and a legacy `CursorMoved/CursorMovedI`

fallback for ancient vim (which is the one to disable if statusline refresh stutters on every keystroke). On each of those events the plugin shells out via `system('powerline vim left ...')`, and — this is the reason the README recommends pre-starting the daemon — with the daemon running that is a microsecond socket round-trip, while without it every refresh fork-execs `powerline-render` cold at ~30 ms.

What the plugin sends per request is chosen to match upstream's `powerline.bindings.vim` exactly, so theme JSON written for the Python vim binding renders identically here. The per-request keys are `mode` (the current vim mode — `n`, `i`, `v`, `R`), `bufnr` (`bufnr('%')`), `winnr` (`winnr()`), and `buf_name` (`expand('%:p')`, omitted when empty). The vim renderer (Chapter 7) turns the resolved segments into vim's `%#HighlightGroup#` statusline markup, and the plugin assigns the result to `statusline`. The statusline theme and colorscheme come from `~/.config/powerline/themes/vim/default.json` and `colorschemes/vim/default.json` — the same tree, same grammar (`segments.left/right`, `extends`, `mode_translations`) as every other target.

ipython / REPL prompts via shell hook

The ipython/REPL binding is the one deliberate non-port in the family, and the README is explicit that it is done “via shell hook, not embedded.” Upstream powerline can embed itself in the ipython process as a Python prompt provider, because upstream *is* Python and lives in the same interpreter. powerliners has no Python runtime and cannot embed in a Python process, so it renders REPL prompts the same way it renders shell prompts — through the external `powerline` client invoked by a hook — rather than as an in-process provider.

This is a place where the port's boundary is honest about a difference in *mechanism* even though the *result* is the same colored prompt. An ipython user gets a powerline prompt; it is produced by the client talking to the daemon rather than by powerline code running inside ipython. For the everyday case this is invisible — the prompt looks and behaves the same — and it keeps powerliners' single-static-binary discipline intact rather than pulling a Python integration back in through a side door. It is a small example of the larger pattern in Chapter 1: where the upstream mechanism depends on being Python, the port reaches the same user-visible result by a different, native path, and says so plainly rather than pretending the mechanisms are identical.

Chapter 9 – Fidelity to Upstream

Compatibility is the product, so this chapter states exactly what “drop-in” means: what is guaranteed identical, what is allowed to differ and why, and how the guarantee is kept honest rather than merely asserted.

What is byte-for-byte identical

The README’s parity notes enumerate the surface that powerliners guarantees identical to Python upstream, and the list is long and specific because vagueness here would defeat the purpose. The markup itself is identical — `tmux #[...]` directives, `vim %#...#`, `shell \x01...\x02` — down to the escaping rules: `#` doubled to `##[ ]` in `tmux`, control characters run through `translate_np`, the non-printing markers around shell escapes. The dividers are identical across every case the code handles: hard, soft, multi- character, empty, and single-character dividers all render the same bytes. The colorscheme resolution is identical: alias chains, fallback groups, gradients, and `cterm/truecolor` encoding with the falsy-hex fallback all resolve to the same colors.

The layout is identical too. Attributes (bold, italic, underline) are bit-packed the same way; `outer_padding`, `spaces`, and left/right side handling produce the same whitespace; empty sides render the same; `before/after` wrapping applies the same; and single-cell Unicode contents measure and truncate the same (with the one bounded exception of East-Asian wide-glyph width, the documented `strwidth` gap of Chapter 7 — the scalar count is exact for every character class a prompt is normally built from). The point of the enumeration is that “identical” is not a hand-wave — it is a checklist of concrete behaviors, each of which the parity harness (Chapter 10) asserts against the live Python interpreter, so that “drop-in” is a claim backed by 462 assertions rather than an aspiration.

The config resolution is identical, which is what makes an existing tree work unchanged. The cascade (`_find_config_files`, `load_json_config`, `mergedicts`) resolves the same files in the same order and merges them with the same deep-merge, REMOVE- sentinel, list-append semantics as upstream; the theme grammar is the same; the segment names are the same. A `~/.config/powerline` tree that renders a certain prompt under Python renders the same prompt under powerliners, and the migration guide’s Step 2 (Chapter 11) is precisely an instruction to *verify* that byte-for-byte before committing to the swap.

Where output may legitimately differ

Honesty about compatibility means naming the places where output can differ, and the README documents exactly three, each an inherent consequence of not running Python’s `psutil`-based sampling rather than a port defect. First, **live-data segments** (`cpu_load_percent`, `network_load`, and the rest) are sampled per-render via subprocess probes in the daemon, while Python uses `psutil` with a different sampling cadence — so the *numeric value* may differ by a sampling-window’s worth of data. The markup framing is identical; the number inside it may be a moment fresher or staler.

Second, **threaded segment caching** differs. Python’s `ThreadedSegment` polls in a background thread and renders the last-known value; powerliners samples on demand. The *content* matches, but the *latency profile* differs — powerliners may block briefly when a network or disk segment samples, where the threaded model would have returned an instant slightly-old value (Chapter 5 discussed the trade-off). Third, **psutil-only features**: Python errors loudly and skips affected segments when `psutil` is missing, while powerliners resolves the same data through OS subprocess probes (`top`, `vm_stat`, `netstat`, `pmset`, `uptime`) and renders successfully. So powerliners is actually *more* robust in the no-`psutil` case — it renders where Python would have skipped — which is a divergence in the user’s favor.

These three are the entire set, and the framing matters: they are the places where byte-identity was never physically possible, because they involve sampling live values at a moment in time, and two different samplers at two different instants will read two different numbers. Everything that is *deterministic* — the markup, the escaping, the colors, the layout, the config resolution — is byte-identical; only the live-sampled numbers, which are inherently timing-dependent, can differ, and only by a sampling window. This is the honest boundary of “drop-in,” stated in the code’s own words.

powerline-lint: catching config errors before they render

Fidelity cuts both ways: powerliners must render a valid config the same as upstream, and it must *reject* an invalid config the same way, so that a config error surfaces as a clear diagnostic rather than a mangled prompt or a silent omission. `powerline-lint` (the binary, mirroring `scripts/powerline-lint`) is the tool for this. It runs the full check pipeline — a `markedjson` loader that preserves source positions, the `Spec` checks that validate structure, and orchestrator integration that resolves the config the way a real render would — and reports authoring errors: unknown segment names, missing color references, malformed `before/after` formatting strings, and the rest.

The lint pipeline lives under `src/ported/lint/` (with `spec.rs`, `checks.rs`, `selfcheck.rs`, `inspect.rs`, `context.rs`, and `imp.rs`), a faithful port of upstream’s lint machinery. Its value in a compatibility story is that it catches the errors that `get_highlighting`’s “groups not found” failure (Chapter 6) would otherwise raise at render time — a colorscheme missing a group a theme references, for instance — and reports them statically, with source positions, before they reach a live prompt. A user migrating a hand-edited config runs `powerline-lint` first and fixes what it flags, rather than discovering the same problems as a broken statusline later.

Linting is also part of how the project keeps its own bundled defaults honest. The test suite includes a self-check path, and the lint checks run against the config the daemon actually resolves, so a bundled theme or colorscheme that referenced a missing group would be caught in CI rather than shipped. Fidelity to upstream includes fidelity in *failure*: the same bad config produces the same class of error, which is exactly what a user who has debugged a Python powerline config before expects, and exactly what makes the migration predictable rather than a source of new mysteries.

Chapter 10 – The Port Methodology

The claim “a faithful port” is only worth what the verification behind it is worth. `powerliners` backs it with a citation rule and a parity harness. This chapter documents how fidelity is *proven*, not asserted.

The citation rule: `// py:NNN` against the upstream source line

The most visible discipline in the `powerliners` source is the citation comment. Nearly every line of a ported function body carries a `// py:NNN` annotation naming the exact upstream source line it ports — you have seen them throughout this book, in `get_highlighting` (py:99, py:100–103, py:104–105, py:107–110), in `mergedicts` (py:38 through py:47), in the `tmux` renderer (py:10, py:16, py:41, py:65), in `daemonize` (sh:321–340). These are not decoration; they are a machine-checkable map from the Rust source to the Python (and C, `sh`: for the shell scripts and daemon) source it descends from, so a reader — or an audit tool — can put the two side by side and verify that the Rust line does what the upstream line does.

A single function shows the discipline in miniature. `mergedicts` (`src/ported/lib/dict.rs:117`) is a deep merge, and its body reads, with citations, roughly:

```
/// Port of `mergedicts()` from powerline/lib/dict.py:33.
pub fn mergedicts(d1: &mut Map<String, Value>, d2: Map<String, Value>, remove: bool) {
    for (k, v2) in d2 {                                     // py:38  for k in d2:
        match d1.get_mut(&k) {
            Some(v1) if v1.is_object() && v2.is_object() // py:40  if k in d1 and ... dict
                => mergedicts(v1.as_object_mut().unwrap(),
                             v2.as_object().unwrap().clone(), remove), // py:41
            _ => mergedicts(...)
                _ if remove && is_remove_sentinel(&v2)      // py:42  elif ... is REMOVE
                    => { d1.remove(&k); }                    // py:43  del d1[k]
                _ => { let mut v = v2; if remove { clear_special_values(&mut v); }
                      d1.insert(k, v); }                     // py:44-47  else: d1[k] = d2[k]
        }
    }
}
```

Set the Rust beside `dict.py` lines 33–47 and every branch lines up: the object/object case recurses (py:40–41), the sentinel case deletes (py:42–43), and the default overwrites after scrubbing nested sentinels (py:44–47). A reader does not have to *trust* that the merge matches upstream — the `// py:NNN` markers say which upstream line each branch descends from, and the two can be read together to confirm it. That is the citation rule doing its job on one function; multiplied across the ported tree, it is a continuous, checkable correspondence between two codebases.

The citations make the port *legible* in a way an uncommented reimplementation never is. When Chapter 6 says `pick_gradient_value` reproduces upstream’s rounding exactly, that is not a claim to take on faith: the function carries the citation `// py:32 grad_list[int(round(gradient_level * (len(grad_list) - 1) / 100))]` right above the Rust that implements it, and you can read both and see they match. Multiply that across the whole ported tree and the citations become a continuous, checkable correspondence between two codebases — which is what turns “faithful port” from a marketing phrase into a property you can inspect line by line.

The citations also serve the port's maintenance. When upstream changes a function, the citations point maintainers at exactly which Rust lines correspond to the changed Python lines, so the port can be updated surgically rather than by re-reading the whole function to find the drift. And they feed the tier classifier below, which measures *citation density* — the fraction of a function's body that is cited — as one of its two criteria for the DONE tier. A function that is genuinely ported line-by-line has high citation density; one that was sketched loosely does not, and the classifier can tell the difference.

The DONE tier: citation density ≥ 0.5 + a `/// Port of <py_fn>()` doccomment

Not every ported function is equal, and the project grades them. The tier classifier (`scripts/gen_port_checklist.py`) assigns each upstream Python function a tier, and the top tier, DONE, requires two things of the corresponding Rust: a `// py:NNN` citation density of at least 0.5 — meaning at least half the function's body lines are pinned to an upstream source line — and a `/// Port of <py_fn>()` doccomment identifying which upstream function this Rust function ports. Both criteria must hold; a function with a doccomment but sparse citations is not DONE, and neither is one densely cited but missing its port doccomment.

The two criteria check different things. The doccomment is a *declaration* — this Rust function claims to port that Python function, by name — which is what the drift gate (below) checks against the canonical function list. The citation density is *evidence* — the port is not just claimed but demonstrated, line by line, to at least the halfway mark. Requiring both means a DONE function is one that says what it ports and shows its work, which is a meaningfully stronger bar than “there is a Rust function with a similar name.” The project's status — 134 of 137 upstream files at DONE — is a count of files where *every* function clears this bar.

The tier system is also what makes the port's honesty auditable rather than self-reported. Anyone can regenerate the per-file tier table from the live source with `python3 scripts/gen_port_checklist.py > docs/PORT_CHECKLIST.md`, and the function-coverage report with `python3 scripts/gen_port_report.py`. The numbers in this book and in the README's badges are not hand-maintained claims; they are outputs of a classifier run over the actual source, which means they cannot drift from reality without the classifier catching it. That is the difference between a project that says it is a faithful port and one that can prove the claim on demand.

The parity harness: 462 assertions run against the live Python interpreter

Citations prove the Rust *looks like* the Python; the parity harness proves it *behaves like* it. The harness — `tests/parity_against_upstream.rs`, `tests/daemon_parity.rs`, and the contract tests — runs 462 assertions in which the real Python interpreter executes the upstream powerline source and the Rust port renders the same input, and the two outputs are compared for byte or value identity. This is not a golden-file comparison against a snapshot someone captured once; it is a live comparison against Python running the actual upstream code, so if upstream's behavior is X, the harness demands the port produce X, and it re-checks that on every CI run.

The daemon parity is the sharpest case, because it exercises the whole pipeline end-to-end. `daemon_parity` covers 45 byte-for-byte scenarios in which a request goes over the wire protocol to the Rust daemon and the rendered markup is compared against what upstream's `powerline-render` produces for the same config and request. The migration guide's Step 2 is the user-facing version of exactly this test: spin up the Rust daemon on a probe socket, send a wire request, and diff the output against `powerline-render tmux right`. If they match for your segments, the port is faithful to *your* config; the harness does the same for 45 representative configs continuously.

The value of testing against the live interpreter rather than a captured snapshot is that it makes the parity *self-correcting*. A snapshot can go stale — upstream changes, the snapshot does not, and the test now

asserts old behavior. Running the real interpreter means the harness always compares against current upstream behavior, so the port is held to what powerline actually does today, not what it did when someone last captured a golden file. This is more expensive (the tests need Python and the upstream source, which is why `vendor/powerline` is checked in as a parity fixture) but it is the only way to make “byte-identical to upstream” a claim that stays true as upstream moves.

The 11 port bugs the harness surfaced and fixed

The harness is not ceremony; it found real bugs, and the project counts them. Eleven port bugs were surfaced by the parity harness and corrected in the Rust port — cases where the Rust looked plausibly like the Python but produced different bytes, and only a live diff against the interpreter caught it. The README notes them (see the git log for the full list), and their existence is the argument for the harness: a port checked only by reading code and running its own unit tests would have shipped these eleven divergences, because each was a place where the code looked right and behaved wrong.

This is the deepest reason the harness runs against the live interpreter rather than the port's own idea of correct. A unit test written by the porter encodes the porter's *understanding* of upstream behavior, and if that understanding was wrong, the test is wrong in the same direction as the code — they agree, and the bug survives. A diff against the actual Python cannot be fooled that way: it does not care what the porter thought upstream did, it runs upstream and compares. The eleven bugs are eleven places where the porter's understanding was subtly off and the live diff caught what a self-authored test would have missed.

The count also functions as an honesty signal. A project claiming a flawless port would be either lying or lucky; a project that says “the harness found eleven bugs and we fixed them” is telling you the harness works and the port is now as good as the harness is thorough. Every one of the eleven is a divergence that no longer exists, and the harness that found them still runs on every push, so the twelfth — if the port ever regresses or upstream shifts under it — gets caught the same way. The number is not an embarrassment; it is the evidence that the verification is real.

The drift gate and `docs/powerline_py_functions.txt`

The last piece of the methodology guards against a slow, silent failure mode: the port falling out of correspondence with upstream's set of functions, not just their behavior. `docs/powerline_py_functions.txt` is the canonical list of every function in the upstream Python source, and the drift gate — the test `ported_fn_names_match_py.rs` — asserts that every function name the port claims to have ported (via its `/// Port of <py_fn>()` doccomments) matches a real entry in that list. A doccomment that names a Python function which does not exist, or misspells one that does, fails the gate.

This catches a class of error the parity harness cannot. The harness checks behavior for the inputs it exercises, but it cannot notice that the port *claims* to port a function that upstream renamed, or that a doccomment points at the wrong upstream function. The drift gate does: it holds the port's declared correspondence against the authoritative function list, so the map from Rust functions to Python functions stays accurate as both sides evolve. The README reports the gate as green — every ported function name matches `docs/powerline_py_functions.txt` — which means the port's self-description of *what* it ports is verified, not just its behavior.

Together the four mechanisms — citations, the DONE tier, the parity harness, and the drift gate — form a defense in depth around the word “faithful.” The citations make the port legible and feed the density metric; the DONE tier grades each function on declared-and- demonstrated correspondence; the parity harness proves behavior against the live interpreter; and the drift gate keeps the declared correspondence honest against the canonical function set. No one of them alone would suffice — citations can be wrong, tiers can be gamed, a harness can miss an untested path, a name list can drift — but together they make “1.1 Rust port” a claim that is checked from four independent angles on every CI run, which is what the phrase has to mean to be worth printing on a badge.

Chapter 11 — Packaging and Deployment

A faithful, fast renderer is only useful if it installs cleanly and swaps in without breaking what was there. This chapter covers the packaging discipline — the single- binary suite, the install paths, and the reversible migration that made the compatibility work of Chapters 6 and 9 pay off in practice.

The five-binary suite as one artifact

`powerliners` publishes as one crate that produces five binaries, declared explicitly in `Cargo.toml` with a `[[bin]]` entry each (`powerline`, `powerline-daemon`, `powerline-config`, `powerline-render`, `powerline-lint`). The explicit declarations exist so `cargo install powerliners` and `cargo publish` ship the *complete* upstream-equivalent CLI surface — install the crate and you get all five, not just a library. The release profile is tuned for a small, fast binary: `lto = "thin"`, `codegen-units = 1`, and `strip = "symbols"`, which is the standard recipe for a production static binary and matches the single-artifact discipline the whole design rests on.

The manifest also carves the repository into what ships and what does not. The `exclude` list keeps `vendor/` (the upstream `powerline` checkout used as a parity fixture), `tests/`, `docs/`, `scripts/` (the Python codegen and audit tools), and `.github/` out of the published crate — so a `cargo install` pulls the Rust source and nothing else, not the megabytes of vendored Python and HTML reports that live in the repo for development. This is the difference between a repository (which carries everything needed to *develop and verify* the port) and a published crate (which carries only what is needed to *build and run* it).

The single-artifact story is the deployment payoff of being native. There is no interpreter to install per machine, no `pip` dependency tree to resolve, no shared library to keep in version-sync with the binary. A container image needs the binaries and your config tree and nothing else; a remote box needs one `scp` or one `brew install`. This is the same deployability that made `awk` and other single-binary tools durable, and `powerliners` keeps it deliberately — the whole point of eliminating the Python runtime is lost if you replace it with a different runtime to install, so the port replaces it with nothing.

Install paths: Homebrew, cargo, and from source

The fastest install is the Homebrew tap, auto-bumped on each release:

```
brew tap MenkeTechnologies/menketech
brew install powerliners
# installs powerline, powerline-daemon, powerline-config, powerline-render, powerline-lint
```

`cargo install powerliners` is the equivalent for a Rust toolchain, and building from source is the third path, useful when testing an unreleased build:

```
cargo build --release --locked \
  --bin powerline --bin powerline-daemon \
  --bin powerline-config --bin powerline-render --bin powerline-lint
```

Release binaries land at `target/release/{powerline,powerline-daemon,powerline-config,powerline-render,powerline-lint}`. All three paths produce the same five binaries; the choice is a matter of which toolchain you already have. The `--locked` flag on the source build pins the exact dependency versions from `Cargo.lock`, which is the reproducibility discipline for a build you intend to trust in place of a working tool.

The install-method-agnostic bundling of Chapter 8 is what makes these paths interchangeable. Because the tmux conf files and the vim plugin are `include_str!`-embedded in the binary and extracted to `~/.cache/powerliners/` on first use, there is no compile-time path baked into the binary that a Homebrew install or a manual `cp` would get wrong. `powerline-config tmux setup` and `powerline-config vim source-path` work identically no matter how the binary got onto the machine — which is a real improvement over upstream’s install-time path baking, and the reason 0.2.3’s move to embedded configs mattered.

The reversible migration

The migration is designed to be tried without commitment and undone without trace, which is the right shape for swapping out a tool you depend on every second. The README’s six steps reduce to: install or build the suite; **verify parity against your own config** before changing anything (spin up the Rust daemon on a throwaway `--socket` and diff its wire-protocol output against `powerline-render tmux right`); stop the Python daemon (`powerline-daemon -k`); symlink the Rust `powerline-daemon` into a `$PATH` directory ahead of the Python one; restart tmux; and confirm the running daemon is the Rust binary (`ps -p $(cat /tmp/powerline-ipc-$UID.pid)`).

The critical property is that the migration touches *only* which binary resolves in `$PATH` — no config edits, no `.tmux.conf` edits, no shell-rc edits. This works because of the wire protocol: the Python C client shipped by `pip install powerline-status` does not need replacing, because it speaks the same protocol to whichever daemon is bound to the socket. You can swap only the daemon and leave the Python client in place, and everything keeps working because the client does not know or care what language the daemon is written in. The compatibility work of Chapters 4, 6, and 9 is what makes a migration this shallow possible — same wire format, same socket path, same config tree, same rendered bytes.

Rollback is symmetric and complete: `powerline-daemon -k` to stop the Rust daemon, `rm` the symlink so `powerline-daemon` re-resolves to the Python script, and `powerline-daemon -q` to start Python again. Because nothing but the symlink changed, there is nothing else to undo — the config is untouched, the tmux and shell config are untouched, and the Python daemon comes back to exactly the state it was in. A migration you can try in a minute and reverse in another is a migration people will actually attempt, and the reversibility is the direct consequence of the port having changed the *implementation* while preserving every *interface*: the wire protocol, the socket path, the config tree, and the rendered output.

Chapter 12 – Status and the Honest Gaps

Every book in this series ends by walking its own boundaries. This chapter states where powerliners stands — 134/137 upstream files at DONE, 2473 lib tests — and is candid about the three remaining files, which are Python `renderer-registry` lipstick with no behavior left to port.

134 / 137 files at DONE (97.8%)

The headline status is 134 of 137 upstream Python files at the DONE tier — 97.8% — with 2473 library tests passing (0 failing, 0 ignored), 462 parity assertions against the live interpreter, and 11 port bugs surfaced and fixed. These numbers are not asserted; they are outputs of the tier classifier and the test runner over the actual source, regenerable on demand (Chapter 10). “At DONE” means every function in the file clears the citation-density-plus-doccomment bar, so 134/137 is a count of files where the port is both declared and demonstrated to be faithful, function by function.

The README states the shape of the remainder precisely: 3 lipstick-only files, 0 files in a “partial/sparse” degraded state, and a green drift gate. The zero degraded files is the number to notice — there is no tier of half-ported files with real behavior missing. Every file with a function body is at DONE; the only files not at DONE are the three below, and they contain no function bodies to port. So “97.8%” understates the behavioral coverage: the behavioral surface — every Python function with a body — is at 100%, and the missing 2.2% is a classifier artifact on files that have nothing to classify.

The test count is worth reading as a claim of its own. 2473 library tests exercise the ported functions in isolation — the merge semantics, the colorscheme resolution, the divider math, the wire parsing, the width fallbacks — and they pass with none ignored, which is the discipline that keeps the port from rotting between parity runs. The parity suite proves behavior against upstream; the library tests prove the port’s own invariants hold; together they are why the status numbers are stable rather than aspirational.

The 3 NEAR files and why 0/0 can’t promote to DONE

The three files not at DONE are `renderers/shell/readline.py` (14 LOC), `renderers/shell/zsh.py` (16 LOC), and `bindings/i3/powerline-i3.py` (52 LOC), and each is NEAR for a classifier reason rather than a port gap. `readline.py` is a class shell holding two string constants — `escape_hl_start = '\x01'`, `escape_hl_end = '\x02'` — and a module-level alias; `py_fn_total == 0`. `zsh.py` is the same shape with different escape constants. Both were discussed in Chapter 7: the escape markers they hold are fully present and correct in the Rust port, carried directly on the `renderer` struct without the Python subclass ceremony, so a shell prompt measures identically.

The classifier tags them NEAR because the DONE tier is a *ratio* — cited-and-doccommented functions over total functions — and these files have zero functions with bodies. A 0/0 ratio cannot be promoted to DONE no matter how complete the port is, because there is nothing in the denominator to be complete *about*. The arithmetic is worth seeing plainly: a normal file with, say, twelve functions all cited above the 0.5 density and all doccommented scores `12/12 = 1.0`, comfortably DONE; a file with eight of twelve cited scores `8/12 = 0.67`, still DONE if each cited function also clears its own density bar; but `readline.py` has `py_fn_total == 0`, so its score is `0/0`, which is not a number the classifier can compare to the `0.5` threshold at all. It is

not that the port fell short of the bar — there is no bar to clear, because the file's entire behavior is two string constants (`escape_hl_start = '\x01'`, `escape_hl_end = '\x02'`) that the Rust holds directly on the renderer struct. The behavior these files carry is not in functions; it is in constants, and the port holds the constants. A classifier amendment could resolve the tag cosmetically, but the project chose to leave it honestly reported — 3 files not at DONE — rather than special-case the classifier to inflate the number, which is itself a small demonstration of the honesty the whole port is built on.

`powerline-i3.py` is the subtler case: it contains one `render` function which is already 100% ported (the port report confirms it), but the file's structure trips the classifier's class-only branch, so it is tagged NEAR despite the function being done. These exist in upstream because Python's renderer registry needs a class per shell flavor to subclass `ShellRenderer` — a language-level requirement of the Python design, not a behavior. The Rust port has no such registry requirement, so it holds the same escape constants and the same `render` logic without the class ceremony. The three NEAR files are, collectively, the places where Python needed structure that Rust does not, and the classifier — measuring Python-shaped structure — cannot score the absence of ceremony as completeness. There is no behavioral surface left in any of them.

Zero Python runtime: what “native binary” buys and costs

The defining constraint — zero Python runtime — is worth accounting honestly, because it buys most of powerliners' value and costs a few specific things. What it buys is the whole performance and packaging story: no interpreter startup on the cold path, sub-millisecond renders on the warm path, a single static binary with no `pip` tree, and a deployment that needs nothing but the binaries and the config. Everything in Chapters 5 and 11 is downstream of there being no Python to boot, import, or install.

What it costs is dynamic extensibility and in-process embedding, and the port pays those costs openly. Upstream's `__import__`-based custom segments — drop a `.py` in `segments/` and reference it by dotted path — cannot work without a Python runtime, so powerliners replaces them with the two subprocess-based `exec` escape hatches of Chapter 3: same expressive power, different mechanism, and one documented highlight-group wrinkle. Upstream's in-process ipython prompt provider cannot embed in a Python REPL without being Python, so powerliners renders REPL prompts through the external client hook (Chapter 8) instead — same result, external mechanism. These are the two places where “native binary” is not a pure win, and the port names both rather than pretending the mechanisms are identical.

The trade is heavily favorable and deliberately chosen. Dynamic Python-module extensibility is a feature a small minority of users exercise, and the subprocess escape hatches cover essentially all of what it was used for; in-process REPL embedding is a niche whose result is reproduced externally. Against those two costs stands the elimination of the startup tax that motivated the entire project. powerliners bets that a shell user would rather have a sub-millisecond prompt and script-based custom segments than a hundred-millisecond prompt and Python-module custom segments, and the whole design follows from taking that bet seriously — including the honesty about what the bet gives up.

Where the port ends and net-new MenkeTechnologies segments begin

The cleanest way to see the boundary between ported and original work is the source tree: `src/ported/` is the faithful port of upstream, and `src/extensions/` is the net-new MenkeTechnologies work, explicitly carved out of the port's fn-name rules by `docs/PORT.md`. The ported tree is graded by the DONE tier and the parity harness; the extensions tree is new code held to the same test discipline but not measured against a Python original, because there is no Python original to measure against. The registry (`ADAPTERS`) merges both into one namespace, so a theme names an extension segment the same way it names a ported one, but the provenance is unambiguous in the layout.

The extensions are a substantial catalogue and they cluster into families. The system-metrics additions — `git_status`, `disk_usage/disk_io`, `gpu_usage_percent/gpu_vram`, `thermal`, `mem_usage`, `process_count` — are richer or more portable takes on what powerline's `psutil`-based segments did, sampled through OS probes so they work without Python's dependencies. The cloud-native additions — `kubecontext`, `containers`, `aws.context`, `gcp.context`, `ci_status` — bring orchestration context into the prompt, mostly through pure filesystem probes of the tools' own config files, with silent fall-through when the tool or context is absent. Each lives in its own `src/extensions/<module>.rs` and follows the same adapter contract, so adding another is the same pattern rather than a special case.

The most distinctive additions are the ones that tie powerliners into the rest of the MenkeTechnologies stack — the cache-stat segments for the sibling tools. `fusevm.jit_cache` reports the Cranelift JIT cache stats (entry count and bytes) for the shared fusevm runtime that powers `zshrs`, `stryke`, and `awkrs`; and `zshrs.rkyv_cache`, `stryke.rkyv_cache`, and `awkrs.rkyv_cache` each stat that tool's authoritative rkyv bytecode archive. They share a uniform four-level resolution chain and an identical `{size}/{bytes}/{logical_size}/{logical_bytes}` token surface, are pure filesystem probes (no subprocess, no daemon RPC), and are unit-tested through pure-functional `default_path_with(get_env, path_exists)` seams so every precedence level is pinned without mutating the environment. A user of the wider stack can put a segment in their prompt that reports the health of their zshrs or awkrs cache — a capability that only makes sense inside this ecosystem, and a clean example of where the faithful port ends and the original, stack-aware work begins.

Chapter 13 — Where powerliners Fits

A closing chapter should place the program in its world, and powerliners' world is two overlapping ones: the powerline ecosystem it descends from, and the MenkeTechnologies stack it belongs to. Understanding it means seeing it in both.

In the powerline ecosystem, powerliners is the answer to a question the ecosystem has lived with for a decade: how do you keep everything powerline gives you — the segmented, colored, context-aware prompt and statusline, and the whole accumulated culture of themes and colorschemes — without paying Python's startup tax on every render? Every prior answer traded something away. Caching helped but did not eliminate startup. Simpler prompts were faster but gave up the features. Rewrites in other languages usually meant a new config format and a migration that abandoned your themes. powerliners' answer is to change nothing you can see and everything you cannot: the same themes, the same grammar, the same wire protocol, the same rendered bytes, running in a native daemon instead of a per-render interpreter. It is a port, not a replacement, and that is precisely why it can drop into a decade-old config without a single edit.

The honesty about that lineage is not a hedge; it is what makes the originality claim survivable. The MenkeTechnologies project bar kills "X but faster" and "X but in Rust" as products, and a faster powerline would die on that bar. powerliners survives it because the faster render is the *floor*, not the pitch — the pitch is the engineering that made a faithful port into a deployable native suite: the wire-compatible client, the double-forking daemon with its abstract-namespace socket, the parity harness that diffs against the live interpreter and caught eleven real bugs, the tier classifier and drift gate that keep the fidelity claim auditable, and the new native segments that extend the library beyond what upstream offers. The segment model is inherited and this book says so on every page; the machine that runs it at native speed, verified against the original, is the original work.

In the MenkeTechnologies stack, powerliners is the prompt and statusline layer, and its extension segments are where that placement becomes concrete. The cache-stat segments for `fusevm`, `zshrs`, `stryke`, and `awkrs` let the prompt report on the health of the very tools the user is running — a fusevm JIT cache size, an awkrs bytecode archive — which is a capability that only makes sense because those tools share an author and an ecosystem. The prompt that powerliners draws is the frame around a session that is likely running `zshrs` as its shell, `zterminal` as its terminal, and `awkrs` in its pipelines; powerliners is the thin, fast, always-present line at the edge of all of it, and its stack-aware segments are how it participates in the ecosystem rather than merely decorating it.

What powerliners finally is, stated plainly, is a demonstration that "faithful port" and "original engineering" are not opposites. The faithfulness is total where it matters — the markup, the colors, the layout, the config resolution, the wire protocol are byte-identical to upstream, proven against the live interpreter — and the originality is real where it counts — the daemon architecture, the native binary suite, the verification harness, and the new segments are work that did not exist before. The Python powerline authors built the model and the culture; powerliners carries both forward, unchanged where a user can see and rebuilt where a user cannot, so that the prompt you spent years tuning renders in under a millisecond and looks exactly the same. That is the whole of it: same prompt, no Python, sub-millisecond, proven. Prompt locked, native.

Appendix A – The Full Segment Catalog

The chapters name segments as they come up; this appendix names all of them, in one place, at the level of detail a theme author needs to *use* one. Everything here is grounded in the `ADAPTERS` registry (`src/bin/shared/render_runtime.rs`, the `pub const ADAPTERS: &[(&str, AdapterFn)]` table around lines 2380–2482) and the adapter implementations under `src/ported/segments/` and `src/extensions/`. For each segment the entry gives its registry key (the dotted path a theme's `function` field uses), what it reads, the `args` it accepts with their defaults, what it emits, the highlight groups it requests, and the condition under which it returns nothing.

Three conventions run through the whole catalog and are stated once here rather than repeated in every entry. **info keys:** adapters read the render environment (`segment_info`) through a small set of keys — `environ` (the forwarded environment map), `getcwd` (the forwarded working directory), and the parsed client arguments `args.jobnum`, `args.last_exit_code`, and `args.last_pipe_status`. **Icons:** a leading `{...-glyph}` or `{icon}` token in a default `format` resolves through `src/extensions/icons.rs`, whose three-tier registry (Nerd Font → Unicode pictogram → ASCII) is selected by `POWERLINERS_ICONS=nerdfont|unicode|ascii`. **Dividers:** most extension segments set `"divider_highlight_group": "background:divider"` so their soft dividers paint in the theme's divider color. Where an adapter “never returns `None`,” it means the sampler falls back to a zeroed reading rather than vanishing; where it returns `None`, the segment is simply absent from the line.

A.1 CPU, load, and uptime

`powerline.segments.common.sys.cpu_load_percent` — CPU utilization as a percentage, the canonical gradient segment. Reads `top -l 1 -s 0 -n 0` on macOS (parsing the `CPU usage:` user+sys fields) and `/proc/stat` on Linux (a 100 ms delta), cached about 1.9 seconds in-process so a burst of prompt redraws does not re-sample. `args: format` (default `"{cpu-glyph} {0:.0f}%"`). Emits one chunk with the percentage substituted into the `{0:.0f} / {0:.1f}` slot. Highlight groups `["cpu_load_percent_gradient", "cpu_load_percent"]` with `gradient_level` equal to the load, so the color shades from calm to hot. Never returns `None` (defaults to `0.0` if sampling fails).

`powerline.segments.common.sys.system_load` — the classic load-average segment, reading `getloadavg`. `args: format` (`"{avg:.1f}"`), `threshold_good` (`1.0`), `threshold_bad` (`2.0`), `track_cpu_count` (`false`), `short` (`false`). Emits one chunk when `short`, otherwise three (the 1/5/15-minute averages). Highlight `["system_load_gradient", "system_load"]`, with the gradient level computed from the thresholds: 0 below `threshold_good`, linearly scaled between, 100 above `threshold_bad`. Returns `None` only if the underlying load read fails.

`powerline.segments.common.sys.uptime` — system uptime, formatted from four part-templates. `args: days_format` (`"{days:d}d "`), `hours_format` (`"{hours:d}h "`), `minutes_format` (`"{minutes:d}m "`), `seconds_format` (`"{seconds:d}s"`), `shorten_len` (`3`, the number of leading nonzero parts to keep). Emits one string; returns `None` when uptime is unavailable.

A.2 Memory and swap

The memory family lives in `src/extensions/mem_usage.rs` under the third-party `powerlinemem.*` namespace (it is a 1:1 port of `mKaloer/powerline_mem_segment`), reading `vm_stat + sysctl -n hw.memsize` on macOS and `/proc/meminfo` on Linux. All four share the highlight groups `["mem_usage_gradient", "mem_usage"]` with `gradient_level = used/total*100`, and none of them returns `None` (a failed read yields a zeroed `MemStats`).

`powerlinemem.mem_usage.mem_usage` — RAM as `USED/TOTAL`. `args: format("{mem-glyph} %s/%s")`, `mem_type` ("used"; one of `used/free/available/total`), `short` (false, toggling Ki/Mi/Gi vs short K/M/G). `.mem_usage_percent` — the percentage variant, `format` default `"{mem-glyph} %d%%"`. `.mem_swap` — swap as `USED/TOTAL`, reading `sysctl -n vm.swapusage` (macOS) or `/proc/meminfo`'s `SwapTotal/SwapFree` (Linux), `format "{swap-glyph} %s/%s"`. `.mem_swap_percentage` — swap percentage, `format "{swap-glyph} %d%%"`.

A.3 Disk and GPU

`powerliners.disk.disk_usage` (`src/extensions/disk.rs`) — a filesystem's used space, reading `df -k -P <mount>` (on macOS `/` is redirected to `/System/Volumes/Data` to read the writable volume). `args: mount ("/")`, `format "{disk-glyph} %s/%s"`, `short` (false). Highlight `["disk_usage_gradient", "disk_usage", "mem_usage_gradient", "mem_usage"]`, `gradient` `used/(used+free)*100`. `.disk_usage_percent` — percentage form, `format "{disk-glyph} {0:d}%"`. `.disk_io` — live read/write throughput, reading `/proc/diskstats` (two 500 ms samples) on Linux and `ioreg -c IOBlockStorageDriver -r` (a cross-call delta) on macOS. `args: device("auto")`, `format "{disk-glyph} {io-glyph} R %s W %s"`, `short` (default true here), `recv_max/sent_max` (`104857600.0` = 100 MiB/s each). Highlight `["disk_io_gradient", "disk_io", "network_load_gradient", "network_load"]`.

`powerliners.gpu.gpu_usage_percent` (`src/extensions/gpu.rs`) — GPU compute percent through a vendor-dispatch chain: `nvidia-smi --query-gpu=utilization.gpu,memory.used, memory.total --format=csv,noheader,nounits`, then `rocm-smi` (Linux), then `intel_gpu_top -j` (Linux), then `ioreg ... -c IOAccelerator` (macOS Apple Silicon). `args: format("{gpu-glyph} {0:d}%"`). Highlight `["gpu_load_gradient", "gpu_load", "cpu_load_percent_gradient", "cpu_load_percent"]` (the CPU groups are a fallback so the segment colors even in a colorscheme that never defined GPU groups), `gradient_level = util`. `.gpu_vram` — VRAM `USED/TOTAL` via the same probe chain, `format "{gpu-glyph}{mem-glyph} %s/%s"`, highlight `["gpu_vram_gradient", "gpu_vram", "mem_usage_gradient", "mem_usage"]`. Neither returns `None`.

A.4 Thermal and processes

`powerliners.thermal.thermal` (`src/extensions/thermal.rs`) — CPU/GPU temperature and fan RPM. On Linux it reads `/sys/class/hwmon/hwmon*/temp*_input` and `fan*_input`, name-filtering the hwmon by family (`coretemp/k10temp/zenpower/cpu_thermal` for CPU, `amdgpu/nouveau/radeon/i915` for GPU); on macOS it reads `sysctl -n machdep.xcpm.cpu_thermal_level` on Intel and derives a chip temperature from `ioreg -r -n AppleSmartBattery` on Apple Silicon (where fan RPM is always unavailable). `args: family("cpu"; cpu/gpu/all)`, `format "{cpu-glyph} %s°C %sRPM"`, or `"{gpu-glyph} %s°C"` for the gpu family), `temp_max` (`95.0`). When the fan reading is absent, the RPM tail (from the second `%s` on) is dropped and the trailing space trimmed. Highlight `["thermal_gradient", "thermal", "system_load_gradient", "system_load"]`, `gradient` `temp/temp_max*100`.

`powerliners.proc.process_count` (`src/extensions/proc_count.rs`) — a POSIX process tally from `ps -eo stat=`, bucketed by the first state character. `args: format("{proc-glyph} {total}")`, `warn_zombie` (default true). Tokens `{total} {running} {sleeping} {zombie} {dwait} {stopped}`.

Highlight `["process_count", "proc"]`, with `"process_count_zombie"` prepended when `warn_zombie` and there are defunct processes — a standing warning in the prompt. Returns `None` only if `ps` fails.

A.5 Version control

`powerliners.vcs.git_status (src/extensions/git_status.rs)` — the richer, MenkeTechnologies-native VCS segment, p10k-style. From the forwarded `cwd` it runs `git status --porcelain=v2 --branch` (one fork for the whole state), plus `git describe --tags --exact-match`, `git rev-parse --short HEAD` (detached HEAD), a `git rev-parse --git-dir` and stat of `MERGE_HEAD/rebase-merge/rebase-apply/CHERRY_PICK_HEAD/REVERT_HEAD/BISECT_LOG` (to detect an in-progress operation), `git stash list`, and `git config --get remote.origin.url` (to pick the GitHub glyph). `args` are all icon overrides: `branch_icon`, `github_icon`, `tag_icon`, `unstaged_icon ("!")`, `untracked_icon ("?")`, `staged_icon ("+")`, `conflict_icon ("~")`, `ahead_icon ("↑")`, `behind_icon ("↓")`, `stash_icon ("*")`, plus `status_colors (false)`. Emits a single chunk: a header (GitHub glyph → branch, optionally `tag` → `tag` → `@shortsha`) followed by count badges in the order action, behind, ahead, stash, conflicts, staged, unstaged, untracked, each shown only when nonzero. Highlight: with `status_colors`, clean → `["git_status_clean", "git_status", "branch_clean", "branch"]` and dirty → the `_dirty` equivalents; without it, `["git_status", "branch"]`. Returns `None` outside a git work tree or when there is nothing to show.

`powerline.segments.common.vcs.branch` — the faithful upstream port, a single chunk holding the branch name. Reads git (`git rev-parse --abbrev-ref HEAD + git status --porcelain`), falling through to Mercurial (`hg branch + hg status`) then Bazaar (`bzr nick + bzr status`). `args`: `status_colors (false)`, `ignore_statuses` (default `[]`, status codes to treat as clean). Highlight `["branch"]`, with `"branch_dirty"` or `"branch_clean"` prepended when `status_colors` is on. Returns `None` outside any VCS repo. `.stash` — the stash count from `git stash list`, highlight `["stash"]`, `None` when there are zero stashes.

A.6 Shell status

These three live in `src/ported/segments/shell.rs` and are the reason the wire protocol forwards the client's parsed arguments. `powerline.segments.shell.jobnum` — the background-job count from `info.args.jobnum`; `args`: `show_zero (false)`; `None` when absent or (zero and not `show_zero`). `.last_status` — the previous command's exit code from `info.args.last_exit_code`; `args`: `signal_names (true`, so exit code `130` renders as `SIGINT`); highlight `["exit_fail"]`; `None` when the code is absent or `0`. `.last_pipe_status` — one chunk per pipeline stage from the `info.args.last_pipe_status` array, each highlighted `["exit_fail"]` (nonzero) or `["exit_success"]` (zero), so a mid-pipeline failure is visible even when the last stage succeeded.

A.7 Identity and environment

From `src/ported/segments/common/env.rs`: `.cwd` — the working directory as one chunk per path component (only the last carries `["cwd:current_folder", "cwd"]`; a `divider_highlight` is set on all); `args`: `dir_shorten_len`, `dir_limit_depth`, `use_path_separator (false)`, `ellipsis`. `.user` — the username, reading `info.envIRON` and `libc::geteuid()`; `args`: `hide_user`, `hide_domain (false)`; highlight `["superuser", "user"]` when `euclid` is `0` (the red-root warning), else `["user"]`. `.environment` — the value of a named variable; `args`: `variable (required)`; `None` when the variable is unset. `.virtualenv` — the active `VIRTUAL_ENV` or `CONDA_DEFAULT_ENV` leaf; `args`: `ignore_venv/ignore_conda (false)`, `ignored_names (["venv", ".venv"])`; `None` when no environment is active.

From `src/ported/segments/common/net.rs`: `.hostname` — the host name from `info.envIRON` (fallback subprocess `hostname`); `args`: `only_if_ssh (false)`, `exclude_domain (false)`. `.internal_ip` — a local interface address, resolving the default-route interface via `netstat -rn -f inet` (macOS) or `ip route show default` (Linux) then parsing `ifconfig`; `args`: `interface ("auto")`, `ip (4)`. `.external_ip` —

the public address via `curl -s https://ipv4.icanhazip.com` (a network round-trip — keep it off the per-keystroke path); no `args`; `None` when the query fails. `.network_load` — DL/UL throughput as two chunks, reading `netstat -ibn` (macOS) or `/sys/class/net/<iface>/statistics/{rx,tx}_bytes` (Linux) across a 500 ms sample; `args`: `interface` ("auto"), `recv_format` ("DL {value:>8}"), `sent_format` ("UL {value:>8}"), `suffix` ("B/s"), `si_prefix` (false), `recv_max/sent_max` (1_000_000.0); highlight per chunk `["network_load_<dir>_gradient", "network_load_gradient", "network_load_<dir>", "network_load"]`.

A.8 Time and battery

`powerline.segments.common.time.date` — the clock, strftime-formatted; `args`: `format` ("%Y-%m-%d"), `istime` (false), `timezone`; highlight `["time", "date"]` when `istime` else `["date"]`; never `None`. `.fuzzy_time` — the human phrasing ("twenty to three"); `args`: `format`, `unicode_text` (false), `timezone`, and `hour_str/minute_str` overrides for the word tables. Both are pure-clock reads and safe on a per-keystroke prompt.

`powerline.segments.common.bat.battery` (`src/ported/segments/common/bat.rs`) — battery charge from `pmset -g batt`. `args`: `format` ("{ac_state} {capacity:3.0%}"), `steps` (5), `gamify` (false, the heart-meter mode), `full_heart` / `empty_heart` ("0"), `online` ("C"), `offline` (" "). Non-gamify highlight `["battery_gradient", "battery"]` with `gradient_level` = `100 - capacity` (so it reddens as it drains); the gamify path emits `battery_full/battery_empty` and `battery_online/battery_offline` variants. `None` when `pmset` fails or there is no battery.

A.9 Media players

The player family (`src/ported/segments/common/players.rs`) mirrors upstream's class hierarchy: each adapter shares `args format` (default `"{state_symbol} {artist} - {title} ({total})"`) and `state_symbols` (override, default `>/-/X/'` for playing/paused/stopped/none), emits a single chunk, requests `["player_<state>", "player"]`, and returns `None` when its player is not running. The members: `.mpd` (reads `mpc`; `args` add `host/port/password`), `.spotify` (macOS `osascript`; Linux returns `None` — no D-Bus path here), `.cmus` (`cmus-remote -Q`), `.mocp` (`mocp -i`), `.rhythmbox` (`rhythmbox-client --print-playing-format`), `.itunes` (macOS `osascript`), `.clementine` (qdbus MPRIS call), and `.dbus_player` (the generic MPRIS path via qdbus; `args`: `player_name` required). `powerline.segments.common.mail.email_imap_alert` — registered with `args username` (required), highlight `["email_alert"]`; note honestly that this is currently a *placeholder* — it emits `"<username>: 0"` rather than performing a real authenticated IMAP probe (the TLS dependency is not wired in), so it is present as an API-compatible stub, not a working mail checker.

A.10 Weather

`powerline.segments.common.wthr.weather` (`src/ported/segments/common/wthr.rs` and `src/extensions/wthr_extensions.rs`) — current conditions and temperature from the OpenWeatherMap API (<https://api.openweathermap.org/data/2.5/weather>), disk-cached, with the location auto-detected or set by query. `args`: `location_query`, `weather_api_key`, `unit` ("C"; C/F/K), `temp_format` (default `"{temp:.0f}<unit>"`), `temp_coldest/temp_hottest` (unit-dependent bounds for the gradient), and an `icons` override map. Emits two chunks — a condition-icon chunk (highlight `["weather_condition_<name>", "weather_conditions", "weather"]`) and a temperature chunk (`["weather_temp_gradient", "weather_temp", "weather"]`, gradient scaled between coldest and hottest). `None` when no weather data is available. Like `external_ip`, this is a network segment for the timer-refreshed statusline, not the keystroke path.

A.11 Cloud and orchestration context

`powerliners.k8s.kubecontext` (`src/extensions/k8s.rs`) — the current `kubectl` context and namespace, via `kubectl config current-context` and `kubectl config view --minify -o jsonpath={..namespace}`, honoring `$KUBECONFIG`. `args: format ("{k8s-glyph} {context}:{namespace}")`, `cli ("kubectl")`, `default_namespace ("default")`, `hide_default (true)`. When `hide_default` and the namespace equals the default, the namespace and one adjacent separator (`://@`) are dropped. Highlight `["kubecontext", "k8s"]`; `None` when `kubectl` is missing or there is no current context.

`powerliners.docker.containers` (`src/extensions/docker.rs`) — container counts via three forks (`docker ps -q`, `ps -aq`, `images -q`). `args: format ("{docker-glyph} {running}/{total}")`, `cli ("docker")`, `show_when_zero (false)`. Tokens `{running}` `{total}` `{images}` `{stopped}`. Highlight `["docker_containers_gradient", "docker_containers", "docker"]`. `None` when the CLI is missing/daemon unreachable, or (unless `show_when_zero`) when there is nothing running and no images.

`powerliners.github.ci_status` (`src/extensions/github_ci.rs`) — the HEAD commit's check-runs. From the cwd it derives owner/repo from `git config --get remote.origin.url` (GitHub only) and the SHA from `git rev-parse HEAD`, then serves from a per-SHA disk cache under `$XDG_CACHE_HOME/powerliners/github_ci/`, falling back to `gh api repos/<o>/<r>/commits/<sha>/check-runs`. `args: format ("{github} {icon} {passed}/{total}")`, `cli ("gh")`, `tty_secs (30)`, and `ok_icon/fail_icon/run_icon`. Tokens `{icon}` `{state}` `{passed}` `{failed}` `{running}` `{total}`. Highlight splits into `github_ci_success/github_ci_failure/github_ci_pending` (with `github_ci` and `information:regular` fallbacks). `None` outside a GitHub tree, when `gh` is missing and no fresh cache exists, or when there are no checks.

`powerliners.aws.context` (`src/extensions/aws_ctx.rs`) — the active AWS profile and region, a pure filesystem probe (no subprocess): environment `AWS_PROFILE/AWS_REGION/ AWS_DEFAULT_REGION`, then `~/.aws/config` (honoring the `[profile NAME]` section quirk). `args: format ("{aws-glyph} {profile}@{region}")`, `hide_default_profile (false)`, `icon`. Tokens `{profile}` `{region}` `{icon}`. Highlight `["aws_context", "aws"]`; `None` in the never-configured case. `powerliners.gcp.context` (`src/extensions/gcp_ctx.rs`) — the active gcloud configuration's project and account, likewise pure-fs: environment `CLOUDSDK_ACTIVE_CONFIG_NAME/CLOUDSDK_CORE_PROJECT/CLOUDSDK_CORE_ACCOUNT`, then `~/.config/gcloud/active_config` and `configurations/config_<NAME>` (`[core]` section). `args: format ("{gcp-glyph} {project}:{account}")`, `hide_account (false)`, `icon`. Tokens `{project}` `{account}` `{config}` `{icon}`. Highlight `["gcp_context", "gcp"]`.

A.12 The MenkeTechnologies stack segments

These tie the prompt to the sibling tools and are pure filesystem probes (no subprocess, no daemon RPC). The four **cache-size** segments share a `{size}/{bytes}/{logical_size}/ {logical_bytes}` token surface (disk-block `du -sh`-style sizes versus stat-style content sums) and a `<segment>_rkyv_cache` → `<segment>` → `information:regular` highlight chain, returning `None` on a missing archive unless `show_when_empty` is set. `.fusevm.jit_cache` (`fusevm_jit.rs`) recursively walks the Cranelift JIT cache root (`$FUSEVM_JIT_CACHE` → `$XDG_CACHE_HOME/fusevm-jit` → `~/.cache/fusevm-jit` → `/tmp/fusevm-jit`), also tracking an `{entries}` count (symlinks counted once, never followed); `format "{fusevm} {count} {entries} {hddisk} {size}"`. `.zshrs.rkyv_cache`, `.stryke.rkyv_cache`, and `.awkr.rkyv_cache` each stat a single `scripts.rkyv` archive through the four-level `$<NAME>_RKYV_CACHE` → `$<NAME>_HOME/scripts.rkyv` → `$XDG_DATA_HOME/<name>/scripts.rkyv` → `~/.<name>/scripts.rkyv` chain. The three **version** segments (`.zshrs.version/.stryke.version/.awkr.version`) parse `<bin> --version`, cache the result in-process for `tty_secs` (default `300`) so the daemon does not fork on every tick, expose `{version}`, and use the `<tool>_version` → `<tool>` → `information:regular` chain.

A.13 The `exec` escape hatch

`powerliners.exec.exec` (and the bare alias `exec`, `src/extensions/exec_segment.rs`) — the user-extensibility segment. It spawns `args.command` with the positional list `args.args` (no shell), and auto-detects the output: stdout beginning with `[` is parsed as a JSON array of segment dicts and used verbatim (full control over `highlight_groups`, `gradient_level`, `divider_highlight_group`, and multi-chunk output); anything else is wrapped as a single plain-text chunk with the `format` template applied (`%s` → stdout, `%%` → literal `%`) and `highlight_groups` attached. `args`: `command` (**required**), `args` (string array), `format`, `highlight_groups`. `None` when `command` is absent, the process exits nonzero, or stdout is not UTF-8. The same code path backs the *dotted-path filesystem dispatch* fallback in `invoke_adapter`: any unresolved dotted `function` name is resolved to `<config_path>/segments/<path>.{sh,py,rb,pl,lua,js,<none>}` — the `SCRIPT_EXTENSIONS` order at `exec_segment.rs:31` — with the first match winning, and `args/format/highlight_groups` passed through exactly as for the explicit adapter.

A.14 Reading the catalog as a whole

Two patterns are visible once the whole table is in front of you, and both are design, not accident. The first is the **cost gradient**: identity and time segments read only in-process state (microseconds); the system-metrics segments fork a probe or read `/proc` (sub-millisecond to a few milliseconds); and a handful — `external_ip`, `weather`, the network players — make network or authenticated calls that can block. The catalog is meant to be read with placement in mind, keeping the third class off the per-keystroke prompt and on the timer-refreshed statusline. The second is the **graceful-absence convention**: nearly every segment has a documented `None` condition, and a segment that returns `None` costs almost nothing, which is what lets a theme list two dozen segments and pay only for the handful that have something to say in the current context. The library is broad *because* absence is cheap.

Appendix B – The Config Schema Reference

Chapter 6 walks the *cascade*; this appendix is the *schema* — the shape of each JSON file the cascade resolves, the keys each level reads, and the exact merge and search rules, so a theme author can read a config tree with the same model the loader uses. Everything here is grounded in `src/ported/mod.rs` (search path), `src/ported/lib/dict.rs` (merge), `src/ported/lib/config.rs` (load), `src/ported/theme.rs` (theme grammar), `src/ported/colourscheme.rs` (colourscheme grammar), and the bundled tree under `src/ported/config_files/`.

B.1 The search path, exactly

`get_config_paths` (`src/ported/mod.rs:278`) builds the ordered list of roots the cascade walks. Reading it from the code, the effective order (index 0 searched first) is:

1. `config_files` — the plugin/bundled-defaults path, inserted first (`mod.rs:304`). This is where `bundled_config.rs` extracts the shipped tree, so the defaults are always on the path.
2. **Reversed** `$XDG_CONFIG_DIRS` — each entry as `<dir>/powerline`, colon-split, empties filtered, inserted front-to-back in reverse (`mod.rs:300`). This is the system-wide tier; `DEFAULT_SYSTEM_CONFIG_DIR` is `None` by default (`src/ported/config.rs:148`), so this tier is empty unless `$XDG_CONFIG_DIRS` is set.
3. `$XDG_CONFIG_HOME/powerline` (default `$HOME/.config/powerline`) — the user tree, the initial seed of the vector (`mod.rs:283`).

Within a root, `_find_config_files` (`mod.rs:104`) appends `.json` to the requested config name and collects every root where that file exists — not the first, all of them — so a `themes/tmux/default.json` present in both the user tree and the bundled defaults yields two paths, later merged rather than one shadowing the other. When no root has the file it raises "Config file not found in search paths (...): `<file>.json`", the same loud failure upstream produces. `load_json_config` (`src/ported/lib/config.rs:48`) reads and parses a single file into a `serde_json::Value`, erroring with "open_file: `<e>`" or "json parse: `<e>`".

B.2 The merge primitives

`src/ported/lib/dict.rs` supplies three merge flavors used at different phases of config assembly. `mergedicts(d1, d2, remove)` (`dict.rs:117`) is the deep merge that makes the cascade a single effective config: for each key in `d2`, if both sides are objects it recurses; if `remove` is on and `d2[k]` is the removal sentinel it deletes the key; otherwise `d2` wins on the scalar. `mergedefaults(d1, d2)` (`dict.rs:153`) is the gap-filler — recurse on object collisions, otherwise `d1` wins (`setdefault` semantics) — used to lay defaults under a config without overriding it. `mergedicts_copy(d1, d2)` (`dict.rs:192`) is the non-mutating variant (clones `d1`, `d2` wins, no sentinel handling).

The **removal sentinel** is worth stating exactly, because it is the port's spelling of upstream's `REMOVE` marker. `REMOVE_THIS_KEY()` (`dict.rs:34`) returns the JSON value `{"__powerliners_remove_this_key__": true}`, and a key in `d2` whose value is that object is *deleted* from `d1` during a `mergedicts` with `remove`.

enabled. This is how a user config *removes* an inherited key rather than only overriding it — for example, deleting a segment the bundled default placed on the right side:

```
{ "segments": { "right": { "__powerliners_remove_this_key__": true } } }
```

B.3 config.json – the main config

The root file. Its keys, from the bundled `config_files/config.json`:

- `common` — cross-extension settings, e.g. `{ "term_truecolor": false }`.
- `ext` — a per-extension map. Each of `shell`, `tmux`, `vim`, `ipython`, `pdb`, `wm` carries a `colorscheme` (name), a `theme` (name), and optionally `local_themes` (a `{matcher_or_key → theme_name}` map, used for the window-lister and per-buffer themes) and an `update_interval`.

A minimal user override that only swaps the shell colorscheme need restate nothing else, because `mergedicts` merges it into the bundled `config.json` key by key:

```
{ "ext": { "shell": { "colorscheme": "solarized" } } }
```

B.4 colors.json – the palette

Two keys, read by `Colorscheme::new`:

- `colors` — `{name → cterm}` or `{name → [cterm, "hexstr"]}`. A bare integer means “derive the hex from the `cterm_to_hex` table”; a two-element array supplies both. This is the indirection that lets a colorscheme name “red” and a palette edit restyle every group that uses it.
- `gradients` — `{name → [cterm_list, hex_str_list]}`, two parallel lists so a gradient exists in both 256-color and truecolor encodings (Appendix C).

B.5 colorschemes/*.json – highlight groups

A colorscheme maps highlight-group names to appearance. Keys:

- `name` — a display label (“Default”).
- `groups` — `{group_name → {fg, bg, attrs}}` for a concrete group, or `{group_name → "other_group"}` for a string alias (chased at resolution, Chapter 6). `attrs` is an array drawn from “bold”, “italic”, “underline”.
- `mode_translations` — `{mode → {"groups": {...}, "colors": {old → new}}}`, the per-mode overrides that let, say, vim’s insert mode recolor the whole line. Read into `Colorscheme.translations` and applied by `get_group_props`.

The cascade assembles a colorscheme from three levels (`load_colorscheme_config_levels`, `mod.rs:935`): `colorschemes/<name>`, `colorschemes/<ext>/__main__`, and `colorschemes/<ext>/<name>`, with level 1 (`__main__`) on the ignore list for the top-level pass — so an extension’s `__main__` supplies shared groups that the named scheme extends.

B.6 themes/*.json – the line grammar

A theme describes the segments and their layout. Keys read by `Theme` (`src/ported/theme.rs:204`):

- `segments` — `{ "left": [ ... ], "right": [ ... ] }`, each entry a segment spec with `function` (the dotted adapter name), `type` (“function”/“string”/“segment_list”, defaulting to function), op-

tional `priority` (an integer for truncation ordering), `args`, `before/after` (strings wrapped around the contents), `width/align/expand` (for spacers), and `name`.

- `segment_data` — `{segment_name → {before, after, contents, args}}`, a table of per-segment overrides keyed by short name ("branch") or fully-qualified path. This is how a theme tweaks one segment's arguments without inlining them at the use site.
- `dividers` — `{"left": {"hard", "soft"}, "right": {"hard", "soft"}}`, the divider glyphs per side and kind, read by `get_divider` (`theme.rs:307`).
- `spaces` (int, read by `get_spaces`), `outer_padding` (int, default 1), `cursor_space/cursor_columns` (optional).

One honest note for authors coming from upstream: the top-theme `extends` inheritance key is *not* implemented in the port's `Theme` — none of the bundled themes use it, and `src/ported/theme.rs` has no `extends` handling. Theme composition in the bundled tree is done through the cascade's level-merging (`load_theme_config_levels`, `mod.rs:956`, appends `themes/<ext>/<name>` with levels 0 and 1 ignored) rather than an in-file `extends`. If a migrated theme relies on `extends`, that is a place to verify with `powerline-lint` and the parity check of Chapter 11 before committing to the swap.

B.7 The bundled tree

`src/extensions/bundled_config.rs` embeds the whole default config tree at compile time — `const BUNDLED: &[(&str, &str)]` (line 29) is a table of 46 `(relative_path, include_str!(...))` pairs, pinned to exactly 46 by the `all_46_files_extracted` test — and `extract()` writes them to `$XDG_CACHE_HOME/powerliners/config_files/` on first use, overwriting each run so a version bump refreshes the defaults. This is the same install-method-agnostic embedding used for the tmux conf files and the vim plugin (Chapter 8): the defaults travel *inside* the binary, so there is no compile-time path for a Homebrew or manual install to get wrong.

Appendix C – The Color and Highlight-Group Reference

Color resolution is the part of powerline most people never look inside, and it is worth a reference of its own because so much of the “drop-in” guarantee lives here: two implementations resolve a highlight group to the same bytes only if they carry the same tables and apply them in the same order. This appendix documents the constants, the `cterm_to_hex` table, the attribute packing, the gradient math, and the standard highlight-group vocabulary — all grounded in `src/ported/colourscheme.rs`.

C.1 The two-value color model

Every color in powerline is *two* values: a 256-color terminal index (`cterm`) and a 24-bit hex value, carried together as a `[cterm, hex]` pair through the whole pipeline. The reason is that the same colorscheme must render on both a truecolor terminal and a 256-color one, and the choice between the two encodings is deferred to the very last step — the renderer’s `term_truecolor` flag (Appendix D) picks the hex form when truecolor is on and the `cterm` index when it is off. `Colourscheme::new(colourscheme.rs:109)` accepts a color as either `[cterm, "hexstr"]` (both given) or a bare `cterm` integer (hex *derived*), so an author can maintain a 256-color-only palette and still render on a truecolor terminal, or vice versa.

The **falsy-hex fallback** is the defined edge case: when a color’s hex value is missing or zero, the renderer falls back to the `cterm` index even on a truecolor terminal. This is why the README lists “`cterm/truecolor` encoding with falsy-hex fallback” among the byte-identical behaviors — the degenerate case has a defined answer, and the port reproduces it.

C.2 The `cterm_to_hex` table

`cterm_to_hex(colourscheme.rs:358)` is a 256-entry `const [u64; 256]` mapping an 8-bit terminal color index to its 24-bit RGB value, a line-for-line port of upstream’s module-level table. Its structure is the standard xterm 256-color layout: indices 0–15 are the system colors, 16–231 are a 6×6×6 RGB cube, and 232–255 are a 24-step grayscale ramp that begins at `0x080808` (index 232) and ends at `0xffffffff` (index 255). Carrying this table verbatim is what lets the constructor derive a hex from a bare `cterm` and lets the renderer resolve a 256-color-only theme on a truecolor display; get one entry wrong and a palette shifts by a shade in one encoding but not the other, which a parity diff would catch immediately.

C.3 Attribute packing

Text attributes are a bitfield. The constants (`colourscheme.rs:30,34,38`) are `ATTR_BOLD = 1`, `ATTR_ITALIC = 2`, `ATTR_UNDERLINE = 4`, and `get_attrs_flag(colourscheme.rs:43)` bit-ORs them by scanning a group’s `attrs` array for the strings “`bold`”, “`italic`”, “`underline`”. So `["bold", "underline"]` packs to `1 | 4 = 5`, and an empty array packs to `0`. The renderers unpack this integer back into their target’s attribute syntax (Appendix D) — tmux’s `bold/nobold` pairs, vim’s `cterm=bold`, the shell’s SGR codes — so the *same* packed flag drives every target consistently.

C.4 Gradients and the rounding rule

A gradient is a list of colors selected by a 0-to-100 level. `pick_gradient_value` (`colorscheme.rs:72`) computes the index as `grad_list[round(gradient_level * (len - 1) / 100)]` — the level is *not* bounds-checked, matching upstream. The load-bearing detail is the rounding: the port's `py_round` (`colorscheme.rs:82`) wraps Rust's `round_ties_even` so that `round(2.5)` is 2, not 3, matching Python 3's banker's rounding exactly. This is precisely the kind of edge that a naive port gets wrong — Rust's default `f64::round` rounds half *away from zero* — and it is why the port carries a dedicated helper rather than calling the standard method. Because a gradient is built as two parallel lists (a cterm list and a hex list, from the constructor's `[cterm_list, hex_str_list]` handling), the same computed index picks a matching color in whichever encoding the terminal needs. `get_gradient` (`colorscheme.rs:208`) maps `pick_gradient_value` over both sub-lists to produce the final `[cterm, hex]` pair, falling back to a flat color of the same name if the gradient is undefined.

C.5 Resolution order in `get_highlighting`

`get_highlighting` (`colorscheme.rs:292`) is the function every colored segment calls. Its order is: (1) look up the mode's `translations` (default empty); (2) walk the segment's `groups` in priority order, calling `get_group_props` for each and stopping at the first that resolves; (3) if none resolves, error with "Highlighting groups not found in colorscheme: <joined>"; (4) extract `fg` and `bg` — from `self.colors` when there is no gradient level, from `get_gradient` when there is — and pack `attrs` via `get_attrs_flag`; (5) return `{fg, bg, attrs}`. `get_group_props` (`colorscheme.rs:242`) is the recursive resolver: a string group value is an alias (recurse), an object with `translate_colors` applies the mode's `colors` remap to `fg/bg`, and a missing group returns `None` so the next group in the segment's list gets its turn. The first-match-wins walk is exactly the graceful-degradation mechanism of Chapter 2: a segment offering `["branch_dirty", "branch"]` still colors under a scheme that defined only `branch`.

C.6 The standard highlight-group vocabulary

Themes and colorschemes agree on group *names*, and a working knowledge of the common ones makes a colorscheme readable at a glance. The names below are drawn from the highlight groups the catalogued adapters actually request (Appendix A) and the bundled colorschemes.

- **Segment-specific groups:** `cpu_load_percent` / `cpu_load_percent_gradient`, `system_load` / `system_load_gradient`, `mem_usage` / `mem_usage_gradient`, `disk_usage`, `disk_io`, `gpu_load`, `gpu_vram`, `thermal` / `thermal_gradient`, `network_load` (with `network_load_recv` / `network_load_sent` and their `_gradient` variants), `battery` / `battery_gradient` (plus `battery_full`, `battery_empty`, `battery_online`, `battery_offline`, `battery_ac_state`), `branch` (with `branch_clean` / `branch_dirty`), `git_status` (with `git_status_clean` / `git_status_dirty`), `stash`, `cwd` (with `cwd:current_folder`), `user` / `superuser`, `hostname`, `date` / `time`, `player` (with `player_playing` / `player_paused` / `player_stopped` / `player_fallback`), `weather` (with `weather_conditions`, `weather_temp` / `weather_temp_gradient`).
- **Shell-status groups:** `exit_fail` / `exit_success` (the `last_status` / `last_pipe_status` pair), `email_alert`.
- **Stack / cloud groups:** `kubecontext`, `docker_containers`, `github_ci` (with `github_ci_success` / `github_ci_failure` / `github_ci_pending`), `aws_context`, `gcp_context`, and the `<tool>_rkyv_cache` / `<tool>_version` families for the MenkeTechnologies segments.
- **Structural / neutral groups:** `information:regular` (the neutral fallback several stack segments end their group chain on, so a chunk renders in *any* colorscheme), `background`, `background:divider` (the soft-divider color), and the `:divider`-suffixed variants that the renderer uses for `divider_highlight`.

The `<name>` / `<name>_gradient` pairing is the pattern to internalize: a gradient segment lists the `_gradient` group *first* and the flat group second, so a colorscheme with a defined gradient gets the shading and one

without it degrades to a flat color rather than erroring. Naming a colorscheme's groups to cover the flat names is the minimum; adding the `_gradient` variants is what turns a static prompt into a shaded one.

Appendix D – The Renderer Reference

Chapter 7 covers the render *loop*; this appendix is the per-target *encoding* reference — the exact bytes each renderer emits for a resolved `{fg, bg, attrs}` — because the encoding is where “byte-identical to upstream” is won or lost one target at a time. Every renderer under `src/ported/renderers/` shares the layout core (`do_render`, `_render_length`, `_render_segments`) in `src/ported/renderer.rs` and differs only in its `hlstyle/hl` translation. The attribute constants are shared with the colorscheme: `ATTR_BOLD = 1`, `ATTR_ITALIC = 2`, `ATTR_UNDERLINE = 4`.

D.1 The shared layout core

`do_render` (`renderer.rs:792`) is the layout algorithm, and it is worth stating as steps. It fetches the theme's segments, prepares them (`_prepare_segments`, which runs `translate_np` over contents and precomputes per-segment lengths), and — when there is no width constraint — measures once and returns. When a width is set, it computes divider widths (`compute_divider_widths`, `renderer.rs:314`), orders segments by priority (`priority` non-null, sorted *descending*, then the null-priority segments), measures with `_render_length`, and if the line is too wide runs the truncation loop: drop segments in priority order, re-measuring after each removal, until the line fits (a fast path handles lines of more than 100 segments in bulk). Finally it distributes any spacer expansion (`expand`-marked segments absorb the leftover width via the `l/r/c` align functions) and joins the rendered highlight fragments. `_render_length` (`renderer.rs:1075`) is the measurement half; `_render_segments` (`renderer.rs:1237`) is the emit half, and both decide each boundary's divider by the same rule — `divider_type = "soft" if compare_bg == seg_bg else "hard"` (`renderer.rs:1186 / 1358`). Non-printing bytes are handled by `translate_np` (`renderer.rs:158`), which maps control characters `0x00-0x1F` to caret forms (`^@...^_`) and invalid bytes to `<hh>` forms — the same `translate_np` the README names among the byte-identical behaviors.

D.2 tmux – renderers/tmux.rs

The most-used emitter. `hlstyle` (`tmux.rs:114`) builds `#[<comma-joined attrs>]` (line 158), returning `"` when the segment has no fg, bg, or attrs. Color encoding is `color_spec` (`tmux.rs:265`): the default sentinel emits `<channel>=default`; a truecolor value with `term_truecolor` on emits `<channel>=#{rgb:06x}`; otherwise `<channel>=colour<cterm>`. Attributes (`attrs_to_tmux_attrs`, `tmux.rs:25`) always emit all three slots: `attrs == None` gives `nobold,noitalics,nounderscore`, and a set flag emits the *on* form (`bold/italics/underscore`) with the other slots *off* — the explicit `no*` forms are mandatory because tmux markup is stateful and a missing `nobold` bleeds bold forward. Character translation (`tmux.rs:104`) maps `#` → `##[]`, since `#` is tmux's own escape lead-in. Example output: `#[fg=#ffffff,bg=#d78700,bold,noitalics,nounderscore]71%`.

D.3 vim – renderers/vim.rs

vim references named highlight groups rather than inlining colors. `hlstyle` (`vim.rs:442`) returns a `%#<GroupName>#` reference (line 527) and, on a cache miss, pushes a `:hi <name> ctermfg=... guifg=... ctermbg=... guibg=... cterm=... gui=...` command into a command list the plugin executes. The group name (`build_group_name`, `vim.rs:395`) is `Pl_<ctermfg>_<guifg>_<ctermbg>_<guibg>_<attrs>` with the gui values as *decimal* RGB ints or `None`. It dedups two ways: it returns `"` when the requested highlight equals

the previous one (upstream's E541 workaround) and when there is nothing to set. Attributes (`attrs_to_hi_attrs`, `vim.rs:417`) emit the plain names `bold/italic/underline` (no `no*` forms — vim's default is `NONE`), rendered into both `cterm=` and `gui=`. Character translation maps `%` → `%%` (vim format strings use `%`).

0.4 Shell — renderers/shell/

The shell base (`shell/mod.rs`) emits ANSI SGR sequences. `hlstyle` (`shell/mod.rs:339`) starts with a reset `[0]`, then appends foreground (`[38, 2, r, g, b]` truecolor or `[38, 5, cterm]`), background (`[48, ...]`), and *one* attribute code (bold `1`, else italic `3`, else underline `4` — a mutually exclusive chain, matching upstream's `elif`). The default assembly is `\x1b[<codes joined by >;m`; an `fbterm` terminal gets a different assembly (`\x1b[1;<n>] / \x1b[2;<n>]`), and `tmux/screen` passthrough wraps the sequence in `\x1bPtmux;...\x1b\ / \x1bP...\x1b\` when those flags are set. This is the one renderer whose `hlstyle` emits an explicit reset — the others do all their coloring inside `hl`.

The per-flavor renderers differ only in the *escape markers* wrapped around each SGR sequence, so the line editor can measure the visible prompt width. From the source:

Flavor	File	<code>escape_hl_start</code>	<code>escape_hl_end</code>	Char translations
zsh	<code>shell/zsh.rs</code>	<code>%{</code>	<code>%}</code>	<code>%</code> → <code>%%</code>
bash	<code>shell/bash.rs</code>	<code>\[</code>	<code>\]</code>	<code>\$</code> → <code>\\$</code> , <code>`</code> → <code>\`</code> , <code>\</code> → <code>\\</code>
readline	<code>shell/readline.rs</code>	<code>\x01</code>	<code>\x02</code>	none
ksh	<code>shell/ksh.rs</code>	<code>\x01</code>	<code>\x01</code>	prepends <code>\x01\r</code>
tcsh	<code>shell/tcsh.rs</code>	<code>%{</code>	<code>%}</code>	<code>%</code> → <code>%%</code> , <code>\</code> → <code>\\</code> , <code>^</code> → <code>\^</code> , <code>!</code> → <code>!\</code>
rcsh	<code>shell/rcsh.rs</code>	<code>\x01</code>	<code>\x02</code>	inherits readline

The base `\x01\x02` pair (SOH/STX) is what zsh's `%{...%}` and bash's `\[...\]` stand in for: each tells that shell's line editor “the bytes between these markers are non-printing,” so a prompt full of color escapes still measures at its visible width and long command lines wrap correctly. The `readline.py` / `zsh.py` files being tagged NEAR (Chapter 12) is a classifier artifact of these being constant-only shells; the constants themselves are present and correct in every flavor above.

0.5 The bar and markup renderers

The remaining renderers do all coloring in `hl` (their `hlstyle` returns `""`):

- **lemonbar** (`renderers/lemonbar.rs`): `%{F#ff<rgb>}` foreground, `%{B#ff<rgb>}` background (note the doubled `ff` alpha prefix), `%{+u}` for underline, terminated by `{F-B--u}`; `render` wraps sides with `{l}/{r}` position markers; `%` → `%%`.
- **i3bar** (`renderers/i3bar.rs`): a JSON object per segment — `{"full_text": ..., "separator": false, "separator_block_width": 0, "color": "#<rgb>", "background": "#<rgb>"}` — with a trailing comma, the shape i3bar's streaming array wants. No attribute support in the protocol.
- **pango_markup** (`renderers/pango_markup.rs`): `<span foreground="#<rgb>" background="#<rgb>" font_weight="bold" font_style="italic" underline="single">...</span>`, attributes as independent flags; `escape` does XML entity replacement (`&` → `&`; then `<`, `>`).

- `pdb` (`renderers/pdb.rs`): a `ReadlineRenderer` subclass (`\x01\x02`) whose `do_render` ASCII-folds the output (non-ASCII $\rightarrow$ `?`), mirroring Python's `.encode('ascii', 'replace')`.
- `ipython` (`renderers/ipython/`): the `prompt_toolkit` token renderers, emitting pygments token names (`Pl_...` on `prompt_toolkit` 1.x, `pl_...` on 3.x) and registering style rules rather than raw escapes — the one target where `powerline` hands *tokens* to the host rather than bytes to a terminal.

0.6 Why the encodings are a fidelity surface, not a formality

It is tempting to treat the encoders as trivia — every one of them is a small function — but they are exactly where a “port” and an “inspired-by” diverge invisibly. Forget `tmux`’s `#` $\rightarrow$ `##[ ]` and any prompt with a hash in it mangles; forget the `no*` attribute forms and everything after a bold segment stays bold; forget the shell escape markers and long command lines wrap into the prompt; use Rust’s default rounding instead of banker’s rounding and a gradient shifts one color at the midpoint. None of these is visible on a simple prompt, and all of them are visible on some real one — which is why the parity harness (Appendix F) diffs the actual bytes for each target against the live interpreter rather than eyeballing a screenshot. The encoders are short, and they are the port.

Appendix E – The Daemon Protocol and Lifecycle Reference

Chapter 4 tells the daemon’s story; this appendix is its specification — the wire format, the flags, the socket paths, the cache key, the lifecycle steps, and the signal behavior — at the level of detail someone reimplementing a compatible client or debugging a live daemon needs. Everything is grounded in `src/ported/scripts/powerline_daemon.rs` (line numbers below refer to it) and the client at `src/bin/powerline.rs`.

E.1 The wire format

Request layout (documented at `powerline_daemon.rs:252`):

```
<numargs_hex> \0 <arg> \0 <arg> ... \0 <cwd> \0 <env=val> \0 ... \0\0
```

The first field is the argument count *minus one* in hexadecimal text; then that many NUL-terminated arguments; then the NUL-terminated cwd; then the environment as NUL-terminated `KEY=VAL` entries; then a terminating double NUL. `parse_args` (`powerline_daemon.rs:249`) splits on NUL, drops empty fragments, reads `usize::from_str_radix(parts[0], 16)` for the count, slices the args, takes the cwd, and splits each remaining entry at its `first =`. Two guards: an empty request returns `None` (the daemon replies `"malformed request"`), and an empty cwd falls back to `environ["PWD"]` then the literal `"/"`. The reader, `do_read` (`:296`), accumulates until the buffer `ends_with(b"\0\0")` or a two-second timeout; a zero-length read (peer EOF) breaks the loop. The client’s marshaller (`powerline.rs:88-130`) writes the mirror image: hex count, args, cwd, `KEY=VAL` env pairs, double-NUL terminator (the byte-by-byte dump is in Chapter 4).

E.2 The EOF sentinel

`EOF = b"EOF\0\0"` (`powerline_daemon.rs:73`) — the exact bytes `E`, `O`, `F`, NUL, NUL. A client that sends exactly this triggers `do_one` to return `Some(0)`, a clean exit. This is how `powerline-daemon -k` stops a running daemon (`kill_daemon`, `:1044`), and it is a faithful port down to the byte so a Python client can stop the Rust daemon and vice versa.

E.3 Socket paths and the two namespaces

`USE_FILESYSTEM()` (`:63`) is `!cfg!(target_os = "linux")`. On non-Linux the socket is a filesystem path `/tmp/powerline-ipc-<uid>` (`:1242`); on Linux it is the abstract-namespace socket `\0powerline-ipc-<uid>` (`:1245`), which has no filesystem entry and so cannot be orphaned by a crash or swept by a `/tmp` cleaner. The pidfile `/tmp/powerline-ipc-<uid>.pid` exists only on the filesystem platforms (`:1250`); Linux skips pidfile locking. An explicit `--socket PATH` is honored *literally on every platform* (`:1234`) — a deliberate, documented divergence from upstream, which prepends `\0` on Linux even for explicit paths; the port does not, so a caller passing a real path gets a real filesystem socket. `check_existing` (`:1018`) removes a stale filesystem socket before binding and returns `Ok(None)` on `AddrInUse` (the “already running” signal).

E.4 The daemon flags

`parse_daemon_argv (:1190)` recognizes exactly five flags into `DaemonArgs { quiet, socket, kill, foreground, replace }`:

- `-q / --quiet` — suppress diagnostic prints; the flag `.zshrc/.tmux.conf` use.
- `-s / --socket <PATH>` — target a non-default socket (honored literally, E.3).
- `-k / --kill` — send the EOF sentinel to stop a running daemon; returns 0 if one was killed, 1 if none was running; errors (exit 2) if combined with `--foreground/--replace`.
- `-f / --foreground` — skip `daemonize()`, staying attached to the terminal (systemd `Type=simple` or interactive debugging).
- `-r / --replace` — loop `kill_daemon` + a 2-second sleep until the old daemon is gone, then start fresh. This is the “reload config with a full restart” path in the absence of a `SIGHUP` handler.

Unknown flags are ignored. Two upstream affordances are *absent* and named as such: there is no `-a` flag and no `SIGHUP` re-exec in the port (Chapter 4).

E.5 Startup: lock, bind, daemonize

The startup sequence, ported from upstream’s `scripts/powerline-daemon`: on filesystem platforms, take the **pidfile lock** — `lockpidfile (:1106)` opens the pidfile `O_WRONLY|O_CREAT`, takes a non-blocking `fcntl(F_SETLK, F_WRLCK)`, truncates and writes the pid, and installs the SIGTERM handler; the returned `PidLock (:1085)` is an RAI guard whose `Drop` cleans up the lockfile and closes the fd. **Bind** via `check_existing`. Then, unless `--foreground`, **daemonize** — `daemonize() (:925)`: `fork #1` (parent exits), `chdir("/")`, `setuid(0)`, `fork #2` (parent exits), and `dup2 /dev/null` onto fds 0/1/2. After it the process is a session-leaderless background server with no controlling terminal and no stray descriptors.

E.6 The accept loop

`main_loop (:854)` sets the listener non-blocking, keeps a `HashMap<RawFd, Conn>`, installs a SIGINT flag, and loops `do_one (:652)` until it returns an exit code. `do_one` builds a `pollfd` vector — the listener always watched for `POLLIN`, each connection watched for `POLLOUT` if it has a queued response else `POLLIN` — and calls `libc::poll` with a 60-second timeout. A readable listener accepts a new connection through `eintr_retry_call (:222)`, which retries on `EINTR`; a readable connection is read to the double-NUL, and if the request is `EOF` the loop returns `Some(0)`, otherwise `get_answer` renders and the response is queued for the writable pass. `poll_readable (:338)` folds `POLLHUP` into “readable” so a peer hangup reads as `EOF`, exactly as upstream’s `select` reported it — the behavior-neutral reason the port could swap `select` for `poll`.

E.7 The Powerline cache key

`PowerlineKey (:117)` is the tuple that decides which warm orchestrator serves a request, a port of upstream’s cache key. Its fields are the extension (`args.ext[0]`), the renderer module, the config/theme/path overrides (`config_override`, `theme_override`, `config_path`), and the three environment values `POWERLINE_THEME_OVERRIDES`, `POWERLINE_CONFIG_OVERRIDES`, and `POWERLINE_CONFIG_PATHS`. It derives `Hash/Eq` and keys the daemon’s orchestrator map, so two renders that share those keys share a warm `Powerline` instance and a render that differs in any of them gets its own, built once and cached. This is why the *second* render of a given kind is even cheaper than the first (Chapter 5): the first built and cached the orchestrator keyed by this tuple.

E.8 Signals and shutdown

`SIGTERM (:1067)`: `sigterm_handler (:1069)` unlinks the pidfile (its path held in a static `Mutex<Option<String>>`) and calls `libc::_exit(1)` — async-signal-safe, no allocation in the handler. `SIGINT (:898)`: sets an `AtomicBool` the main loop polls, breaking with exit code 0 (upstream's `KeyboardInterrupt → SystemExit(0)`). There is deliberately no `SIGHUP` handler. Config changes are picked up instead by the per-extension mtime-staleness reload in `src/bin/powerline-daemon.rs`, which rebuilds the config cache when the underlying files change, and by `--replace` for a full restart.

E.9 The client and its fallback

`src/bin/powerline.rs` parses only a leading `--socket PATH` (consuming `argv[1..=2]`), requires at least one remaining argument (else prints `"Must provide at least one argument."` and exits 1), marshals the wire request, connects to the socket (abstract-name connect on Linux), and copies the response to stdout. On a connect failure it does not error — it `execvps powerline-render` with `argv[0] = "powerline-render"` and the client's own arguments (`exec_render_fallback, powerline.rs:155`), so a dead daemon degrades to a slower but working direct render rather than a broken prompt. That transparent fallback is the resilience property that makes the daemon safe to depend on.

Appendix F – The Parity Harness, the Tiers, and the Bug Log

Chapter 10 argues that “faithful port” is only worth its verification; this appendix documents that verification as machinery — how the harness is built, how the tier classifier scores a file, how the drift gate works, and what *classes* of bug the harness has caught — so the claim can be inspected rather than trusted. It is grounded in `tests/parity_against_upstream.rs`, `tests/daemon_parity.rs`, `tests/ported_fn_names_match_py.rs`, and the `scripts/gen_port_*.py` classifiers.

F.1 How a parity assertion works

The harness does not compare against captured golden files; it runs the *live* Python interpreter on the vendored upstream source and diffs. The mechanism is `py_eval` (`tests/parity_against_upstream.rs:22`): it shells out to `python3`, evaluates an upstream expression (importing the real `powerline` package from `vendor/powerline`), captures stdout, and strips exactly the trailing newline `print()` adds while preserving any trailing whitespace that is part of the value. `python_available (:51)` lets an assertion *skip* rather than fail when Python is absent, so the suite degrades gracefully on a machine without the interpreter while still running fully in CI where Python is present. Each assertion computes the same input two ways — once through the Rust port, once through `py_eval` on upstream — and asserts equality.

The file is organized by upstream module, section by section: `version.py`, `lib/humanize_bytes.py`, `colorscheme.py` (the `ATTR_*` constants, `get_attrs_flag`, `pick_gradient_value`, the full 256-entry `cterm_to_hex` table), `lib/dict.py` (`mergedicts/mergedefaults`), `config.py` layout invariants, `lib/path.py`, and so on down through the renderers, segments, and bindings. The 462 assertions accreted in numbered “rounds” (the git log shows `round-5` through `round-19` and beyond), each round adding a batch targeting a specific module’s edge cases — which is the visible fossil record of the port being pinned one behavior at a time.

F.2 The daemon parity suite

`tests/daemon_parity.rs` is the sharpest test because it exercises the whole pipeline end-to-end: it sends a request over the wire protocol to the Rust daemon and diffs the rendered markup against what upstream’s `powerline-render` produces for the same config and request. It covers 45 byte-for-byte scenarios — representative configs and requests across the `tmux`, `shell`, and `vim` extensions — and it is exactly the test the migration guide’s Step 2 has a user run by hand: spin up the Rust daemon on a probe socket, send a wire request, diff against `powerline-render tmux right`. Alongside it, `daemon_e2e.rs`, `daemon_fuzz.rs`, and `daemon_stress.rs` exercise the socket lifecycle, malformed-request handling, and concurrency, so the protocol is checked for robustness as well as for byte-fidelity.

F.3 The tier classifier

`scripts/gen_port_checklist.py` assigns each upstream Python file a tier, and `DONE` — the top tier — requires two things of the corresponding Rust: a `// py:NNN` citation density of at least 0.5 (half the function body’s lines pinned to an upstream source line) and a `/// Port of <py_fn>()` doccomment naming the

ported function. Both must hold. The current output is 134 DONE / 3 NEAR / 0 PARTIAL / 0 SPARSE / 0 STUB-HEAVY — the three NEAR files being the constant-only shell classes of Chapter 12, which cannot promote a 0/0 function ratio no matter how complete the port. The classifier is re-runnable (`python3 scripts/gen_port_checklist.py > docs/PORT_CHECKLIST.md`), so the badge numbers are outputs of a tool over the actual source rather than hand-maintained claims — they cannot silently drift from reality.

F.4 The drift gate

`tests/ported_fn_names_match.py.rs` guards a failure mode the parity harness cannot see: the port claiming to have ported a function that upstream renamed or that never existed. It asserts that every `/// Port of <py_fn>()` doccomment names a real entry in `docs/powerline_py_functions.txt`, the canonical list of upstream functions. A misspelled or stale doccomment fails the gate. Where the parity harness checks *behavior* for the inputs it exercises, the drift gate checks the *declared correspondence* against the authoritative function set — the two catch disjoint classes of error, which is why the port runs both.

F.5 What classes of bug the harness has caught

The project counts 11 port bugs surfaced by the parity harness and corrected (the README points to the git log for the running record rather than freezing a canonical list). Reading the log and the edge-case tests, they fall into recognizable classes, each a place where the Rust *looked* right and produced different bytes — exactly the kind a self-authored unit test would have missed because it would have encoded the same wrong assumption:

- **Rounding:** the `pick_gradient_value` midpoint. Rust's default `f64::round` rounds half away from zero; Python 3 rounds half to even. A gradient at exactly 50% picked the wrong color until the port adopted `round_ties_even` (`py_round`).
- **Width fallback:** the `strwidth` N/Na precedence and char-vs-byte width (`987cd2437d` in the log) — measuring the line by the wrong unit throws off truncation.
- **Escaping:** the tmux `#` → `##[ ]`, bash `\[...\]`, and tcsh `%/\^/!` translations, each pinned by its own parity round (`round-10`, `round-11`) — a forgotten translation mangles exactly the prompts that contain the escape character.
- **Merge semantics:** the top-level `REMOVE` sentinel in `mergedicts` and the empty-overlay case in `mergedicts_copy` (contract tests) — a deep-merge that handled the sentinel one level too shallow would fail to delete an inherited key.
- **Size accounting:** the fusevm cache `{size}/{bytes}` reporting disk allocation (`du -sh`) versus logical sum (`1a709a30dc`) — a real divergence in an extension segment, caught the same way.
- **Glyph handling:** a glyph bug (`ae3c3a4c99`) in icon substitution.

The pattern across all of them is the argument for testing against the live interpreter: a port checked only by reading code and running its own unit tests would have shipped these, because each is a place where the code looked plausible and behaved wrong, and only a diff against the actual Python — which does not care what the porter *thought* upstream did — distinguishes the two. The eleven are eleven divergences that no longer exist, and the harness that found them runs on every push, so the twelfth gets caught the same way.

Appendix G – Performance Measurement Methodology

Chapter 5 states the performance numbers; this appendix is about *how to measure them yourself*, because a performance claim you cannot reproduce is a marketing claim. The point is not to re-derive the headline figures but to give a reader the method to confirm them on their own machine, and to separate the three distinct wins so a measurement attributes each to the right cause.

G.1 What to measure, and against what

The comparison that matters is same-command, same-config, three ways: Python `powerline` cold, `powerliners` cold (`powerline-render`, no daemon), and `powerliners` warm (the client against a running daemon). Holding the command and config fixed is what isolates *startup* as the variable — the render itself is identical work in all three, so any difference is overhead around the render, which is the whole thesis. A useful harness is a loop that runs the same `powerline shell left` invocation many times and reports the distribution, not a single timing, because the interesting quantity for an interactive prompt is the *per-invocation* cost under repetition, and a single run is dominated by noise.

```
# Warm daemon path - start the daemon, then time N client round-trips
powerline-daemon --replace -q
python3 - <<'PY'
import subprocess, time
N = 200
t0 = time.perf_counter()
for _ in range(N):
    subprocess.run(["powerline", "shell", "left", "--last-exit-code=0"],
                  stdout=subprocess.DEVNULL)
dt = (time.perf_counter() - t0) / N * 1000
print(f"warm client: {dt:.3f} ms/render")
PY
```

Swapping the invoked binary — the Python `powerline` client and daemon, then the Rust ones — while keeping this loop identical is what makes the numbers comparable. The same loop against `powerline-render` (bypassing the daemon) measures the cold native path; against the Python `powerline-render` script it measures the cold Python path.

G.2 Reading the three tiers honestly

The measurement should reproduce three tiers, and Chapter 5's decomposition says which cause owns each. **Python cold, ~50–150 ms**: essentially all interpreter startup and import, paid in full every invocation because a fresh process cannot amortize its own startup. **powerliners cold, ~30 ms**: a native binary's cold start — the kernel mapping a static executable plus the config parse — with no interpreter to boot, which is why it is several times faster than Python cold even with no warm state. **powerliners warm, sub-millisecond**: a UNIX-socket round-trip plus the microseconds of the actual render, with the config parse, adapter setup, and orchestrator all already paid once at daemon start and cached under the `PowerlineKey` (Appendix E.7).

The honest framing is that these are three separate wins, and a measurement that blurs them misattributes

the cause. The daemon deletes per-render startup — the big win, available in principle to any language. The native binary makes the client and the cold fallback cheap — the Rust win. The single static binary makes deployment trivial — the packaging win. A measurement that only compares “Python” to “Rust” collapses all three into one number and loses the ability to say which lever did what; measuring the cold and warm native paths *separately* keeps them distinct.

G.3 The caveats that keep a benchmark honest

Two things make a naive benchmark lie, and both are worth controlling for. First, the **page cache**: Python’s import cost is dominated by filesystem reads, so a cold-cache run (first invocation after boot, or on a network filesystem) is dramatically slower than a warm-cache run, and quoting one without saying which overstates or understates the gap. The ~50–150 ms range is wide precisely because it spans warm-cache to cold-cache and fast-disk to slow-disk. Second, the **live-data segments**: a config with `external_ip` or `weather` on the measured line will show latency spikes that are network round-trips, not render cost, and on powerliners specifically the on-demand sampling model (versus Python’s threaded caching, Chapter 9) means those spikes land on the render that samples rather than being hidden behind a background thread. A performance measurement should use a config of *cheap* segments (identity, time, git) to measure render overhead, and treat the expensive samplers as a separate, placement-dependent concern — which is the same advice the segment catalog gives for building a prompt in the first place.

Appendix H – A Complete Worked Example: From Config to Bytes

The chapters trace pieces; this appendix traces the *whole* path once, end to end, so the reader can see every stage of the pipeline act on a single concrete input. It reuses the worked examples of Chapters 2, 4, 6, and 7 and stitches them into one continuous render.

H.1 The inputs

A user's `~/.config/powerline` tree, trimmed to the essentials. `config.json`:

```
{ "common": { "term_truecolor": true },
  "ext": { "shell": { "colorscheme": "demo", "theme": "demo" } } }
```

`themes/shell/demo.json`:

```
{ "segments": {
  "left": [
    { "function": "powerline.segments.common.net.hostname", "priority": 10 },
    { "function": "powerline.segments.common.sys.cpu_load_percent",
      "args": { "format": "{0:.0f}%" }, "priority": 30 }
  ],
  "right": [] },
  "dividers": { "left": { "hard": " ", "soft": " " },
    "right": { "hard": " ", "soft": "f" } },
  "spaces": 1 }
```

`colorschemes/shell/demo.json` and `colors.json` are the two files from Chapter 6's worked example — `gray0/white/orange` in the palette, the `cpu_load_percent_gradient` gradient, and the `hostname`, `cpu_load_percent_gradient`, and `cpu_load_percent` groups.

H.2 Stage 1 – the request arrives

The shell's `precmd` runs `powerline shell left --last-exit-code=0`. The client marshals `argv + cwd + env` into the NUL-delimited wire blob (Appendix E.1) and writes it to the per-UID socket. The daemon's `do_read` accumulates to the double NUL, `parse_args` reconstructs `(["shell", "left", "--last-exit-code=0"], {...env...}, "/home/j")`, and `get_answer` builds the `PowerlineKey` — `ext = "shell"`, the overrides empty — and finds (or builds and caches) the warm orchestrator for that key.

H.3 Stage 2 – config resolution

The orchestrator was built once. `get_config_paths` produced the search roots (user tree, then the bundled defaults under `config_files`); `_find_config_files` located `config.json`, `themes/shell/demo.json`, `colorschemes/shell/demo.json`, and `colors.json` across them; `load_json_config` parsed each; and `mergedicts` deep-merged the user files over the bundled defaults. `term_truecolor` resolved to `true`. The `Colorscheme` was constructed from the merged colorscheme and palette, deriving `gray0`'s hex from `cterm_`

`to_hex` and loading the gradient's two parallel lists. The `Theme` holds the two left segments, the divider glyphs, and `spaces = 1`. None of this re-runs on this render — it is warm state.

H.4 Stage 3 — segment resolution

`Theme::get_segments` iterates the left side. For each segment, `process_segment` runs the adapter via the registry. `hostname` reads the environment and returns `{contents: "jbox", highlight_groups: ["hostname"]}`. `cpu_load_percent` samples CPU (say 71%), formats "71%" through the `{0:.0f}%` arg, and returns `{contents: "71%", highlight_groups: ["cpu_load_percent_gradient", "cpu_load_percent"], gradient_level: 71.0}`. Each segment's groups go to `get_highlighting`: `hostname` resolves to its flat colors (white on gray0); `cpu_load_percent_gradient` resolves through the gradient at level 71 to `[136, 0xd78700]` for both fg and bg (Chapter 6's trace). Both segments are now `{contents, highlight: {fg, bg, attrs}}` records in `parsed_segments`, carrying their `priority` values (10 and 30).

H.5 Stage 4 — layout

`do_render` measures the two-segment line with `_render_length`. There is no width constraint from a shell prompt, so no truncation runs — but the divider decision still fires per boundary: the two backgrounds differ (gray0 vs the gradient orange), so the boundary is a **hard** divider. `_render_segments` emits each segment's contents wrapped in one space of padding (`spaces = 1`), with the hard chevron between them drawn fg = gray0's bg on bg = orange.

H.6 Stage 5 — encoding and reply

The shell renderer's `hlstyle` turns each resolved highlight into an ANSI SGR sequence wrapped in the flavor's escape markers (`%{...%}` for zsh). With `term_truecolor` true, colors emit as `[38;2;r;g;b]` / `[48;2;r;g;b]`; the bold attribute on the gradient segment emits `1`. The finished byte string — non-printing escapes wrapped so zle measures the prompt correctly — is written back over the socket. The client copies it to stdout, and zsh assigns it to `PS1`. Total time on the warm path: a socket round-trip plus microseconds of render. The same two lines of theme JSON that a Python powerline user wrote years ago produced the same bytes, in under a millisecond, from a process that never paid a startup cost to do it. That is the whole of powerliners, traced once from keystroke to cell.

Appendix I – The Bindings Cookbook

Chapter 8 explains *how* each binding works; this appendix is the copy-paste reference — the exact lines that wire each target to the daemon, what each invocation carries, and the troubleshooting table for when a prompt misbehaves. Because the wire protocol is upstream's, these are the same lines a Python powerline user already has; the only thing that changes in a migration is which `powerline/powerline-daemon` resolves in `$PATH`.

I.1 tmux

Two lines in `~/.tmux.conf`:

```
run-shell "powerline-config tmux setup"
run-shell -b "powerline-daemon -q &>/dev/null || exit 0"
```

The first invokes `powerline-config tmux setup`, which is `setenv` then `source` (`src/ported/commands/config.rs`): it sets the `_POWERLINE_*` tmux environment variables and sources the version-appropriate conf, pointing tmux's `status-left`, `status-right`, and window-status formats at the `powerline` client. The conf files are embedded in the binary — `powerline-base.conf` plus seven version-specific variants — and extracted to `$XDG_CACHE_HOME/powerliners/tmux/` on first call, so the command works identically however the binary was installed. The second line starts the daemon in the background so the client's per-tick, per-window invocations hit a warm socket. Confirm the swap took by checking that the running daemon is the Rust binary: `ps -p $(cat /tmp/powerline-ipc-$(UID).pid)` on a filesystem platform, or `ss -x | grep powerline-ipc` on Linux (the abstract socket has no file).

I.2 zsh

A `precmd` hook rebuilds the prompt before each render:

```
powerline-daemon -q          # start the warm daemon once, at login

function powerline_precmd() {
    PS1="$(powerline shell left -r zsh_prompt --last-exit-code=$?
    ~ --last-pipe-status="$pipestatus" --jobnum=%j)"
    RPROMPT="$(powerline shell right -r zsh_prompt --last-exit-code=$?)"
}
function powerline_setup() {
    autoload -Uz add-zsh-hook
    add-zsh-hook precmd powerline_precmd
}
powerline_setup
```

The invocation carries the shell-local facts the daemon cannot see: `--last-exit-code=$?` feeds `last_status`, `--last-pipe-status="$pipestatus"` feeds `last_pipe_status`, and `--jobnum` feeds `jobnum`. The renderer flavor `-r zsh_prompt` selects the zsh shell renderer, whose `%{...%}` escape markers let zle measure the prompt width correctly. `RPROMPT` gets the same treatment; because zsh redraws the right prompt on interactive events, keeping the daemon warm is what keeps those redraws from each booting an interpreter.

I.3 bash

The same idea through `PROMPT_COMMAND`:

```
powerline-daemon -q
_update_ps1() { PS1="$(powerline shell left -r bash_prompt --last-exit-code=$?)"; }
if [ "$TERM" != "linux" ]; then PROMPT_COMMAND="_update_ps1; $PROMPT_COMMAND"; fi
```

The bash-specific wrinkle is the readline escape markers: the `bash_prompt` renderer wraps non-printing bytes in `\[...\]` (Appendix D.4) so readline computes the visible width, which is what keeps long command lines from wrapping into the prompt or corrupting history navigation. `powerline-config shell command` prints the preferred binding command for the current shell, and `powerline-config shell uses tmux / uses prompt` exit 0 when those components are enabled — the hooks a more elaborate rc file can branch on.

I.4 vim / neovim

One line in `.vimrc`:

```
execute 'source' trim(system('powerline-config vim source-path'))
```

`powerline-config vim source-path` extracts the embedded `powerline.vim` to `~/.cache/powerliners/vim/powerline.vim` and prints its path; there is no `+python3` requirement, and the plugin uses vim8-compatible syntax so it runs on vim 7.4+, vim 9, and neovim. The plugin sets `laststatus=2` and wires an `augroup powerliners` of autocmds — `VimEnter` for the first render, `WinEnter/BufWinEnter/BufEnter/TabEnter` for context switches, `ModeChanged` on modern vim, and a legacy `CursorMoved/CursorMovedI` fallback for ancient vim. Each fires `system('powerline vim left ...')`, which is a microsecond socket round-trip *with the daemon running* and a ~30 ms cold `powerline-render` without it — hence the recommendation to pre-start the daemon for vim. The per-request keys match upstream's `powerline.bindings.vim: mode (n/i/v/R)`, `bufnr (bufnr('%'))`, `winnr (winnr())`, and `buf_name (expand('%:p'))`, omitted when empty). The renderer emits vim's `##HighlightGroup#` markup and the plugin assigns it to `&statusline`.

I.5 ipython / REPL

The REPL binding is a deliberate non-port (Chapter 8): rather than embedding in the interpreter as upstream's Python provider does, powerliners renders REPL prompts through the external client hook, the same mechanism as a shell prompt. The result is the same colored prompt; the mechanism is external, which keeps the single-static-binary discipline intact.

I.6 Troubleshooting

Distilled from the README's troubleshooting table:

Symptom	Cause / fix
empty statusline in vim	<code>laststatus=2</code> was stomped later in <code>.vimrc</code> ; re-set it after the source line
garbled escape codes	terminal lacks truecolor; ensure <code>\$TERM=xterm-256color</code> so the cterm fallback path is used

Symptom	Cause / fix
colors don't match terminal	colorscheme JSON missing your palette; copy the relevant <code>colorschemes/<ext>/default.json</code> into your user tree
<code>write() to daemon failed</code>	daemon isn't running; <code>powerline-daemon -q</code> (the client would otherwise fall back to <code>powerline-render</code>)
refresh stutters on every keystroke in vim	the legacy <code>CursorMoved</code> fallback is firing (vim < 8.2.2871); upgrade vim or disable that autocmd
exit code never shows	the binding isn't forwarding <code>--last-exit-code=\$?</code> , so <code>last_status</code> has nothing to read

Appendix J — The Custom-Segment Cookbook

Chapter 3's `exec` section states the mechanism; this appendix is the working reference for building custom segments, with complete scripts and the exact output protocol, grounded in `src/extensions/exec_segment.rs`.

J.1 Pattern A — the explicit `exec` adapter

Spell the command out inline in the theme:

```
{ "function": "exec",
  "args": {
    "command": "/usr/local/bin/cpu_temp.sh",
    "args": ["--unit", "C"],
    "format": "%s°C",
    "highlight_groups": ["exec"]
  } }
```

The daemon spawns `command` with the positional `args` (no shell), reads stdout, and wraps the result. Because of the highlight-group caveat (J.4), name the colorscheme group `"exec"` — the function name — unless you take control of the groups via JSON-array output.

J.2 Pattern B — dotted-path filesystem dispatch

Reference the segment by dotted path and drop an executable under the config tree:

```
{ "function": "myseg.cpu_temp" }
```

```
mkdir -p ~/.config/powerline/segments/myseg
cat > ~/.config/powerline/segments/myseg/cpu_temp.sh <<'EOF'
#!/bin/sh
echo "${osx-cpu-temp | sed 's/°C//'}"
EOF
chmod +x ~/.config/powerline/segments/myseg/cpu_temp.sh
```

`resolve_dotted_path` (`exec_segment.rs:45`) turns `myseg.cpu_temp` into `<config_path>/segments/myseg/cpu_temp.{sh,py,rb,pl,lua,js,<none>}` — the `SCRIPT_EXTENSIONS` order at `exec_segment.rs:31`, first match wins, `sh` beating `py`, the empty extension matching a bare executable — searched across the *full config-path cascade*, so a script in any config root on the cascade becomes available. Pattern B is the tidier of the two because its natural function name (`cpu_temp`) is also a sensible colorscheme group name.

J.3 The output protocol

Both patterns inspect stdout at the first non-whitespace byte. If it is `[`, stdout is parsed as a JSON array of segment dicts and used verbatim — full control over `highlight_groups`, `gradient_level`, `divider_highlight_group`, and multi-chunk output. Anything else is treated as plain text and wrapped as one chunk with the `format` template applied (`%s` → trimmed stdout, `%%` → literal `%`). So a two-line shell script becomes a segment, and a richer script that prints JSON becomes a fully-styled multi-chunk one:

```
#!/usr/bin/env python3
import json, psutil
print(json.dumps([
    {"contents": "[] ", "highlight_groups": ["thermal_icon"]},
    {"contents": f"{psutil.cpu_percent():.0f}%",
      "highlight_groups": ["thermal_gradient", "background"],
      "gradient_level": psutil.cpu_percent()}
]))
```

The JSON path is how a script drives its own gradient and multi-chunk coloring — the plain-text path is for the common “echo a number” case.

J.4 The highlight-group caveat

A documented Rust-port divergence: `gen_segment_getter` (`src/ported/segment.rs:1011`) uses a function segment’s resolved `function_name` as its *only* highlight group, ignoring a theme’s `highlight_groups` override (upstream respects it). The practical consequences: for Pattern A, define a colorscheme group literally named `"exec"` (or take control via JSON-array stdout that emits explicit `highlight_groups`); for Pattern B, the dotted path’s trailing component is the group name, so `myseg.cpu_temp` colors under a `"cpu_temp"` group. This is why Pattern B is the recommended shape for anything that needs custom coloring — its function name and its group name are the same string, so the caveat simply does not bite.

J.5 When to reach for a native adapter instead

The `exec` hatches cover essentially everything upstream’s `__import__`-based custom segments did, at the cost of a subprocess per render. For a segment you will use everywhere and want at full speed, the alternative is a native adapter under `src/extensions/` — a Rust closure of the `fn(args, info) -> Option<Value>` shape wired into the `ADAPTERS` table, held to the same test discipline but carved out of the port’s fn-name rules by `docs/PORT.md`. That is how the whole stack-aware and system-metrics families were added; adding another is the same pattern rather than a special case, and it is the path for a segment that a subprocess fork would make too slow for the per-keystroke prompt.

Appendix K – Listers and `segment_list` Expansion

Chapter 2 introduces the third segment kind — the `segment_list`, or lister — and this appendix documents the mechanism, because it is the part of the model most themes never touch directly but that powers the “one segment per window / per workspace” displays. It is grounded in `process_segment_lister` (`src/ported/segment.rs:578`) and the listers under `src/ported/listers/`.

K.1 The fan-out mechanism

A lister segment expands into a variable number of sub-segments discovered at render time. `process_segment_lister` takes a `lister` closure (which enumerates the things to render one segment each), a slice of `subsegments` (the template to instantiate per item), a `display_condition` predicate, and a `contents_func`. For each item the lister yields, it filters by `display_condition` and processes each subsegment against that item’s *patched* environment — the item’s data folded into a copy of `segment_info`. So a window lister yields one entry per tmux window with the window’s number and name folded in, and the subsegment template renders once per window with those values available. The count is never in the theme; it is discovered when the lister runs, which is exactly why the theme can say “render one segment per X” without knowing how many Xs there will be.

K.2 The i3 workspace and output listers

`src/ported/listers/i3wm.rs` ports the i3 listers. `output_lister` (`i3wm.rs:41`) yields one subsegment per xrandr output; `workspace_lister` (`i3wm.rs:88`) yields one per i3 workspace, setting the `workspace` and `output` keys in each item’s `segment_info` and honoring `only_show` (which workspaces to list) and `output` (restrict to one output). One honest boundary: the live i3 connection (`get_i3_connection().get_workspaces()`) depends on the i3-ipc Python bindings, which have no live Rust equivalent wired in, so the lister’s real data source returns nothing until that is connected — but the fan-out logic itself is ported and tested against a synthetic workspace list through a `workspace_lister_for` seam, so the *mechanism* is faithful even where the *data source* is stubbed. This is the port being explicit about a boundary rather than pretending a native i3-ipc client exists.

K.3 The vim tabpage and buffer listers

`src/ported/listers/vim.rs` ports the vim listers used by the tabline: `tablister` yields one subsegment per tabpage and `bufferlister` one per buffer. `tabpage_updated_segment_info` (`vim.rs:26`) is the environment-patching function — it returns a copy of `segment_info` with the tabpage-local `tabpage/tabnr/window/winnr/window_id/buffer/bufnr` keys set — so each tab’s subsegment renders with that tab’s window and buffer in scope. As with i3, the fan-out is ported and tested against synthetic tabpage/buffer lists; the live vim objects come from the vim bindings.

K.4 Why the machinery is more than most themes need

A single segment does not need any of this — `gen_segment_getter` resolves it once and `process_segment` runs it — which is why the `lister` path is a separate function most themes never invoke directly. But it is the only way to render a count that is unknown until render time, and it is a faithful port down to the generic closures and the per-item environment patching. When you see a `tmux` statusline with one colored block per window, or an `i3` bar with one block per workspace, that is `process_segment_lister` fanning a single theme entry across a runtime-discovered list — the segment model's most general case, carried in full even where its upstream data sources are native dependencies the port names rather than fakes.

Appendix L – The Five Binaries in Detail

Chapter 4 names the five binaries; this appendix documents each one's command surface — the `argparse` it implements, the subcommands it dispatches, and the upstream file it mirrors — so a reader can drive the whole suite from the shell. The manifest declares all five explicitly (`[[bin]]` entries in `Cargo.toml`) so `cargo install powerliners` ships the complete upstream-equivalent CLI.

L.1 `powerline` – the client

Mirrors `client/powerline.c`. Surface: an optional leading `--socket PATH` (which must come *before* the extension name, per the C-client convention), then the extension arguments passed straight through (`shell left, tmux right, vim left, ...`). It marshals `argv + cwd + env` to the socket and copies the response to stdout; on connect failure it `execvps powerline-render` with the same arguments (Appendix E.9). It is the binary shells, tmux, and vim actually invoke through `$POWERLINE_COMMAND`. It has no rendering logic of its own — every flag it does not consume is forwarded to whichever renderer serves the request.

L.2 `powerline-render` – the direct renderer

Mirrors `scripts/powerline-render`, and its `argparse` (`src/ported/commands/main.rs`) is the richest in the suite because it is the full render surface the client forwards to. Positional: `ext` (`nargs=1`, e.g. `shell`, `tmux`, or a `wm.*` extension) and `side` (`nargs='?'`, choices `left/right/above/aboveleft`). Flags:

- `-r, --renderer-module MODULE` — the renderer flavor (`.bash`, `.zsh`, `.tmux`, ...).
- `-w, --width INT` — the maximum width, which triggers the priority truncation of Appendix D.1.
- `--last-exit-code INT` — the previous exit code (an `int_or_sig`, so a signal name is accepted), feeding `last_status`.
- `--last-pipe-status LIST` — the space-separated pipeline exit codes, feeding `last_pipe_status`.
- `--jobnum INT` — the background-job count, feeding `jobnum`.
- `-c, --config-override KEY.KEY=VALUE` (append) — overrides into `config.json`.
- `-t, --theme-override THEME.KEY.KEY=VALUE` (append) — theme-specific overrides.
- `-p, --config-path PATH` (append) — additional config roots, prepended to the cascade.
- `-R, --renderer-arg KEY=VAL` (append) — renderer arguments (e.g. `client_id`, and the vim per-request keys), the general-purpose channel for values the segments read out of `segment_info`.

`powerline-render` runs the whole pipeline in one process — `argparse`, config resolution, segment run, encode — and is both a standalone tool and the client's cold fallback.

L.3 `powerline-daemon` – the server

Mirrors `scripts/powerline-daemon`; its five flags (`-q/-s/-k/-f/-r`) and full lifecycle are Appendix E. It is the warm process that holds the parsed config, the adapters, and the color palettes, keyed by `PowerlineKey`, and answers wire requests. In a login rc it is one line — `powerline-daemon -q` — and it then serves every shell, tmux, and vim for the session.

L.4 powerline-config — setup and dispatch

Mirrors `scripts/powerline-config`. It is the tool the tmux and vim bindings invoke. Its subcommands (`src/ported/commands/config.rs`):

- `tmux source` — source the version-specific tmux confs.
- `tmux setenv` — set the `_POWERLINE_*` tmux environment variables.
- `tmux setup` — `setenv` then `source` (what `~/.tmux.conf` calls).
- `shell command` — print the preferred shell binding command for the current shell.
- `shell uses tmux / shell uses prompt` — exit 0 if that component is enabled (a branchable predicate for rc files).
- `vim source-path` — extract the embedded `powerline.vim` and print its path.

It handles the same tmux/shell known-function dispatch upstream did in its config script, with the conf files and the vim plugin embedded in the binary (Chapter 8).

L.5 powerline-lint — the validator

Mirrors `scripts/powerline-lint`. Its argparse (`src/ported/commands/lint.rs`) is small: `-p/--config-path PATH` (append; the roots to check, in order) and `-d/--debug`. Behind it sits the full check pipeline under `src/ported/lint/` — a `markedjson` loader that preserves source positions, the `spec.rs` structural checks, `checks.rs`, `selfcheck.rs`, `inspect.rs`, and `context.rs` — which flags unknown segment names, missing color references, and malformed `before/after` strings *statically*, with source positions, before they become a broken prompt. It is the tool a migrating user runs first (Appendix M, step 2) to catch the errors that `get_highlighting`'s “groups not found” failure would otherwise raise at render time.

Appendix M – The Migration and Roll-back Runbook

Chapter 11 argues the migration is reversible; this appendix is the runbook — the exact sequence, with the parity verification against your *own* config that makes the swap safe. The governing property is that the migration touches only which binary resolves in `$PATH`: no config edits, no `.tmux.conf` edits, no shell-rc edits, because the wire protocol, socket path, and config tree are all unchanged.

M.1 Install

```
brew tap MenkeTechnologies/menketech && brew install powerliners
# or: cargo install powerliners
# or, from source:
cargo build --release --locked \
  --bin powerline --bin powerline-daemon \
  --bin powerline-config --bin powerline-render --bin powerline-lint
```

All three paths produce the same five binaries; the `--locked` flag on the source build pins the exact `Cargo.lock` dependency versions, the reproducibility discipline for a build you intend to trust in place of a working tool.

M.2 Verify parity against your own config – before changing anything

This is the step that makes the migration safe rather than a leap of faith. Run the Rust daemon on a *throwaway* socket and diff its output against the tool you are replacing, for your actual config:

```
# lint your config first – catch authoring errors statically
powerline-lint -p ~/.config/powerline

# spin up the Rust daemon on a probe socket that does not touch the real one
powerline-daemon --replace -q --socket /tmp/powerliners-probe

# render the same request both ways and diff the bytes
diff <(powerline --socket /tmp/powerliners-probe tmux right) \
  <(powerline-render tmux right)
```

If they match for your segments, the port is faithful to *your* config — the same check the `daemon_parity` suite runs on 45 representative configs continuously (Appendix F.2). The deterministic parts (markup, colors, layout, config resolution) will match byte-for-byte; the only legitimate differences are the live-sampled numeric values of Chapter 9 (a `cpu_load_percent` sampled a moment apart), which is why a diff is best run on a config of cheap, deterministic segments first.

M.3 Swap

```
powerline-daemon -k                # stop the Python daemon (sends the EOF
^ sentinel)
ln -sf "$(command -v powerline-daemon)" ~/bin/  # put the Rust daemon ahead in $PATH
# restart tmux so it re-runs its setup and starts the new daemon
tmux kill-server; tmux new-session
# confirm the running daemon is the Rust binary
ps -p "$(cat /tmp/powerline-ipc-$$UID.pid)"    # or: ss -x | grep powerline-ipc (Linux)
```

Because the Python C client speaks the same wire protocol, you can even leave it in place and swap only the daemon — the client does not know or care which language the daemon behind the socket is written in.

M.4 Rollback

Symmetric and complete:

```
powerline-daemon -k                # stop the Rust daemon
rm ~/bin/powerline-daemon          # the symlink; powerline-daemon re-resolves to the Python
^ script
powerline-daemon -q                # start Python again
```

Nothing but the symlink changed, so there is nothing else to undo — the config, the tmux config, and the shell rc are all untouched, and the Python daemon comes back to exactly the state it was in. A migration you can try in a minute and reverse in another is one people will actually attempt, and the reversibility is the direct consequence of the port changing the implementation while preserving every interface.

Appendix N — The Config Cascade, Worked

Appendix B gives the cascade's rules; this appendix runs them once on a concrete three-file overlap, because deep-merge semantics are far clearer traced than described. Suppose two roots are on the search path: the bundled defaults (searched first, via `config_files`) and the user tree (searched last).

N.1 The two files

Bundled `themes/tmux/default.json` (trimmed):

```
{ "segments": {
  "left": [ { "function": "hostname" }, { "function": "branch" } ],
  "right": [ { "function": "date", "args": { "format": "%H:%M" } } ],
  "spaces": 1,
  "dividers": { "left": { "hard": "", "soft": "" } } }
```

User `themes/tmux/default.json`:

```
{ "segments": {
  "right": [ { "function": "date", "args": { "format": "%H:%M:%S" } } ],
  "spaces": 2 }
```

N.2 The merge, key by key

`_find_config_files` collects *both* files (not the first only), and `mergedicts` merges the user file *over* the bundled one, recursing on object collisions:

- `segments` — both sides are objects, so recurse. `segments.left` exists only in the bundled file, so it survives unchanged (`hostname`, `branch`). `segments.right` exists on both; it is a *list*, not an object, so the merge does not recurse into it — the user's list *replaces* the bundled list wholesale. The right side is now the user's single `date` segment with format `%H:%M:%S`.
- `spaces` — a scalar on both; the user's `2` wins.
- `dividers` — exists only in the bundled file, so it survives (`" "/" "`).

The effective theme is: left side `hostname` + `branch` (from defaults), right side `date %H:%M:%S` (from the user), `spaces` `2` (user), dividers from defaults. Note the two different behaviors on `segments.right` versus `spaces`: because `right` holds a *list*, the merge could not recurse into it and the user's list replaced the default's; because `spaces` is a scalar, the user's value simply overwrote. This list-vs-object distinction is the single most common surprise when hand-editing a config, and it is faithful to upstream: to *change one segment* on a side you must restate the whole side's list, whereas to change a nested `args` field you can override just that field.

N.3 Removing rather than overriding

To *delete* the bundled right side rather than replace it, the user file uses the REMOVE sentinel (Appendix B.2):

```
{ "segments": { "right": { "__powerliners_remove_this_key__": true } } }
```

During the merge with `remove` enabled, `mergedicts` sees the sentinel value on `segments.right` and *deletes* the key from the merged result, so the effective theme has no right side at all — a thing an overwrite cannot express, since overwriting with `[]` would leave an empty-but-present side. This is the mechanism that lets a user subtract from the bundled defaults, not only add to or replace them.

Appendix 0 – Glossary

A compact reference for the terms this book uses in their powerliners-specific sense.

- **Adapter** — a Rust closure of shape `fn(args, info) -> Option<Value>` that produces a segment's contents, looked up by name in the `ADAPTERS` registry. Returning `None` renders nothing.
- **ADAPTERS** — the static registry (`src/bin/shared/render_runtime.rs`) mapping a segment's dotted name to its adapter, the port's replacement for upstream's dynamic `__import__`.
- **Cascade** — the ordered set of config roots (`config_files`, system dirs, user tree) that `_find_config_files` searches and `mergedicts` deep-merges into one effective config.
- **cterm** — a 256-color terminal color index (0–255), one half of every resolved color; the other half is the 24-bit hex value.
- **Divider** — the separator between two segments: *hard* (a solid chevron, drawn when backgrounds differ) or *soft* (a thin muted separator, drawn when they match). Chosen per boundary by comparing backgrounds, never placed by the author.
- **Gradient** — a color list selected by a 0–100 level via `pick_gradient_value`, used to shade a segment (e.g. `cpu_load_percent`) from calm to hot.
- **Highlight group** — a name both a segment and the colorscheme agree on (e.g. `branch_dirty`), resolved by `get_highlighting` into `{fg, bg, attrs}`.
- **Lister / segment_list** — a segment that expands into a runtime-discovered number of sub-segments (one per window, workspace, tabpage) via `process_segment_lister`.
- **PowerlineKey** — the daemon's cache key (extension + renderer + overrides + three env vars) that selects which warm orchestrator serves a request.
- **REMOVE sentinel** — `{"__powerliners_remove_this_key__": true}`, the value that *deletes* an inherited key during a merge rather than overriding it.
- **segment_info** — the render environment (forwarded environ, cwd, and parsed client args) the daemon reconstructs from the wire request and hands to each adapter.
- **term_truecolor** — the renderer flag that chooses the hex encoding (truecolor) or the cterm index (256-color) at the last step of encoding a resolved color.
- **Tier (DONE / NEAR)** — the port classifier's grade for a file: DONE requires ≥ 0.5 citation density plus a `/// Port of ...` doccomment per function; NEAR is the constant-only-file case that cannot promote a 0/0 function ratio.
- **Wire format** — the NUL-delimited `hex(argc-1) argv cwd env \0\0` request the client sends and `parse_args` demarshals, copied exactly from upstream's C client.

Appendix P – The Truncation Algorithm, Worked

Appendix D.1 states the truncation loop in the abstract; this appendix runs it once on a narrow tmux pane, because priority truncation is the render feature most authors under-use and most misunderstand. It is grounded in `do_render` ([src/ported/renderer.rs:792](#)) and `_render_length` ([renderer.rs:1075](#)).

P.1 The setup

A tmux `status-right` with five segments, each with a `priority` the theme set (lower number = more important = dropped last), and their rendered widths including padding and dividers:

Segment	priority	width (cells)
<code>branch</code>	10	12
<code>cpu_load_percent</code>	30	8
<code>mem_usage</code>	40	14
<code>date</code>	20	7
<code>hostname</code>	50	10

The full line measures $12 + 8 + 14 + 7 + 10 = 51$ cells. Now the pane is resized so the right side has only 36 cells. `do_render` is called with `width = 36`.

P.2 The loop

`do_render` first orders the segments for dropping. It collects the segments that *have* a priority, sorted *descending* by priority value (so the least-important goes first): `hostname` (50), `mem_usage` (40), `cpu_load_percent` (30), `date` (20), `branch` (10). Segments with a null priority (none here) are never dropped. It measures the line at 51, sees $51 > 36$, and enters the drop loop, removing one segment at a time and re-measuring with `_render_length`:

1. Drop `hostname` (priority 50): width $51 - 10 = 41$. Still > 36 .
2. Drop `mem_usage` (priority 40): width $41 - 14 = 27$. Now $27 \leq 36$ — stop.

The rendered right side is `branch`, `cpu_load_percent`, `date` — the three lowest-priority-number (most important) segments — and the two least important dropped entirely. Crucially, dividers re-flow around the survivors: because the drops happen before `_render_segments` emits bytes, the boundary comparisons run over the *surviving* list, so the chevrons connect the segments that remain, not the ones that were there before. The line does not wrap, overflow, or show a truncated segment; it shows fewer segments, cleanly.

P.3 Why priority is a design tool, not a fallback

The lesson for a theme author is that `priority` is how you tell the renderer what to sacrifice when space runs short — and that a segment with *no* priority is never dropped, so the essential segments (a branch, an

exit code) can be left priority-less to pin them, while the decorative ones (a hostname, a memory readout) carry high priority numbers so they are the first to go on a narrow pane. The alternative renderers behave the same: the truncation is in the shared `do_render` core, so a narrow vim statusline or a short lemonbar drops the same segments in the same order. One honest note carried from Chapter 7: a segment's `truncate callable` (upstream's per-segment shortening hook) is not invoked by the port — it cannot call a callable off a `serde Value` — so the port drops whole segments rather than shortening them in place, a documented gap that affects only themes that relied on in-place truncation rather than whole-segment dropping.

Appendix Q – Platform Notes: macOS, Linux, and the BSDs

powerliners runs the same config and renders the same markup on every platform, but *how* it samples live data and where it binds its socket differ by OS, because the underlying facilities do. This appendix consolidates the platform-specific behavior scattered through the catalog, so a user debugging a segment on a specific OS has it in one place. All of it is grounded in the adapter sources under `src/extensions/` and `src/ported/segments/`.

Q.1 The socket namespace

The single most visible platform split. `USE_FILESYSTEM()` is `!cfg!(target_os = "linux")`, so on **Linux** the daemon binds an abstract-namespaced socket `\0powerline-ipc-<uid>` with no filesystem entry (visible only in `ss -x / lsof -U`), and skips pidfile locking; on **macOS and the BSDs** it binds a real file `/tmp/powerline-ipc-<uid>` with a matching pidfile `/tmp/powerline-ipc-<uid>.pid`. The abstract socket cannot be orphaned by a crash or swept by a `/tmp` cleaner — it vanishes exactly when the daemon exits — which is why Linux gets it by default; the filesystem platforms unlink a stale socket before binding to get the equivalent robustness.

Q.2 CPU, memory, and load

- **cpu_load_percent**: macOS parses `top -l 1 -s 0 -n 0`; Linux reads `/proc/stat` with a 100 ms delta.
- **mem_usage / mem_swap**: macOS uses `vm_stat + sysctl -n hw.memsize` (and `sysctl -n vm.swapusage` for swap); Linux reads `/proc/meminfo`.
- **system_load / uptime**: `getloadavg` and the platform uptime source, cross-platform.

The README's third documented divergence is the reason these use OS probes rather than `psutil`: Python upstream errors and skips the affected segment when `psutil` is missing, whereas powerliners resolves the same data through `top/vm_stat/netstat/pmset/uptime` and renders — so the port is *more* robust in the no-`psutil` case, a divergence in the user's favor.

Q.3 Disk, network, GPU, thermal

- **disk_usage**: `df -k -P <mount>` on both, with macOS redirecting `/` to `/System/Volumes/Data` to read the writable volume. **disk_io**: Linux `/proc/diskstats` (two 500 ms samples); macOS `ioreg -c IOBlockStorageDriver -r` with a cross-call delta.
- **network_load / internal_ip**: macOS resolves the default-route interface via `netstat (-rn -f inet, -ibn for counters)`; Linux via `ip route show default` and `/sys/class/net/<iface>/statistics/{rx,tx}_bytes`.
- **gpu_usage_percent / gpu_vram**: a vendor-dispatch chain — `nvidia-smi`, then (Linux) `rocm-smi`, then (Linux) `intel_gpu_top`, then (macOS Apple Silicon) `ioreg ... -c IOAccelerator` — so one segment works regardless of GPU vendor.

- **thermal**: Linux reads name-filtered `/sys/class/hwmon/hwmon*/temp*_input` and `fan*_input`; macOS reads `sysctl -n machdep.xcpm.cpu_thermal_level` on Intel and derives a chip temperature from `ioreg -r -n AppleSmartBattery` on Apple Silicon (where fan RPM is always unavailable, so the RPM tail of the format is dropped).

Q.4 The consequence for parity

This is the concrete source of Chapter 9's “where output may legitimately differ”: two different samplers reading at two different instants read two different numbers, so a `cpu_load_percent` on macOS `top` and one on Python's `psutil` can differ by a sampling window's worth of data. The markup framing is byte-identical — same format, same gradient, same colors — and only the number inside can differ, and only by the sampling window. Everything deterministic is identical across platforms and against upstream; the live-sampled numbers are the one place where “identical” was never physically possible, and the platform matrix above is *why*.

Appendix R – The Extension Provenance Map

`docs/PORT.md` carves `src/extensions/` out of the port's fn-name rules, and this appendix maps the provenance so the boundary between faithful port and net-new work is unambiguous at the file level. Everything under `src/ported/` is graded by the DONE tier and the parity harness against a Python original; everything under `src/extensions/` is new code held to the same test discipline but with no Python original to measure against.

R.1 Ports of third-party powerline segments

- `src/extensions/mem_usage.rs` — a 1:1 port of `mKaloer/powerline_mem_segment`, which is why its dotted keys use the third-party `powerlinemem.*` namespace. It carries `humanize_bytes` (the same K/M/G rounding as powerline's own `lib/humanize_bytes.py`) so a memory readout matches upstream byte-for-byte.

R.2 Richer or more-portable takes on upstream segments

- `git_status.rs` — a p10k-style single-chunk VCS segment, richer than the ported `branch` (which lives in `render_runtime.rs`). Both are registered; a theme picks whichever fits.
- `disk.rs`, `gpu.rs`, `thermal.rs`, `proc_count.rs` — system-metrics segments sampled through OS probes so they work without Python's `psutil` dependency (Appendix Q).

R.3 Net-new cloud and stack segments

- `k8s.rs`, `docker.rs`, `github_ci.rs`, `aws_ctx.rs`, `gcp_ctx.rs` — orchestration and cloud context, mostly pure filesystem probes of the tools' own config files with silent fall-through.
- `fusevm_jit.rs`, `zshrs_rkyv.rs`, `stryke_rkyv.rs`, `awkrs_rkyv.rs` — cache-stat segments for the sibling MenkeTechnologies runtimes, sharing a resolution chain and token surface.
- `zshrs_version.rs`, `stryke_version.rs`, `awkrs_version.rs` (with `bin_version.rs`) — version readouts with a TTL cache so the daemon does not fork per tick.
- `exec_segment.rs` — the two subprocess escape hatches that replace upstream's `__import__` custom segments; `icons.rs` — the three-tier glyph registry; `bundled_config.rs` — the compile-time embed of the default config tree; `diag_log.rs`, `bin_version.rs`, `wthr_extensions.rs` — supporting glue.

R.4 The test seam pattern

The extension segments are unit-tested through pure-functional seams that pin behavior without touching the environment — for example the cache segments' `default_path_with(get_env, path_exists)` and `default_root_with(get_env)` seams (Chapter 12), which let every precedence level of the resolution

chain be asserted with no env-var mutation and no thread-safety hazard. This is the discipline that lets net-new code be held to the same bar as the ported tree even though there is no Python original to diff against: where the ported code is verified by parity, the extensions are verified by seams that make their logic deterministic and exhaustively testable.

Appendix S — The Test Suite Map

The `tests/` directory is the executable form of every claim in this book, and this appendix maps it so a reader can find the test behind a claim. The library's own 2473 unit tests live *inside* `src/` alongside the code they test (in-module `#[cfg(test)]` blocks); the integration and parity tests live under `tests/`.

- `parity_against_upstream.rs` — the 462-assertion parity harness (Appendix F.1), organized by upstream module, diffing the port against the live Python interpreter.
- `daemon_parity.rs` — the 45 byte-for-byte end-to-end daemon scenarios (Appendix F.2).
- `daemon_e2e.rs`, `daemon_fuzz.rs`, `daemon_stress.rs` — the socket lifecycle, malformed-request handling, and concurrency for the daemon.
- `ported_fn_names_match_py.rs` — the drift gate (Appendix F.4), asserting every `/// Port of ...` doccomment names a real entry in `docs/powerline_py_functions.txt`.
- `contract_colorscheme_theme.rs`, `contract_dict_merge_gradient.rs`, `contract_round4_misc.rs` — contract tests pinning the colorscheme/theme resolution, the merge-and-gradient behavior, and assorted edges.
- `lib_edge_cases_boundary.rs`, `lib_edge_cases_round5.rs`, `lib_strwidth_fallback.rs` — the numeric and width edge cases (saturation, UTF-8 byte encoding, surrogate max, the `strwidth N/Na` fallback of Chapter 7).
- `bin_powerline_client.rs` — the client's wire marshalling and fallback.
- `vim_e2e.rs` — the vim renderer end to end.
- **The `*.sh` gate scripts** — repository-hygiene checks (final newlines, no trailing whitespace, man-page synopsis sections, README badges, docs structure) that keep the shipped artifacts consistent.

Reading the test map alongside the source is the fastest way to confirm any specific claim in this book: nearly every behavior described here has a test that pins it, and the two together — the prose that explains the design and the test that proves it holds — are what make the book's account of powerliners something you can check rather than something you have to take on faith.

Appendix T – The `segment_info` Reference

Every adapter reads the world through one map, `segment_info`, and this appendix documents what is in it, where each key comes from, and which segments read it — because “the daemon cannot see the calling shell” is only half the story; the other half is exactly what the client *does* send and where a segment finds it. It is grounded in the wire parse (`parse_args`, `powerline_daemon.rs:249`), the renderer’s environment assembly (`get_segment_info`, `renderer.rs:642`), and the per-target overrides in the tmux and vim renderers.

T.1 What the wire deposits

The client sends `argv + cwd + env` (Appendix E.1), and the daemon reconstructs three things into `segment_info`:

- **environ** — the full forwarded environment as a map. Segments that read it: `environment` (by its `variable` arg), `virtualenv` (`VIRTUAL_ENV/CONDA_DEFAULT_ENV`), `hostname`, `user`, `kubecontext` (`$KUBECONFIG`), `aws.context` / `gcp.context` (their `CLOUDSDK_*` / `AWS_*` overrides), and the cache/version segments (their `$*_HOME` / `$XDG_*` resolution chains). This is why a render reflects the *caller’s* environment, not the daemon’s stale one.
- **getcwd** — the forwarded working directory, aliased from `PWD` when the `cwd` field was empty. Segments that read it: `cwd`, `branch`, `stash`, `git_status`, `ci_status` — anything path-relative. `get_segment_info` (`renderer.rs:642`) sets `getcwd` from the request so a path-relative segment anchors on the shell’s directory rather than the daemon’s `chdir("/")`.
- **The parsed client arguments** — deposited under `args.*`: `args.jobnum` (`--jobnum`), `args.last_exit_code` (`--last-exit-code`), and `args.last_pipe_status` (`--last-pipe-status`). Segments that read them: `jobnum`, `last_status`, and `last_pipe_status` respectively. These are the shell-local facts the daemon has no other way to learn, which is the whole reason the wire protocol forwards `argv`.

T.2 The `mode` and per-target keys

`get_segment_info` also sets `mode` — the current editor/shell mode — which the colorscheme’s `mode_translations` key on, so vim’s insert mode or zsh’s vi-command mode can recolor the line. The per-target renderers patch in their own keys: the tmux renderer resolves `getcwd` from `pane_current_path` or a `TMUX_PWD_<pane_id>` environ var and sets `width_adjust` (`tmux.rs:217`); the vim renderer’s binding forwards `mode`, `bufnr`, `winnr`, and `buf_name` (Appendix I.4), which the vim segments and the tabline lists read; the pdb renderer patches `pdb`, `initial_stack_length`, and `curframe`. The general-purpose channel for anything else is `--renderer-arg KEY=VAL (-R)`, which lands in `segment_info` for a custom segment to read.

T.3 Why one map, not many arguments

Threading a single `segment_info` map through every adapter — rather than a fixed argument list — is what lets the segment signature stay `fn(args, info) -> Option<Value>` across 54 adapters with wildly

different needs: a `date` segment ignores `info` entirely, a `cwd` segment reads `getcwd`, a `last_pipe_status` segment reads `args.last_pipe_status`, and a custom `exec` segment can read whatever `--renderer-arg` deposited. The map is the extensibility seam: a new segment needs no new plumbing, only a new key to read, and the lister mechanism (Appendix K) works by *patching* this same map per item, which is how a per-workspace segment sees its workspace's data in exactly the place a normal segment would look.

Appendix U – Worked Segment Outputs

Tokens and format strings are abstract until you see the string a segment actually emits. This appendix collects concrete output examples for the segments whose formatting is most configurable, so a theme author can predict what a `format` change does. All are grounded in the adapters' `render_format` helpers.

U.1 kubecontext and hide_default

With `format "{k8s-glyph} {context}:{namespace}"` in context `prod-cluster`, namespace `kube-system`, and the default Nerd Font glyph, the segment emits `☼ prod-cluster:kube-system`. Now the subtle case: the same format in the `default` namespace with `hide_default` on (the default). `render_format_drop_ns (k8s.rs:128)` does not merely blank the `{namespace}` token — it also strips the adjacent separator, so `{context}:{namespace}` becomes `prod-cluster`, not `prod-cluster:`. This is the “stay quiet in the boring case, speak up when it could bite you” behavior: a forgotten `kube-system` shows nothing, but a switch to `production` shows `prod-cluster:production` in full. An unknown token is left intact (`render_format` only substitutes the tokens it knows), so `{context}/{namespace} v{version}` renders the literal `v{version}` verbatim — a deliberate choice so a typo degrades to visible text rather than a silent drop.

U.2 docker.containers

With `format "{docker-glyph} {running}/{total}"` and three running of eight total, the segment emits `▢ 3/8`. The available tokens are `{running}`, `{total}`, `{images}`, and `{stopped}` (the last derived as total minus running), so `"R{running} S{stopped} I{images}"` on the same state emits `R3 S5 I12`. With `show_when_zero` off (the default) and nothing running and no images, the segment returns `None` and vanishes rather than showing `0/0`.

U.3 process_count and the zombie warning

With `format "{proc-glyph} {total}"` and 214 processes, the segment emits `214`. The tokens `{total}` `{running}` `{sleeping}` `{zombie}` `{dwait}` `{stopped}` let a fuller readout — `"{total}p {running}R {zombie}Z"` — emit `214p 2R 0Z`. The interesting behavior is `warn_zombie` (default on): when a defunct process appears, the highlight chain gains `"process_count_zombie"` at the front, so a colorscheme can flip the whole segment to a warning color the moment a zombie shows up — a standing signal that something is not reaping its children.

U.4 The cache-size tokens

The cache segments distinguish *disk allocation* from *logical content size*, and the tokens make both available. For a `fusevm.jit_cache` with 40 entries totaling 1.2 MiB of content but 1.5 MiB of block allocation, `"{fusevm} {entries} {size}"` emits `☼ 40 1.5M` (the `{size}/{bytes}` tokens report `du -sh`-style disk allocation, the fix from `1a709a30dc`), while `"{logical_size}"` reports `1.2M` (the `stat`-style content sum). The distinction

matters for a cache you are watching grow: block allocation is what actually consumes the disk, which is why it is the default `{size}`.

Appendix V – The tmux Conf Files and Version Dispatch

The tmux binding is the most-used one, and its install path has a subtlety worth documenting: tmux changed its status-format syntax across versions, so a single conf cannot serve every tmux. This appendix documents how powerliners handles that, grounded in [src/ported/bindings/tmux/](#) and Chapter 8.

V.1 The eight embedded confs

The binary embeds eight tmux conf files via `include_str!`: `powerline-base.conf` (the version-independent wiring) plus seven version-gated variants — `powerline_tmux_1.7_plus.conf`, `powerline_tmux_1.8.conf`, `powerline_tmux_1.8_minus.conf`, `powerline_tmux_1.8_plus.conf`, `powerline_tmux_1.9_plus.conf`, `powerline_tmux_2.1_plus.conf`, and `powerline.conf`. `powerline-config tmux setup` extracts them to `$XDG_CACHE_HOME/powerliners/tmux/` on first call and sources the ones appropriate to the running tmux version, detected at setup time. The version split exists because tmux’s status-format capabilities changed — the `#{...}` format expansion, the per-window status format, and the `status-interval` behavior differ across the 1.7-through-2.1+ range — so the correct wiring depends on which tmux is running.

V.2 Why embedding beats install-time path baking

Upstream powerline baked config-file paths at install time, which broke when the tool was moved or installed a different way (a `pip` install, a distro package, a manual copy each put the confs somewhere different, and the baked path could be wrong). powerliners embeds the confs *in the binary* and extracts them to a cache dir keyed off `$XDG_CACHE_HOME`, so there is no compile-time path to get wrong: `powerline-config tmux setup` works identically whether the binary arrived via `brew install`, `cargo install`, or a manual `cp` into `$PATH`. This is the install-method-agnostic bundling of Chapter 8, and it is the same mechanism the vim plugin and the default config tree use (`bundled_config.rs`) — the artifacts that a user needs travel *inside* the one static binary, which is the deployment payoff of being native carried through to the last detail.

V.3 The setup handshake

`powerline-config tmux setup` is `setenv` then `source (src/ported/commands/config.rs)`: `setenv` exports the `_POWERLINE_*` environment variables tmux’s status formats reference, and `source` loads the version-appropriate conf that points `status-left/status-right/window-status` at the `powerline` client. From then on every `status-interval` tick and every window switch runs the client, which forwards to the warm daemon — the per-window multiplication (eight windows, eight renders per tick) that made the Python startup tax so painful and where the warm daemon pays off most.

Appendix W – Design-Rationale FAQ

Some questions recur, and answering them in one place captures the reasoning that is otherwise spread across the chapters. Each answer is the book's position stated compactly.

Why a daemon rather than just a faster binary? Because a fresh process cannot amortize its own startup: any model built on fresh-process-per-render pays startup every time by construction. A Rust binary that still forked per render would pay a (small) Rust startup every render — better than Python, but not sub-millisecond. The daemon is the load-bearing idea; the native language is what makes the daemon's client cheap and its cold fallback tolerable. The three wins are separate: the daemon deletes per-render startup, the native binary makes the client and cold path cheap, and the single static binary makes deployment trivial (Chapter 5).

Why keep the redundant `function_name_dup` field? Because the port is a port. Upstream's `gen_segment_getter` returns a five-tuple in which the function name appears twice, and preserving the shape — even where it is redundant — is what “faithful port” means when the goal is structure-and-byte fidelity rather than a cleaner API. Cleaning it up would be a behavioral change (Chapter 2).

Why not implement `SIGHUP` reload and the `truncate` callable? Because the port names its gaps rather than faking them. `SIGHUP` re-exec is not implemented; config changes are picked up by the mtime-staleness reload and `--replace` instead (Appendix E.8). The per-segment `truncate` callable cannot be invoked off a serde `Value`, so the port drops whole segments rather than shortening them (Appendix P). Both are documented divergences, tracked in the open, which is the same honesty the whole port is built on.

Why is `email_imap_alert` a placeholder? Because the honest state of the code is that its TLS IMAP probe is not wired in, so it emits a stub rather than a real unread count (Appendix A.9). It is present as an API-compatible stub so a theme that names it does not error, and the book says so rather than describing a capability the code does not yet have.

Is “1:1 port” marketing? No — it is a claim checked from four independent angles on every CI run: the `//py:NNN` citations (legibility and the density metric), the DONE tier (declared *and* demonstrated correspondence per function), the parity harness (behavior against the live interpreter, which found and fixed eleven real bugs), and the drift gate (declared correspondence against the canonical function set). No one of them alone suffices; together they make the phrase inspectable rather than aspirational (Chapter 10, Appendix F).

What did the project give up for “zero Python runtime”? Dynamic Python-module extensibility (replaced by the two subprocess `exec` hatches, Appendix J) and in-process REPL embedding (replaced by the external client hook, Appendix I.5). Both are niches whose result is reproduced by a different mechanism, and both are named plainly. Against them stands the elimination of the startup tax that motivated the whole project — a trade the design takes deliberately and describes honestly (Chapter 12).

Appendix X – The Divider Math, Worked

Chapter 7 states the divider rule; this appendix runs it across a full line, because the soft-versus-hard decision and the width accounting are where a “looks right” implementation and a byte-faithful one part ways. It is grounded in `_render_length` (`renderer.rs:1075`) and `_render_segments` (`renderer.rs:1237`), whose divider decision is `divider_type = "soft" if compare_bg == seg_bg else "hard"` (`renderer.rs:1186 / 1358`).

X.1 A four-segment left side

Take a left side of four segments with these resolved backgrounds (foregrounds omitted; they do not affect the divider decision):

#	segment	bg
1	mode	darkblue
2	branch	gray0
3	git_status	gray0
4	cwd	gray2

The renderer walks the boundaries left to right, comparing each segment’s background to the *next* non-literal segment’s background:

- 1→2 (darkblue vs gray0): different → **hard** divider. A solid chevron drawn `fg = darkblue` (segment 1’s bg) on `bg = gray0` (segment 2’s bg), producing the arrow that appears to point the blue block into the gray one.
- 2→3 (gray0 vs gray0): same → **soft** divider. A thin separator drawn *within* the gray block, in the muted `divider_highlight` color (the `divider_highlight` group if the segment set `divider_highlight_group`, else the segment’s own highlight). This is why two adjacent same-colored segments read as one block with a hairline between them, not two arrows.
- 3→4 (gray0 vs gray2): different → **hard** divider, `fg = gray0` on `bg = gray2`.

So the same four segments produce two hard chevrons and one soft separator, and the theme author placed *none* of them — they fell out of the coloring. Re-color `branch` to a different background and the 1→2 and 2→3 boundaries both change type automatically, which is the property that makes powerline themes composable: dividers re-flow when colors change.

X.2 The width accounting

`_render_length` must know every cell to make the truncation math (Appendix P) correct, so it sums, per segment: `outer_padding` on the first-left and last-right segments (`theme.outer_padding`), the `divider_spaces` around each divider (`theme.get_spaces()`), and the divider’s own display width from `compute_divider_widths` (which ran `strwidth` over each divider glyph once at the top of the render). A soft divider and a hard divider can have different widths — a multi-character hard chevron versus a single-character soft bar — so the accounting looks up the *specific* width for the boundary’s chosen type. The README lists `outer_padding`, `spaces`, and the multi-char/empty/single-char divider cases among the byte-identical behaviors precisely because this accounting has to match: a prompt that padded one cell differently would render off by

one and stop being a drop-in. The unglamorous sum of pads and divider widths is as much a fidelity surface as the colors are.

Appendix Y – Upstream Lineage and the Vendor Fixture

A faithful port is faithful to a *specific* upstream at a *specific* version, and this appendix pins that relationship so the parity claim has a definite referent. It is grounded in `src/ported/version.rs`, the `Cargo.toml` version, and the `vendor/` tree.

Y.1 The two version numbers

`powerliners` carries two distinct versions and it is worth not conflating them. The **product version** is the crate's own, `0.2.15` at the time of writing (`Cargo.toml`), bumped on each release of the port. The **upstream version** is the `powerline-status` version the port reproduces, `__version__ = "2.8.4"` (`src/ported/version.rs:17`), which `get_version` exposes exactly as upstream's `version.py` does — the port even reproduces the optional `b<count>` git-revision suffix and the fallback-with-`traceback` behavior when git is unavailable. So a `powerline-config` that prints a version prints upstream's `2.8.4`, because that is the version of the tool it is from a config's point of view, while the crate that shipped it is `0.2.15`.

Y.2 The vendored parity fixture

`vendor/powerline` is a checked-in copy of the upstream `powerline-status` source, and it exists for exactly one reason: the parity harness (Appendix F.1) runs the *live* Python interpreter on it. A golden-file snapshot could go stale — upstream changes, the snapshot does not, and the test now asserts old behavior — but running the real interpreter on the vendored source means the harness always compares against the actual upstream code the port claims to reproduce. This is more expensive (the tests need Python and the vendored tree, which is why `vendor/` is excluded from the *published* crate but kept in the repo, `Cargo.toml`'s `exclude` list) but it is the only way to make “byte-identical to upstream” a claim that stays true as the referent is pinned. The `vendor/powerline/client/powerline.c` in that tree is also the source the wire protocol is copied from, line-referenced in the daemon's comments (`sh:NN` citations), so the C client and the Rust client are provably the same protocol.

Y.3 What “tracking upstream” commits the port to

Pinning a version is also a maintenance contract. When upstream releases past `2.8.4`, the port's `// py:NNN` citations point maintainers at exactly which Rust lines correspond to the changed Python lines, so the port can be updated surgically rather than re-read wholesale; the drift gate (Appendix F.4) flags any ported function whose name no longer matches the canonical list; and the parity harness, run against the updated `vendor/` tree, immediately surfaces any behavior that moved. The lineage is not a one-time act of copying — it is a live correspondence, versioned on both sides, that the four verification mechanisms keep honest as the upstream moves.

Appendix Z — Man-Page Cross-Reference

This book is the continuous account; the man pages are the fast-fact reference, and the two are complementary. This appendix maps them so a reader knows which to reach for. `powerliners` ships five man pages, one per binary, and each corresponds to a region of this book.

- `powerline(1)` — the client. The extension/side arguments and `--socket`. Book: Chapter 4 (the client and its fallback), Appendix I (the bindings that invoke it), Appendix L.1.
- `powerline-daemon(1)` — the server. The five flags, the socket paths, the lifecycle. Book: Chapter 4, Appendix E (the full protocol and lifecycle), Appendix L.3.
- `powerline-render(1)` — the direct renderer. The full argparse surface (`ext`, `side`, `-r/-w/--last-exit-code/--last-pipe-status/--jobnum/-c/-t/-p/-R`). Book: Chapter 7 (the render loop), Appendix D (the encoders), Appendix L.2.
- `powerline-config(1)` — setup and dispatch. The `tmux/shell/vim` subcommands. Book: Chapter 8 (the bindings), Appendix I, Appendix L.4, Appendix V (the `tmux` confs).
- `powerline-lint(1)` — the validator. The check pipeline and its flags. Book: Chapter 9 (fidelity in failure), Appendix L.5.

For the config *format* rather than the CLI, the comprehensive man page (the `powerline(1)` family's configuration section) is the enumerated reference; this book's Chapter 6 and Appendices B and C are the explained version. The rule of thumb: reach for a man page when you need one flag, one path, or one key fast; reach for this book when you want to understand *why* the flag, the path, or the key is shaped the way it is. The man pages tell you what `powerliners` does; this book tells you what it is.

Appendix AA — Reading the Bundled Shell Theme

The best way to consolidate the whole model is to read a real config file with it, so this appendix walks the bundled `themes/shell/default.json` — the theme a fresh install renders — line by line. It is the exact file under `src/ported/config_files/themes/shell/`.

AA.1 The file

```
{ "segments": {
  "left": [
    { "function": "powerline.segments.shell.mode" },
    { "function": "powerline.segments.common.net.hostname", "priority": 10 },
    { "function": "powerline.segments.common.env.user", "priority": 30 },
    { "function": "powerline.segments.common.env.virtualenv", "priority": 50 },
    { "function": "powerline.segments.shell.cwd", "priority": 10 },
    { "function": "powerline.segments.shell.jobnum", "priority": 20 }
  ],
  "right": [
    { "function": "powerline.segments.shell.last_pipe_status", "priority": 10 },
    { "function": "powerline.segments.common.vcs.stash", "priority": 50 },
    { "function": "powerline.segments.common.vcs.branch", "priority": 40 }
  ]
}
```

AA.2 The left side, read

Six segments, each a `function` segment (the default `type`) named by its fully-qualified dotted path, which the three-tier `ADAPTERS` lookup (Chapter 3) resolves. `mode` carries no priority, so it is *never* dropped on a narrow prompt — the mode indicator is deemed essential. `hostname` (priority 10), `user` (30), `virtualenv` (50), `cwd` (10), and `jobnum` (20) all carry priorities, so on a narrow terminal they drop in descending-priority order: `virtualenv` first (50), then `user` (30), then `jobnum` (20), and the two priority-10 segments (`hostname`, `cwd`) last — a sensible sacrifice order that keeps the working directory and host visible longest while shedding the decorative context first. Each segment requests its own highlight group (`hostname` asks for `["hostname"]`, `user` for `["superuser", "user"]`, and so on), and several return `None` in the common case — `virtualenv` when no environment is active, `jobnum` when there are no background jobs — so a plain interactive shell collapses this six-segment left side to just the mode, host, and cwd.

AA.3 The right side, read

Three segments. `last_pipe_status` (priority 10) is the most important — it shows a failed pipeline stage even when the last stage succeeded — and it reads `args.last_pipe_status` off the `segment_info` the binding forwarded, rendering `None` (nothing) when the whole pipeline succeeded. `branch` (40) and `stash` (50) are the VCS pair: `branch` shows the git/hg/bzr branch and returns `None` outside a repo, `stash` shows the stash count and returns `None` at zero stashes. So in a non-repo directory after a successful command, the

entire right side is three segments all returning `None` — the right prompt is empty, which is exactly right, and cost almost nothing because three `None`-returning adapters is close to free (Chapter 3’s graceful-absence convention).

AA.4 What the theme does not say

Read what is *absent* as much as what is present. The theme names no colors — those come from the paired `colorschemes/shell/default.json`, which maps each segment’s highlight group to a `{fg, bg, attrs}` (or an alias), so swapping the colorscheme restyles this exact theme without touching it. It names no dividers here — the shell theme inherits divider glyphs from the cascade — and it places no dividers between segments, because the hard/soft decision falls out of the resolved colors at render time (Appendix X). It sets no `args` on most segments, taking each adapter’s defaults (Appendix A). This is the whole compatibility argument made concrete in one file: nine segment references and their priorities, and everything else — the colors, the encoding, the dividers, the layout — is supplied by the colorscheme, the renderer, and the adapters, each a faithful port. A user who wrote this file years ago for Python powerline gets the identical prompt from powerliners, in under a millisecond, having changed nothing. That is the book’s thesis, resting on one real theme: same prompt, no Python, sub-millisecond, proven.

Appendix AB — Common Pitfalls and Their Fixes

The troubleshooting table of Appendix I.6 covers the binding-level symptoms; this appendix covers the subtler ones — the behaviors that are *correct* per the port's design but surprise a user who does not know the design. Each is a place a chapter explains in full; here they are collected as a debugging checklist.

AB.1 A custom `exec` segment renders in the wrong color

The symptom: an `exec` segment shows its contents but ignores the `highlight_groups` you set in the theme. This is the documented `gen_segment_getter` divergence (Chapter 3, Appendix J.4): a function segment uses its resolved `function_name` as its *only* highlight group, ignoring the theme override. The fix is to name the colorscheme group after the function — "`exec`" for Pattern A, or the dotted path's trailing component for Pattern B — or to emit a JSON-array stdout that carries explicit `highlight_groups`. It is not a bug; it is a known port limitation with two clean workarounds.

AB.2 A wide-glyph theme is one cell off

The symptom: a statusline packed with CJK text or wide emoji truncates or aligns a cell short. This is the `strwidth` gap (Chapter 7, Appendix D.1): the port counts Unicode scalar values, not display cells, so a two-cell ideograph measures as one. The fix, until wide-character width lands, is to avoid relying on cell-exact alignment of wide glyphs in a width-constrained line; the overwhelming common case (ASCII, powerline glyphs, Latin text) is exact. It is a bounded, documented divergence, not a silent one.

AB.3 Everything after one segment is bold (or the wrong color)

The symptom, on tmux: a bold segment leaves every following segment bold. If you see this with the shipped renderer it is a colorscheme problem, not a renderer one — the renderer always emits the explicit `nobold`/`noitalics`/`nounder_score` off-forms (Appendix D.2) precisely to prevent it. Check that a hand-written custom `exec` segment emitting raw tmux markup is not itself setting `bold` without a matching `nobold`; the port's own segments cannot cause this, but a JSON-array `exec` script that hand-rolls markup can.

AB.4 A live-data segment shows a different number than Python did

The symptom: `cpu_load_percent` (or `network_load`, or `mem_usage`) reads a slightly different value than the Python powerline did. This is expected and is one of the three documented divergences (Chapter 9, Appendix Q.4): the port samples via OS probes on demand, Python samples via `psutil` on a different cadence, and two samplers at two instants read two numbers. The *markup* is byte-identical; only the sampled number

differs, and only by a sampling window. If the number matters exactly, it was never going to be reproducible — it is a live measurement.

AB.5 A network segment stalls the prompt

The symptom: the prompt occasionally hangs for a moment. Almost always this is an `external_ip`, `weather`, or `email_imap_alert` segment on the interactive path: the port samples on demand rather than in a background thread (Chapter 5), so a network round-trip lands on the render that triggers it. The fix is placement, not configuration — move the expensive samplers to the timer-refreshed statusline and keep the per-keystroke prompt to cheap segments (identity, time, git). Appendix A.14’s cost gradient is the guide.

AB.6 The daemon “isn’t being used” after a swap

The symptom: you swapped in the Rust daemon but the prompt feels the same speed as before. Confirm which daemon is actually serving: `ps -p $(cat /tmp/powerline-ipc-$(UID).pid)` on macOS/BSD, or `ss -x | grep powerline-ipc` on Linux (the abstract socket has no file to `ls`). If the client cannot reach the socket it silently `execvps powerline-render` (Appendix E.9) — a *working* but cold ~30 ms render — so a prompt that renders correctly but slowly is the tell that the daemon is down and the client is falling back. Restart it with `powerline-daemon --replace -q`.

Appendix AC – Quick-Reference Tables

The appendices above are the prose reference; this one is the lookup reference — three tables that collapse the whole book to something scannable. Everything here indexes material grounded earlier; this is the at-a-glance version.

AC.1 The segment quick-reference

Every built-in segment, its registry key (short form), its cost class, and where its full entry lives (Appendix A). Cost class: **cheap** (in-process / clock, safe on the keystroke path), **probe** (a subprocess or `/proc` read, sub-ms to a few ms), **network** (a network or authenticated call — statusline only).

Segment	key	cost	full entry
CPU load	<code>cpu_load_percent</code>	probe	A.1
System load	<code>system_load</code>	cheap	A.1
Uptime	<code>uptime</code>	cheap	A.1
Memory / swap	<code>mem_usage / mem_swap (+_percent)</code>	probe	A.2
Disk usage	<code>disk_usage (+_percent)</code>	probe	A.3
Disk I/O	<code>disk_io</code>	probe	A.3
GPU load / VRAM	<code>gpu_usage_percent / gpu_vram</code>	probe	A.3
Thermal	<code>thermal</code>	probe	A.4
Process count	<code>process_count</code>	probe	A.4
Git (rich)	<code>git_status</code>	probe	A.5
Branch / stash	<code>branch / stash</code>	probe	A.5
Job count	<code>jobnum</code>	cheap	A.6
Exit status	<code>last_status / last_pipe_status</code>	cheap	A.6
Cwd	<code>cwd</code>	cheap	A.7
User	<code>user</code>	cheap	A.7
Env var	<code>environment</code>	cheap	A.7
Virtualenv	<code>virtualenv</code>	cheap	A.7
Hostname	<code>hostname</code>	cheap	A.7
Internal IP	<code>internal_ip</code>	probe	A.7
External IP	<code>external_ip</code>	network	A.7
Network load	<code>network_load</code>	probe	A.7
Date / time	<code>date / fuzzy_time</code>	cheap	A.8
Battery	<code>battery</code>	probe	A.8
Players	<code>mpd/spotify/cmus/itunes/...</code>	probe	A.9
Mail	<code>email_imap_alert</code> (stub)	network	A.9
Weather	<code>weather</code>	network	A.10
Kubernetes	<code>kubecontext</code>	probe	A.11
Docker	<code>containers</code>	probe	A.11
GitHub CI	<code>ci_status</code>	network	A.11
AWS / GCP	<code>aws.context / gcp.context</code>	cheap	A.11
Cache stats	<code>fusevm.jit_cache / *.rkyv_cache</code>	cheap	A.12

Segment	key	cost	full entry
Tool versions	<code>zshrs/stryke/awkrs.version</code>	probe	A.12
Custom	<code>exec</code> (+ dotted-path dispatch)	probe	A.13

AC.2 The render-pipeline stages

The full path from wire request to bytes, with the function that owns each stage and the chapter that explains it. This is the spine the whole book hangs on.

Stage	Function / location	Chapter
Client marshals request	<code>powerline.rs:88-130</code>	4 / E
Daemon reads to <code>\0\0</code>	<code>do_read, powerline_daemon.rs:296</code>	4 / E
Parse wire request	<code>parse_args, :249</code>	4 / E
Select warm orchestrator	<code>PowerlineKey, :117</code>	5 / E
Locate config files	<code>_find_config_files, mod.rs:104</code>	6 / B
Load + merge config	<code>load_json_config + mergedicts</code>	6 / B
Build colorscheme	<code>Colorscheme::new, colorscheme.rs:109</code>	6 / C
Resolve segment getter	<code>gen_segment_getter, segment.rs:934</code>	2
Run adapter	<code>process_segment, segment.rs:692</code>	2 / A
Resolve colors	<code>get_highlighting, colorscheme.rs:292</code>	6 / C
Fan out lists	<code>process_segment_list, segment.rs:578</code>	2 / K
Measure + truncate	<code>do_render / _render_length, renderer.rs:792/1075</code>	7 / D / P
Decide dividers	<code>bg-compare, renderer.rs:1186/1358</code>	7 / X
Encode to target	<code>per-renderer hlstyle/hl</code>	7 / D
Reply / fallback	<code>socket write / execvp powerline-render</code>	4 / E

AC.3 The source-file map

Where to look in the tree for each subsystem, so a claim in this book can be traced to code.

Subsystem	Path
Segment model	<code>src/ported/segment.rs</code>
Adapter registry	<code>src/bin/shared/render_runtime.rs</code>
Ported segments	<code>src/ported/segments/</code>
Extension segments	<code>src/extensions/</code>
Colorscheme	<code>src/ported/colorscheme.rs</code>
Config cascade	<code>src/ported/mod.rs,</code> <code>src/ported/lib/{dict,config}.rs</code>
Theme grammar	<code>src/ported/theme.rs</code>
Render core	<code>src/ported/renderer.rs</code>
Renderers	<code>src/ported/renderers/</code>
Daemon	<code>src/ported/scripts/powerline_daemon.rs</code>
Client	<code>src/bin/powerline.rs</code>
Command binaries	<code>src/ported/commands/</code> , <code>src/bin/*.rs</code>
Lint	<code>src/ported/lint/</code>
Listers	<code>src/ported/listers/</code>
Bundled config	<code>src/ported/config_files/</code> , <code>src/extensions/bundled_config.rs</code>
Parity fixture	<code>vendor/powerline/</code>
Tests	<code>tests/</code> (integration/parity), in-module <code>#[cfg(test)]</code> (unit)

Read this book front to back for the *why*; use these three tables and the man pages for the *what* and the *where*. Between them, every claim about powerliners is either explained in the prose, indexed in a table, or pinned by a test — which is the whole point of a book that set out to be grounded in the program rather than in a description of it.

Appendix AD – The `mergedicts` Deep-Merge, Worked

Chapter 6 named `mergedicts` as the function that collapses the config cascade into one effective tree, and Appendix N walked the cascade at the file level. This appendix works the merge itself at the value level, because the deep-merge semantics are subtle enough that a theme author who overrides “just one field” and gets a surprise almost always tripped over a rule stated here.

AD.1 The three merge flavors

The port carries three merge functions, matching upstream’s three (`src/ported/lib/dict.rs`), and choosing the wrong mental model for which one runs is the usual source of confusion. `mergedicts(d1, d2)` mutates `d1` in place, letting `d2` win on conflict — this is the cascade merge, where a later, higher-priority file overrides an earlier one. `mergedefaults(d1, d2)` is the inverse bias: it fills gaps in `d1` from `d2` without ever overwriting a value `d1` already holds, which is how the bundled defaults supply keys a user config omits without clobbering the keys it sets. `mergedicts_copy(d1, d2)` is the non-mutating variant, returning a fresh tree so the inputs survive for a later render. The cascade uses the first; default-filling uses the second; anywhere the port must merge without scribbling on a cached dict uses the third.

AD.2 The recursion rule, stated exactly

For each key `k` in the incoming dict `d2`, `mergedicts` applies exactly one of three actions, tested in this order (`dict.rs:117`): if `d1[k]` and `d2[k]` are *both* objects, recurse and merge them key-by-key; else if `d2[k]` carries the removal sentinel and removal is enabled, delete `k` from `d1`; else overwrite `d1[k]` with `d2[k]` wholesale. The load-bearing word is *both*: the recursion happens only when both sides are objects. If the base holds an object and the override holds a scalar — or an array — the override *replaces* the object entirely; there is no merge into a scalar and, critically, **no element-wise merge of arrays**. A theme’s `segments.left` is an array, so an override that supplies its own `segments.left` replaces the whole list, it does not splice into it. This is the single most common merge surprise, and it is not a bug: arrays are opaque values to the merge, and only objects recurse.

AD.3 The REMOVE sentinel

The overwrite rule can set or replace a key but cannot, by itself, *delete* an inherited one — and sometimes a user config needs to. Upstream solved this with a REMOVE marker; the port spells it `__powerliners_remove_this_key__` and honors it in the second branch of the recursion. When an override sets a key to that sentinel and removal is enabled, the merge deletes the key from the base rather than overwriting it, so a bundled default that a user does not want is not merely shadowed but excised. This matters because shadowing and deletion differ downstream: a shadowed key still exists (with the override’s value), while a deleted key is gone, and code that iterates the effective dict sees one or zero entries accordingly. The sentinel is the only way for a cascade — which is additive by nature — to be subtractive.

AD.4 A worked merge

Take a bundled `themes/tmux/default.json` fragment as the base `d1`:

```
{ "segments": {
  "left": [ { "function": "hostname" }, { "function": "cwd" } ] },
  "segment_data": {
    "cwd": { "args": { "max_depth": 3, "shorten_home": true } } }
```

and a user `~/.config/powerline/themes/tmux/default.json` fragment as the override `d2`:

```
{ "segment_data": {
  "cwd": { "args": { "max_depth": 5 } },
  "hostname": { "args": { "only_if_ssh": "__powerliners_remove_this_key__" } } }
```

`mergedicts(d1, d2)` walks `d2`'s keys. `segment_data` is an object on both sides, so it recurses. Inside, `cwd` is an object on both sides, so it recurses again: `args` is an object on both sides, recurse once more; `max_depth` exists on both as a scalar, so `5` overwrites `3`; `shorten_home` exists only in the base, so it survives untouched. The net `cwd.args` is `{ "max_depth": 5, "shorten_home": true }` — the override changed one field and left the sibling alone, which is exactly the promise of a deep merge. Under `hostname.args`, `only_if_ssh` carries the sentinel, so if it existed in the base it would be deleted; here it did not, so the branch is a no-op. Crucially, `d2` never mentioned `segments`, so the base's `segments.left` array is untouched and both segments render — but had `d2` supplied its own `segments.left`, that whole array would have replaced the base's two-element list, because arrays do not merge. One override, three rules, and an effective config that differs from the base in precisely the fields the user named.

Appendix AE – The Divider Glyph Cascade and the Fallback Chain

Appendix X worked the divider *math* — when a boundary is hard versus soft, and how the arrow fills. This appendix covers the divider *glyphs*: which characters the renderer actually emits, the four slots they occupy, and the fallback chain that keeps a line legible on a terminal with no powerline font installed.

AE.1 The four divider slots

A powerline line has not one divider glyph but four, because “hard versus soft” crosses with “left side versus right side.” The theme (or the cascade) supplies a `dividers` object with two keys, `left` and `right`, each holding a `hard` and a `soft` glyph — four glyphs in total. The left-side hard divider is the familiar right-pointing chevron that fills with the *previous* segment's background; the right-side hard divider is its mirror, pointing left and filling with the *next* segment's background, so a right-aligned statusline's arrows point back toward the center. The soft dividers are the thin separators drawn between two segments that share a background, one per side. The renderer picks the slot by side and by the hard/soft decision of Appendix X; the theme author supplies the glyphs and never picks the slot.

AE.2 Why the fill direction matters

The hard divider is not merely a glyph — it is a glyph painted in a specific two-color scheme that produces the arrow illusion, and the direction of the fill is what makes the chevron read as “segment A flowing into segment B” rather than a stray character. On the left side, the hard divider is drawn in the *previous* segment's background as its own foreground, over the *next* segment's background, so the solid part of the chevron matches the segment behind it and the arrow appears to extrude forward. On the right side the roles swap. Getting the fill direction wrong does not crash anything — it produces a chevron that points the right way but is colored backward, a subtle wrongness that the parity harness catches because the emitted bytes differ even when the glyph is identical. The port reproduces upstream's fill assignment exactly for this reason.

AE.3 The ASCII fallback

Not every terminal has a patched powerline font, and a line full of missing-glyph boxes is worse than no dividers at all, so powerline ships — and powerliners carries — an ASCII fallback theme in which the hard divider is a plain `>` (or `<` on the right) and the soft divider is a `|`. This is not automatic detection; it is a *configuration* choice, selected by pointing the cascade at the ASCII divider set (upstream's `powerline_config` distinguishes the unicode and ascii dividers, and the bundled config files carry both). The consequence for a migrating user is reassuring: a config that already selected ASCII dividers under Python selects them identically under powerliners, because the divider glyphs are ordinary config values resolved through the same cascade as everything else, not a hardcoded renderer constant.

AE.4 Spaces, padding, and the glyph's neighbors

A divider never touches the segment text directly; there is padding between them, and the padding is part of what a faithful port must reproduce. Each segment carries a **before** and **after** string and a left/right padding count, and the renderer lays out **before** + padding + contents + padding + **after** + divider for each segment in turn. The soft divider, in particular, is drawn *inside* a shared-background run, so it sits between two segments' padding rather than at a color boundary, which is why a soft divider is a thin muted separator and a hard divider is a full-height arrow: one lives inside a color region and one lives at its edge. The width these paddings and glyphs consume is exactly what the truncation pass (Appendix P) must account for when it decides whether a line fits, which is why the divider machinery and the width machinery are two halves of one problem.

Appendix AF – The Mode Machine, End to End

Chapter 6 walked how a colorscheme’s `mode_translations` recolors a line for a given mode, and Appendix AA noted the `mode` segment on the shell theme’s left. This appendix connects the two ends: where a mode *comes from*, how it rides the wire into the render, and how it fans out into both a segment and a colorscheme override.

AF.1 What a mode is

A “mode” in powerline is a short string naming the state of the program the prompt belongs to. In vim it is the editor mode — `n` for normal, `i` for insert, `v/V/^V` for the visual family, `R` for replace, `c` for command-line — read from vim’s own `mode()` at statusline-evaluation time. In a shell the concept is thinner: the shell reports a mode only where it has one to report (a vi-mode zsh line editor distinguishes insert from command), and otherwise the mode defaults to a single neutral value. tmux has no editor modes, so its statusline renders in the default mode throughout. The mode is therefore a vim-centric feature that the other extensions carry trivially, which is why the visible payoff — “the prompt turns green in insert mode” — is almost always a vim story.

AF.2 How the mode reaches the render

The mode is supplied by the binding as part of the render request, not discovered by the renderer. A vim statusline expression evaluates `mode()` and passes the result to the client; a vi-mode shell prompt passes its editor state; the tmux and default cases pass the neutral mode. On the daemon path the mode travels in the request alongside the argv and environment (Chapter 4), so the daemon renders against the caller’s *current* mode rather than any stale notion of its own — the same freshness argument that governs the cwd and environment applies to the mode. This is why a warm daemon can serve a vim that is flipping between normal and insert dozens of times a second: each render carries the mode live, and the daemon simply resolves against whatever it was handed.

AF.3 The two consumers: the segment and the colorscheme

Once inside the render, the mode fans out to two independent consumers, and keeping them distinct clarifies a lot of theme behavior. The first is the `mode segment` (`powerline.segments.shell.mode` and its vim analogue): a function segment whose contents are the mode string, formatted through a `mode_translations` display map that turns `n` into `NORMAL`, `i` into `INSERT`, and so on. This is the visible label. The second is the colorscheme’s `mode_translations` (Chapter 6, Appendix B.5), which recolors *any* group for the current mode — including groups that have nothing to do with the mode segment. These are orthogonal: you can have the recoloring without the label (delete the `mode` segment, keep the colorscheme translations) or the label without the recoloring (keep the segment, define no translations). A theme that turns green in insert mode but shows no `NORMAL/INSERT` text is doing the second without the first, and a theme that shows the text on an unchanging background is doing the first without the second.

AF.4 Why `mode` is never truncated

Appendix AA noted a detail worth its own line here: in the bundled shell theme the `mode` segment carries *no* priority, which means the truncation pass (Appendix P) never drops it, no matter how narrow the terminal. This is deliberate and it is a correctness choice, not an aesthetic one. The mode indicator tells you whether your next keystroke is a command or literal text; a prompt that silently dropped the mode label on a narrow window to save two cells would be actively dangerous in a vi-mode editor, because you would lose the one piece of state that tells you what typing will do. So the mode is the segment most protected from truncation — priority-less, and therefore undroppable — which is the truncation algorithm being used to encode a judgment about which information a user cannot afford to lose.

Appendix AG — Signal Handling and Async-Signal-Safety in the Daemon

Chapter 4 named the daemon's `SIGTERM` and `SIGINT` handling in passing. A long-lived server's signal discipline is subtle enough — and easy enough to get subtly wrong — that it earns a full accounting, because the difference between a correct and an almost-correct signal handler is the difference between a daemon that dies cleanly and one that corrupts its own shutdown.

AG.1 The two termination signals

The daemon must exit cleanly on two signals and it treats them differently in intent though similarly in effect. `SIGTERM` is the polite kill — what `powerline-daemon --replace` sends the old daemon, what a service manager sends on stop, what a `kill` with no signal sends. The port installs `sigterm_handler` (`powerline_daemon.rs:1069`), which unlinks the pidfile and calls `_exit` so the process leaves no stale lock and no half-flushed state. `SIGINT` is the terminal interrupt — Ctrl-C on a `--foreground` daemon — and the port maps it to a clean exit with status 0, mirroring upstream's `KeyboardInterrupt` → `SystemExit(0)`: an interactive interrupt of a foreground daemon is a normal, expected way to stop it, not an error, so it exits 0 rather than with a signal-death status.

AG.2 Why the handler must be async-signal-safe

A signal handler runs at an arbitrary point in the program — potentially in the middle of a `malloc`, a `printf`, or a `HashMap` insert — and the set of functions it may legally call is tiny: only the async-signal-safe ones, which excludes essentially all allocation, all buffered I/O, and most of the standard library. A handler that calls `println!`, formats a string, or frees memory can deadlock or corrupt the heap if the signal arrived while the main thread held an allocator lock. This is why `sigterm_handler` does the two narrowest possible things — an `unlink(2)` on a fixed path and an `_exit(2)` — and nothing else. `unlink` and `_exit` are both on the async-signal-safe list; anything richer would be a latent crash that fires only under the precise timing of a signal landing inside an unsafe critical section, which is exactly the kind of bug that passes every test and fails in production once a year.

AG.3 `_exit` versus `exit`

The choice of `_exit` over the ordinary `exit` is deliberate and is the subtle part. `exit(3)` runs `atexit` handlers, flushes and closes buffered streams, and runs C++/Rust destructors registered for process teardown — all of which allocate, take locks, and touch buffered state, and none of which is async-signal-safe. `_exit(2)` does none of that: it is the raw syscall that terminates the process immediately, letting the kernel reclaim memory and close descriptors without running any userspace teardown. In a signal handler that is precisely what you want — the process is ending anyway, and the only teardown that *must* happen (removing the pidfile) is done explicitly by hand before the `_exit`. Running the full `exit` path from a signal handler would reintroduce every hazard the handler is trying to avoid, so the port terminates the hard way, on purpose.

AG.4 The pidfile and the RAI guard, reconciled

There is an apparent tension worth resolving: Chapter 4 said the pidfile is released by a `PidLock` RAI guard on `Drop`, and this appendix says the signal handler unlinks it by hand. Both are true, and they cover different exit paths. On a *normal* exit — the EOF sentinel, a clean end of the accept loop — the `PidLock` value is dropped as its scope unwinds, and its `Drop` impl removes the pidfile; this is the ordinary Rust path and it runs safely because it is not in a signal context. On a *signal* exit the stack is not unwound — `_exit` skips destructors entirely — so `Drop` never runs, and the handler must remove the pidfile itself before exiting. The two mechanisms are complementary: RAI for the paths where destructors run, an explicit `unlink` for the path where they cannot. A daemon that relied on RAI alone would leak a stale pidfile on every `SIGTERM`; one that relied on the handler alone would leak on every clean exit. The port carries both, which is why a stale pidfile from a crashed daemon is the *only* case that leaves one behind — and Chapter 4's lock-the-pidfile mechanism is exactly what keeps that case from blocking a restart.

Appendix AH — The poll Loop and What Changed from select.select

Chapter 4 noted that the daemon’s accept loop uses `libc::poll` where upstream used `select.select`, and called the substitution “documented and behavior-neutral.” Behavior-neutral is a claim, and a claim about an event loop deserves to be shown rather than asserted, because `select` and `poll` differ in ways that a careless port would leak into observable behavior.

AH.1 What the loop must do

The accept loop (`main_loop`, `powerline_daemon.rs:854`) has one job: wait until *something* is ready — either the listening socket has a new connection to accept, or one of the live client connections has bytes to read (or has hung up) — then service exactly the ready descriptors and go back to waiting. It must never busy-spin, never block on one slow client while others wait, and never miss a hangup. `select` and `poll` are two kernel interfaces for the same “tell me which of these descriptors is ready” question, and either can implement this loop; the port chose `poll` and the question is whether the choice changed anything a client can see.

AH.2 The three differences that matter

`select` and `poll` differ in three ways, and the port neutralizes each. First, `select` uses a fixed-size bitset (`fd_set`) capped at `FD_SETMAX` (typically 1024), so a daemon with many connections could in principle exceed it; `poll` takes a vector of `pollfd` with no such cap, so the port is strictly *more* robust here, not less — a difference that can only remove a failure mode, not add one. Second, `select` reports readiness as three separate sets (read, write, error) while `poll` reports per-descriptor event bitmasks; the port asks for `POLLIN` and reads back `POLLIN | POLLHUP | POLLERR`, which brings us to the third and subtlest difference. `select` folds a peer hangup into the read set — a hung-up socket reads as “readable,” and the subsequent `read` returns 0 (EOF); `poll` reports the hangup separately as `POLLHUP`. To stay behavior-identical, the port explicitly folds `POLLHUP` back into “readable” (Chapter 4), so a peer that closes its end is serviced exactly as `select` would have serviced it: treated as readable, read, and found to be at EOF. Without that fold, a hung-up connection would sit ignored in the poll set forever, a busy-loop or a leak depending on the surrounding code — the port avoids both by making `poll` report the way `select` did.

AH.3 EINTR and the retry

A signal can interrupt a blocking `poll` or `accept`, causing it to return `EINTR` rather than a result. Naively this looks like an error and a naive loop would either die or drop the event; the correct behavior is to retry the interrupted call, because `EINTR` means “a signal arrived, nothing went wrong, ask again.” The port routes its blocking calls through `eintr_retry_call` (Chapter 4), which retries on `EINTR` and propagates any other error, so a `SIGCHLD` from a reaped subprocess or a `SIGWINCH` from a foreground terminal resize does not perturb the loop. This mirrors upstream’s implicit behavior — Python’s socket calls retry `EINTR` under the hood on modern versions — and making it explicit in the port is part of why the substitution is behavior-neutral rather than merely similar: every place `select` would have transparently resumed, `poll` explicitly resumes.

AH.4 The read timeout

One more loop detail governs liveness: `do_read` does not block forever waiting for a request to finish arriving. It loops reading until the buffer `ends_with(b"\0\0")` or a two-second timeout elapses (Chapter 4), so a client that connects, sends a partial request, and stalls cannot pin a connection open indefinitely. Two seconds is generous for a local UNIX-socket write of a few kilobytes — the real request completes in microseconds — so the timeout never fires on a healthy client and fires promptly on a broken one. This is the defensive counterpart to the accept loop's fairness: the loop will not block on one slow client because `poll` multiplexes, and a single request will not block its own connection forever because `do_read` has a deadline. Together they are what let one single-threaded daemon serve every shell, tmux, and vim on the machine without a stall in one starving the rest.

Appendix AI — The `powerline-lint` Check Catalog

`powerline-lint` earns mentions throughout the book — Chapter 4 lists it among the five binaries, Chapter 9 leans on it for fidelity, Appendix L details it as a binary — but the checks it actually runs are never enumerated in one place. This appendix is that enumeration, because a linter is only as useful as the specific mistakes it catches, and knowing the catalog is how a theme author reads a lint failure.

AI.1 What the linter is for

The linter exists because a powerline config is a graph of cross-references — themes name segments, segments name highlight groups, colorschemes name colors, formatting strings name fields — and a broken reference does not fail at load time in an obvious way; it fails at *render* time, on a live prompt, often as a missing segment or a loud color error in the middle of your statusline. The linter's whole purpose is to move those failures from render time to a deliberate check you run before deploying a config (Appendix M's migration runbook runs it first, on purpose). It parses the config with a location-tracking JSON loader — the `markedjson` loader (Appendix L) — so every complaint can point at the exact file, line, and column of the offending value, not just report that *something* is wrong.

AI.2 The reference checks

The core of the catalog is reference validation, and there are four kinds. **Unknown segment names:** a theme that names a `function` the `ADAPTERS` registry does not contain is flagged, because that segment would silently render nothing (or error) live. **Missing color references:** a colorscheme group that names a color absent from `colors.json` is flagged, because it would fail the `get_highlighting` lookup at render time (Chapter 6). **Missing highlight groups:** a theme whose segments request groups that no active colorscheme defines is flagged — the loud “Highlighting groups not found” error of Chapter 6, caught statically instead of live. **Dangling aliases:** a colorscheme group aliased to another group that does not exist is flagged, because the alias chase (Appendix AF's cousin in Chapter 6) would dead-end. Each of these is a cross-reference the render path assumes holds, lifted into a check that runs before the render path ever runs.

AI.3 The formatting checks

The second family validates the `before`, `after`, and `format` strings that segments carry, because a malformed format string is a class of bug that is invisible until the exact segment renders with the exact data that trips it. The linter checks that a segment's format placeholders name fields the segment actually emits — a `{branch}` in a segment that produces no `branch` key is flagged — and that the `before/after` decorations are well-formed strings rather than, say, an accidental object. These are the checks that catch a copied-and-edited theme where a format string outlived the segment it was written for.

AI.4 Reading a lint result

A lint run either prints nothing and exits 0 — the config is clean — or prints one diagnostic per problem, each anchored to a file, line, and column by the markedjson loader, and exits non-zero. The non-zero exit is what makes the linter scriptable: the migration runbook (Appendix M) runs `powerline-lint` as a gate before swapping daemons, so a config with a dangling reference is caught by the exit code before it can become a broken prompt. The discipline the linter enforces is the same one the whole port enforces on itself — make the failure loud, early, and located, rather than silent, late, and mysterious — applied to the user's config instead of to the port's own source.

Appendix AJ – The `segment_info` Environment Contract

Adapters read the world through a single map, `segment_info`, and Appendix A stated its three main key families in one paragraph. This appendix is the full contract: every key an adapter may read, who populates it, and why the indirection exists at all. An adapter that reads anything *not* in this map is reading the daemon's stale process state rather than the caller's, which is the bug the contract exists to prevent.

AJ.1 Why adapters do not read the process environment directly

The obvious way to write a `hostname` segment is to call `gethostname`, and the obvious way to write a `cwd` segment is to call `getcwd`. Both are wrong in a daemon. The daemon is a long-lived process with its own hostname (fine, it does not move) but its own working directory, its own environment, and its own notion of the controlling terminal — all of which are stale relative to the shell that requested the render. An adapter that read `getcwd()` directly would report the daemon's directory (`/`, after the `chdir` of daemonization) on every render, for every shell, regardless of where the user actually is. So adapters do *not* read the process; they read `segment_info`, which the render path populates from the *caller's* forwarded argv, cwd, and environment (Chapter 4). The contract is the mechanism that makes a shared daemon render faithfully for a caller it does not share state with.

AJ.2 The key families

`segment_info` carries a small, fixed set of keys, grouped by source. `environ` is the caller's full environment map, demarshalled from the wire request — every `KEY=VAL` the client sent — and it is what `environment`, `virtualenv`, `kubecontext` (via `$KUBECONFIG`), and every other environment-reading segment consults. `getcwd` is the caller's working directory, the `cwd` field of the wire request (with the `$PWD`-then-`/` fallback of Chapter 4), and it anchors `cwd`, `branch`, and every path-relative segment. `args` holds the parsed client arguments that segments need: `args.jobnum` (the background-job count the shell passed), `args.last_exit_code` (the exit status of the last command), and `args.last_pipe_status` (the per-stage exit codes of the last pipeline). `mode` is the current mode (Appendix AF). Each of these is data the *caller* had and the daemon did not, forwarded explicitly precisely because the daemon could not have known it.

AJ.3 The shell-status keys in detail

The three `args` status keys deserve their own note because they are the ones a shell binding must work to supply. A shell knows its own last exit code (`$?`), its own job count (`jobs` builtin), and, in a pipeline-aware shell, its own per-stage pipe status (`${pipestatus[@]}` in zsh, `${PIPESTATUS[@]}` in bash). The binding captures these at prompt time and passes them to the client as `--last-exit-code`, `--jobnum`, and `--last-pipe-status` arguments (Appendix I's cookbook shows the exact invocations), which the daemon parses into `args`. This is why the order of operations in a shell prompt hook matters: the exit code must be captured *first*, before any other command in the hook overwrites `$?`, or the segment reports the hook's exit code instead

of the user's command's. The contract's `args` keys are only as correct as the binding's capture discipline, which is why Appendix I is careful about hook ordering.

AJ.4 The patched environment of a lister

There is one place `segment_info` is not merely forwarded but *extended*, and it is the lister fan-out of Chapter 2. When `process_segment_lister` renders one subsegment per workspace or window, it folds that item's data into a *copy* of `segment_info` before processing the subsegment — so a subsegment can read the specific workspace's name or the specific window's index through the same `segment_info` interface it uses for everything else. The contract is uniform: whether a segment is a top-level prompt segment reading the caller's cwd or a lister subsegment reading a workspace's title, it reads through `segment_info`, and the render path is responsible for populating that map with the right data for the segment's context. The adapter never knows whether its `segment_info` came straight off the wire or was patched by a lister; it just reads its keys, which is exactly the decoupling that lets one adapter contract serve both cases.

Appendix AK – Building, Installing, and Cross-Compiling the Suite

Chapter 11 covered packaging from the user’s side — `cargo install`, the five binaries, the deployment story. This appendix is the builder’s side: how the suite compiles, what it links against, and what it takes to produce the static artifact that makes the deployment story true. The claims of Chapter 11 — “a single static binary, no runtime to install” — rest on build choices worth naming.

AK.1 One crate, five binaries

The suite is one Cargo crate with five `[[bin]]` targets (Chapter 4), which is why `cargo build` produces all five in one pass and they share one compiled copy of the library code — the segment model, the colorscheme resolver, the renderer, the config cascade — rather than five copies. The shared `render_runtime.rs` (Chapter 3) is compiled once and linked into both the daemon and the direct renderer, so the daemon and `powerline-render` cannot drift in how they resolve a segment: they are the same code. This is a build-level guarantee of the parity between the warm and cold paths that Chapter 5 asserts behaviorally — they render identically because, below the transport, they *are* identical.

AK.2 What it links against

The dependency surface is deliberately thin, because every dependency is something a deployment must carry or a static link must absorb. The port leans on `libc` for the syscalls a daemon needs — `fork`, `setsid`, `poll`, `unlink`, the socket calls — and on a small set of pure-Rust crates for JSON, serialization, and the like; it does *not* link a Python interpreter, a UI toolkit, or a heavy async runtime, because none of those is needed for a synchronous, single-threaded, UNIX-socket server. The absence is the point: the fewer shared libraries the binary needs at runtime, the closer it gets to the “copy the binary and run it” deployment of Chapter 11. On most platforms the only runtime dependency is `libc` itself, which every system already has.

AK.3 The static-linking path

The strongest form of the deployment promise is a *fully* static binary — one that needs not even a system `libc`, and runs on any kernel of the right architecture regardless of the distribution’s library versions. On Linux this is the `x86_64-unknown-linux-musl` (or `aarch64-...-musl`) target: `musl libc` links statically cleanly where `glibc` resists it, producing a binary with no dynamic dependencies at all. `cargo build --release --target x86_64-unknown-linux-musl` yields five artifacts that run on a bare container, an old distribution, or a rescue environment with equal ease. On macOS the platform `libc` is not statically linkable by policy, so the “static” claim is bounded to the system libraries every mac has — which is still a single self-contained binary with no `pip` tree, just not a zero-dynamic-dependency one. The port’s thin dependency surface is what makes the `musl` path straightforward: a crate that pulled in C libraries with their own link requirements would fight the static target, and this one does not.

AK.4 Cross-compilation and the release matrix

Because the suite is pure Rust over `libc`, cross-compiling it is the ordinary Rust cross-compile — add the target with `rustup target add`, build with `--target`, and for the musl and cross-arch cases use a cross linker (the `cross` tool or a configured `CC/linker` in `.cargo/config.toml`). There is no per-platform C build step to orchestrate, no interpreter to match to the target, no native extension to rebuild — the things that make cross-compiling a Python or a C-extension project painful are simply absent. A release matrix of `{linux-musl, linux-gnu, macos-arm, macos-x86}` is four `cargo build --target` invocations producing twenty artifacts (five binaries times four targets), each a self-contained native executable. This is the build-side reason the packaging story is short: compilation produces exactly what deployment needs and nothing that deployment must then assemble.

Appendix AL – The Cost of Absence: None-Returning Segments

The book repeatedly notes that a segment can return `None` and that a `None`-returning segment is “nearly free” (Chapter 3’s graceful-absence convention, Appendix AA.3’s empty right side). “Nearly free” is a performance claim and, like the latency budget of Chapter 5, it deserves to be shown rather than asserted, because a prompt full of conditionally-absent segments is the common case and its cost is what determines whether you can afford to *configure* segments you only sometimes see.

AL.1 Why segments return nothing

A large fraction of the segment library is conditional by design: `virtualenv` shows a Python environment only when one is active, `jobnum` shows a count only when there are background jobs, `stash` shows a stash count only when the stash is non-empty, `branch` shows a branch only inside a repository, `last_pipe_status` shows a failure only when the pipeline failed. In every one of these cases the common state is *absence* — most directories are not repos, most shells have no background jobs, most pipelines succeed — so the segment’s most frequent behavior is to render nothing. A prompt is configured with more segments than it usually shows, and the ones it does not show are the ones whose condition is currently false.

AL.2 What “nothing” costs

The cost of a `None`-returning segment is the cost of the check that decided it should return nothing, and the port keeps that check cheap for the common cases precisely because they are common. `jobnum` reads an integer the binding already forwarded in `args` and compares it to zero — no syscall, a branch. `last_pipe_status` reads the forwarded pipe-status array and checks whether any stage is non-zero — no syscall, a scan of a tiny array. `virtualenv` reads an environment variable from the forwarded `environ` — no syscall, a map lookup. These are the cheap-check absences, and a prompt full of them costs, in aggregate, a handful of comparisons: effectively free, which is why the shell theme can carry six left segments and three right segments (Appendix AA) and collapse to three visible ones with no measurable penalty for the six that vanished.

AL.3 The absences that are not free

Not every absence is cheap, and the distinction is exactly the segment-placement decision of Chapter 3. `branch` returning `None` outside a repo is *not* a pure environment check — deciding whether the current directory is inside a git working tree means walking up the directory tree looking for a `.git`, which is a sequence of `stat` calls. It is fast (a few syscalls) but it is not free, and it runs on *every* render whether or not you are in a repo, because the only way to know you are not in a repo is to look. `git_status` is heavier still. The lesson the book draws repeatedly is visible here in miniature: the cheap absences (integer and environment checks) can be configured freely, but the probe absences (filesystem walks, subprocesses) cost their probe every render regardless of outcome, so a prompt tuned for the keystroke path keeps the probe segments on the timer-refreshed statusline (Appendix AB.5) even though they usually render nothing — because “usually renders nothing” and “usually costs nothing” are not the same claim, and the difference is a `stat` loop.

AL.4 Why absence is rendered, not skipped in configuration

A reasonable question is why a user configures a segment that is absent most of the time rather than adding it only when needed — and the answer is that the prompt is static configuration evaluated against dynamic state, so the *configuration* names every segment that could ever appear and the *render* decides which ones do. You cannot “add `branch` only when in a repo” because the config is written once and the repo-ness changes per directory per render; the segment must be configured unconditionally and return `None` when its condition is false. This is the whole reason the `None`-returning convention exists: it is what lets a single static theme describe a prompt whose visible shape changes continuously as you move through directories, start and finish jobs, activate environments, and succeed or fail commands. The absence is not a degenerate case of the segment model; it is the segment model's answer to configuring a prompt for states that come and go.

Appendix AM — The Reproducibility Ledger

The book is candid throughout about where powerliners' output is byte-identical to upstream and where it legitimately cannot be (Chapter 9, Appendix AB.4, Appendix Q.4). Those admissions are scattered across the chapters that raise them; this final appendix collects them into one ledger, because the precise boundary between “must match exactly” and “cannot match and should not claim to” is the book's central honesty and deserves a single place a reader can check it.

AM.1 The three tiers of reproducibility

Every byte powerliners emits falls into one of three tiers. **Structurally reproducible** output must be byte-identical to upstream and is proven so by the parity harness (Appendix F): the tmux/shell/vim markup, the #-doubling and escape encoding, the color resolution to (`cterm`, `hex`), the divider placement and fill, the truncation order, the divider math. Given the same config and the same inputs, these produce the same bytes, and a diff that finds a difference is a bug. **Input-dependent but deterministic** output depends on state the caller forwards — the cwd, the environment, the exit code, the mode — and is reproducible *given the same forwarded state*: two renders with the same `segment_info` produce the same bytes, and the parity harness controls the inputs precisely so it can assert this. **Live-sampled** output depends on a measurement taken at render time — CPU load, memory use, network counters, the current time, a weather reading — and is *not* reproducible even in principle, because two samplers reading a moving quantity at two instants read two values.

AM.2 The live-sampled set, named

Honesty requires naming the third tier exactly rather than gesturing at it, because a reader deserves to know which specific segments will differ. The live-sampled segments are the metrics and clocks: `cpu_load_percent`, `system_load`, `mem_usage/mem_swap`, `disk_usage/disk_io`, `gpu_usage_percent/gpu_vram`, `thermal`, `network_load`, `battery`, `uptime`, `date/fuzzy_time`, `external_ip` and `weather` (network reads), and the mail and player segments. For every one of these, the *markup* — the colors, the format, the gradient bucket the value lands in — is structurally reproducible, and only the *number inside the markup* is not. A parity assertion on `cpu_load_percent` therefore checks the encoding and the gradient selection against a fixed injected value, not against a live sample, because a live sample was never going to match across two processes sampling at two instants. This is not a gap in the port; it is a property of measuring a moving world, and the port marks it rather than pretending a clock reads the same twice.

AM.3 The documented divergences

Beyond the live-sampled tier there are exactly three *behavioral* divergences the book flags as known and bounded, and collecting them closes the ledger. First, the `strwidth` gap (Chapter 7, Appendix AB.2): the port measures Unicode scalar values, not display cells, so a line packed with wide CJK or emoji glyphs can be one cell off in truncation and alignment; the common case (ASCII, powerline glyphs, Latin text) is exact. Second, the `gen_segment_getter` highlight-group divergence (Chapter 3, Appendix AB.1): a `"type"`:

"function" segment uses its resolved `function_name` as its sole highlight group rather than honoring a theme's explicit `highlight_groups` override, with the two documented workarounds. Third, the daemon's config reload (Chapter 4): upstream's `SIGHUP`-driven re-exec is replaced by the per-`ext` mtime-staleness reload plus `--replace`, a lifecycle difference with no output consequence. These three are the whole of the behavioral divergence surface — everything else is either structurally identical or a live measurement — and the port names all three in the open rather than letting a user discover them as surprises.

AM.4 Why the ledger is the honesty

The reason this ledger is the right note to end the appendices on is that it *is* the book's thesis stated as a boundary. powerliners claims to be a faithful port, and a faithful-port claim is only meaningful if it is falsifiable — if there is a precise statement of what "faithful" covers and a mechanism that would catch a violation. The ledger is that statement: structurally-reproducible output is byte-checked against the live interpreter and any difference is a bug; input-dependent output is reproducible given the inputs and the harness controls them; live-sampled output is honestly excluded because it is not reproducible in physics, not merely in this implementation. A port that claimed byte-identical output including live CPU load would be lying, and a port that hid its three behavioral divergences would be hiding the places a careful user could catch it. powerliners does neither. It draws the line exactly, proves fidelity on the correct side of it, and names the exceptions on the other — which is, in the end, the difference between a program that says it is a faithful port and one that has made its faithfulness auditable. Same prompt, no Python, sub-millisecond, proven — and the word *proven* has a ledger behind it.

```
// END OF BOOK // PROMPT LOCKED, NATIVE //
```